CSSMedia User Guide



Copyright © 2026 Maxprograms

Table of Contents

Features **1**

Get started **2**
- Install CSSMedia 2
- Prepare your XML 2
- Run CSSMedia 3
- Your first stylesheet 3
- Try the samples 4

Selectors **7**
- Basic selectors and namespaces 7
- Combinators 8
- Pseudo-classes 10
- Pseudo-elements 11
- Specificity 12
- Selector reference 13

Define pages **21**
- Page setup reference 24

Page orientation, bleed and marks **27**

Named strings **28**
- Named strings reference 29

Links **31**
- Link reference 33

Footnotes **34**
- Footnote reference 37

Table of contents **40**

Text and fonts **41**
- Font selection and sizing 41
- Font resources and descriptors 45
- Font variants and features 50
- Variable and color fonts 55

Languages and hyphenation **59**
- Document language 59

Ruby and vertical text **61**

Boxes, backgrounds and borders **62**
- Box sizing, margins and padding 62
- Box dimensions 67
- Background color and images 71
- Background positioning and size 72
- Background repetition and attachment 73
- Background clipping, shorthand and blending 74

Shadows, opacity, transforms and masks **76**

Multi-column layout **77**

Flexbox and grid **78**

Images, SVG and MathML **79**

Tables **80**

Cascade and declarations **81**
- Cascade order and layers 82
- Inheritance and default values 84
- Shorthand properties 86
- Custom properties and substitution 88
- Conditional rules 89

Complete reference **94**
- Selector reference 94
- Font reference 101
- Box model properties 110
- Dimension properties 112
- Background and image reference 117
- Display and positioning 120
- Page reference 122
- Links and generated content reference 125
- Footnotes 131
- Links 132
- Named strings 133
- Cascade reference 134

Licenses **142**

Features

CSSMedia offers these main features.

- **XML format agnostic.** CSSMedia works with any well-formed XML. It does not assume anything about element names or attributes.
- **Any grammar.** XML is parsed with TypesXML, which handles any vocabulary. You can pass the grammars of your documents to CSSMedia with an OASIS Catalog.
- **CSS based.** The stylesheet is the only source of styling. Selector matching, the cascade, inheritance and value resolution follow the CSS specifications.
- **PDF for paged media.** Page sizes, margins, page breaks, named pages, headers, footers and page numbers are controlled with CSS paged media features.
- **Multilingual support.** Documents can be in any language and script, including right-to-left languages and CJK languages.
- **Hyphenation by language.** Text is hyphenated with the patterns of the language declared with `xml:lang`.
- **Fonts.** OpenType, TrueType, font collections, WOFF and WOFF 2.0 fonts, loaded from `@font-face` rules or a fonts folder, plus the 14 standard PDF fonts.
- **Shared fonts folder.** You can pass a central fonts folder to CSSMedia and reuse it from any project.
- **Generated content.** Counters, quotes, list markers, named strings, running elements, cross-references and tables of contents.
- **Links and bookmarks.** Internal and external links and PDF bookmarks.
- **Footnotes.** Any element can be a footnote. CSS moves it to the bottom of the page, numbered with a counter that you can restart on each page.
- **Images.** PNG, JPEG, JPEG 2000, GIF, BMP, TIFF and WebP images.
- **SVG and MathML.** SVG and MathML found in a document are drawn as images.

Get started

CSSMedia turns an XML file and a CSS file into a PDF.

To produce a PDF, install CSSMedia, prepare your XML and run it from the command line with your stylesheet. These topics show how:

- Install CSSMedia
- Prepare your XML
- Run CSSMedia
- Your first stylesheet
- Try the samples

Install CSSMedia

Follow these steps to build CSSMedia from source or to install it as a global command.

Procedure

1. Clone the CSSMedia project and build it with npm:

   ```
   git clone https://github.com/maxprograms-com/CSSMedia.git
   cd CSSMedia
   npm install
   npm run build
   ```

2. Check that the build works by running CSSMedia:

   ```
   node dist/cssmedia.js
   ```

   Without parameters, CSSMedia displays its help.

3. To use the `cssmedia` command from any folder, install the package globally:

   ```
   npm i -g file:cssmedia-x.x.x.tgz
   ```

   Download or build the package first, and replace `x.x.x` with its version.

Prepare your XML

CSSMedia generates a PDF from almost any well-formed XML file, provided that you give it an appropriate CSS file.

Some XML projects consist of multiple files that need pre-processing before they are ready for publishing. Projects based on DITA or DocBook need to merge all the files that make up a project into one XML file.

- If you work with DITA, use TypesDitac to process your topics and generate one combined XML file for CSSMedia.
- If you work with DocBook, TypesBook helps you merge your files into a single document ready for CSSMedia.

Run CSSMedia

Run CSSMedia from the command line with an XML file, a CSS file and the name of the PDF to write.

This command generates a PDF from your XML and your stylesheet:

```
cssmedia -x document.xml -c document.css -p document.pdf
```

If you built CSSMedia from source and did not install it globally, run `node dist/cssmedia.js` instead of `cssmedia`.

Running CSSMedia without parameters displays this help:

```
Usage: cssmedia -x <file> -c <file> -p <file> [-f <fontsFolder>] [-g <catalog>]

  -x, --xml <file>       XML file to format
  -c, --css <file>       CSS stylesheet to apply
  -p, --pdf <file>       PDF file to write
  -f, --fonts <folder>   (optional) alternative fonts folder
  -g, --catalog <file>   (optional) OASIS catalog to use when parsing XML
```

Use `-f` to pass a fonts folder that you reuse from any project. Use `-g` to pass an OASIS Catalog when your XML refers to grammars.

When something in the XML or the stylesheet cannot be used, for example an image that cannot be read, CSSMedia still writes the PDF and prints a warning for each problem after the run. If a file is missing or an error stops the run, CSSMedia prints the error and does not write the PDF.

Your first stylesheet

CSSMedia takes two files: an XML document and a CSS stylesheet. XML contains the document's elements and text. CSS contains rules that describe how selected elements should be styled.

Procedure

1. Look at the XML

   Suppose your XML contains this element:

   ```
   <chapter>
       <title>Using the device</title>
       <p>Follow these steps before starting.</p>
   </chapter>
   ```

   The names `chapter`, `title`, and `p` belong to this XML document. They are not special CSS names. You can use the names from your own XML in a stylesheet.

2. Write a rule

   A CSS rule has a selector and a declaration block:

   ```
   title {
       font-size: 18pt;
   }
   ```

The selector `title` chooses XML elements with that name. The declaration `font-size: 18pt;` sets a property to a value. The braces contain the declarations for the selected elements.

You can add more declarations to the same rule:

```
title {
    font-family: serif;
    font-size: 18pt;
    color: navy;
}
```

Each declaration has a property name, a colon, a value, and a semicolon. CSSMedia checks property values against the CSS syntax it supports. A selector only matches when it describes elements in your XML.

3. Use names from your own XML

   The element names in the examples are placeholders. If your XML uses `heading` instead of `title`, write `heading` as the selector:

   ```
   heading {
       font-size: 18pt;
   }
   ```

   CSSMedia does not infer that `heading` is a title or that `p` is a paragraph. You decide which XML names to select and which CSS properties to set.

4. Generate the PDF

   Save the XML as `document.xml` and the stylesheet as `document.css`, then run CSSMedia:

   ```
   cssmedia -x document.xml -c document.css -p document.pdf
   ```

   Open `document.pdf`. The `title` element is styled by the rule you wrote.

Try the samples

The CSSMedia repository includes sample documents, stylesheets, fonts and the PDFs they produce. Follow these steps to generate a PDF from a sample and change its stylesheet.

About this task

The `samples` folder of the repository has these samples:

Sample	Content
xml_gap	A DocBook article in English, Arabic, Japanese and Russian, formatted with one stylesheet, `xml_gap.css`.
Swordfish	The User Guide of a real product, written in DITA and merged into one XML file with TypesDitac and with the DITA Open Toolkit. The stylesheets `ditac.css`, `ditac_hyphens.css` and `dita-ot.css` format the merged files.
fonts	The Noto fonts that the stylesheets load with `@font-face` rules.

Each sample comes with the PDF that CSSMedia generated from it.

Procedure

1. Open a terminal in the `samples` folder.
2. Generate the PDF of the English article:

   ```
   cssmedia -x xml_gap/xml_gap.xml -c xml_gap.css -p xml_gap/xml_gap.pdf
   ```

   The XML files declare a DocBook DTD. Without a catalog, CSSMedia prints the warning `External subset not found: docbook/docbookx.dtd` and still generates the PDF. To avoid the warning, pass a catalog that resolves the DTD with `-g`, such as the one in TypesBook.
3. Open the PDF and compare it with the one in the repository.
4. Edit `xml_gap.css` and run the command again.

   For example, change the `font-size` of `article` or the `color` of `section > title`, and see what changes in the PDF.
5. Generate the Swordfish User Guide:

   ```
   cssmedia -x Swordfish/typesditac/swordfish.xml -c ditac.css -p Swordfish/
   typesditac/swordfish.pdf
   ```

Results

The `xml_gap.css` stylesheet uses these techniques:

Result	CSS
Local fonts	`@font-face` rules with `src: url("./fonts/NotoSans-Regular.ttf")`
Page size, margins and a footer with the document title and the page number	`@page` with `@bottom-left` and `@bottom-right`, `string-set: doc-title content()` on the title, and `counter(page)`
A different font for each language	Selectors such as `article[lang|="ar"]` and `article[lang|="ja"]`
Hyphenation in English and Russian	`hyphens: auto` on `article:lang(en)` and `article:lang(ru)`
Right-to-left text in Arabic	`direction: rtl`
Stricter line breaking in Japanese	`line-break: strict`
Links from an XML attribute	`-cssmedia-link: attr(url)` on `ulink`
An image from an XML attribute	`content: attr(fileref url)` on `imagedata`
The photo at the start side of the text, on the right in Arabic and on the left in the other languages	`float: inline-start` on `mediaobject`

Result	CSS
An arrow after each link	`ulink::after { content: " \2197"; }`
Code listings in a monospace font	`font-family: "Noto Sans Mono", monospace` with `white-space: pre`

Selectors

A selector tells CSSMedia which element or elements in the XML file a CSS rule applies to. A rule has a selector followed by braces. Inside the braces, write declarations that set property values:

```
chapter {
    color: navy;
}
```

Here, `chapter` is the selector and `color: navy;` is a declaration. CSSMedia reads XML names and attributes as data. It does not assign vocabulary-specific meanings to them.

The guides are grouped by selector type and matching behavior:

- Selector reference: a compact reference to the selector forms and their PDF behavior.
- Basic selectors and namespaces: element names, IDs, classes, attributes and XML namespaces.
- Combinators: selecting elements by their relationship to other elements.
- Pseudo-classes: structural tests, selector logic, language and other states.
- Pseudo-elements: selecting generated content, text parts and other special targets.
- Specificity: how selectors help resolve competing declarations.

Basic selectors and namespaces

Basic selectors match an element by its name, attributes or namespace. Combine selector parts when you want to require more than one condition.

Element names and any element

A type selector is an element name, such as `chapter` or `title`. It matches elements with that name. The universal selector `*` matches any element.

```
chapter { color: navy; }
* { font-family: serif; }
```

An unprefixed name uses the default namespace declared with `@namespace`. Without a declared default namespace, it matches an element with no namespace.

IDs and classes

An ID identifies an element. Write # followed by the ID value to select it. CSSMedia checks the `id` and `xml:id` attributes.

```
#overview { color: navy; }
```

A class selector starts with a period, as in `.warning`. Selectors Level 4 leaves the meaning of class membership to the document language, and XML has no universal class attribute. For that reason, class selectors only match inside SVG and MathML content, which define a `class` attribute.

If your XML has a `class` attribute with space-separated names, select it with an attribute selector. The ~= operator matches one whole word of the value:

```
[class~="warning"] { color: red; }
```

Attributes

An attribute selector checks an XML attribute. Put its name in square brackets to require that the attribute exists. Add an operator and value to compare its contents.

```
[role] { color: navy; }
[role="notice"] { color: navy; }
[class~="warning"] { color: red; }
[lang|="en"] { font-family: serif; }
[href^="https"] { color: blue; }
[href$=".pdf"] { color: purple; }
[data-kind*="reference"] { color: teal; }
[role="notice" i] { color: navy; }
```

= compares the whole value. ~= checks for a space-separated word. |= matches the value itself or a value followed by a hyphen. ^=, $= and *= check for text at the start, end or anywhere in the value. The `i` flag ignores ASCII letter case. The `s` flag requests case-sensitive comparison.

An attribute selector without a namespace prefix checks an attribute with no namespace. Use a declared prefix to select an attribute in a namespace.

XML namespaces

XML elements and attributes can belong to a namespace. The `@namespace` rule of CSS assigns a prefix to a namespace URI. Use that prefix in a selector to choose elements in that namespace.

```
@namespace svg url("http://www.w3.org/2000/svg");

svg|rect { color: navy; }
*|rect { color: teal; }
|rect { color: gray; }
```

`svg|rect` selects `rect` elements in the declared SVG namespace. `*|rect` selects `rect` elements in any namespace. `|rect` selects `rect` elements with no namespace. The CSS prefix is an alias for the URI. It does not need to match the prefix used in the XML.

A bare attribute selector such as `[id]` always selects an attribute with no namespace. An undeclared prefix makes that selector invalid. CSSMedia ignores a style rule if any selector in its selector list is invalid.

Combinators

A combinator describes the relationship between two elements. Put the related element's selector on the left and the element you want to match on the right.

Descendant and sibling relationships

Use a space to select a descendant at any depth. Use > for a direct child, + for the next element sibling and ~ for any later element sibling with the same parent.

```
chapter title { color: navy; }
chapter > title { color: teal; }
title + paragraph { color: gray; }
title ~ paragraph { color: purple; }
```

Example:

```
<chapter>
    <title>Setup</title>
    <paragraph>Install the tool.</paragraph>
    <paragraph>Start the tool.</paragraph>
    <section>
        <title>Options</title>
        <paragraph>Choose an option.</paragraph>
    </section>
</chapter>
```

- `chapter title` selects both titles, `Setup` and `Options`, because the second one is inside a `section` that is inside the chapter.
- `chapter > title` selects only `Setup`, the title that is a direct child of the chapter.
- `title + paragraph` selects `Install the tool.` and `Choose an option.`, the paragraphs that come right after a title.
- `title ~ paragraph` selects all three paragraphs, because each one has a title before it among its siblings.

Column combinator

The column combinator `||`, from Selectors Level 5, selects the cells of the column that a column element describes. CSSMedia finds columns and cells by the `display` values `table-column`, `table-column-group` and `table-cell`, not by element names. Columns must come before the first row. CSS gives a cell no span, so each cell takes the next column of its row.

```
colspec || entry { color: navy; }
```

This selects every `entry` in a column described by a `colspec`, when the stylesheet gives `colspec` the display value `table-column` and `entry` the value `table-cell`.

Reference combinators

A reference combinator, from Selectors Level 5, selects an element that is referred to by an attribute of another element.

```
note /target/ step { color: navy; }
```

The name between slashes is the attribute that holds the reference. The rule selects every `step` whose `id` or `xml:id` equals the `target` attribute of a `note`. The attribute value can start with `#`.

Example:

```
<note target="s2">Check the cable first.</note>
<step id="s1">Open the cover.</step>
<step id="s2">Unplug the cable.</step>
```

The rule selects only the second step, `Unplug the cable.`, because the note refers to `s2`.

Pseudo-classes

A pseudo-class starts with `:` and tests something about an element that its name and attributes do not tell, such as its position among its siblings or its language.

Write a pseudo-class after a selector. The rule applies only to elements that pass the test:

```
chapter:first-child { color: navy; }
```

The selector reference lists every pseudo-class that CSSMedia accepts.

Child and sibling position

Structural pseudo-classes select elements by their position among the elements that share a parent. Text, comments and processing instructions are not counted.

```
chapter:first-child { color: navy; }
chapter:last-child { color: teal; }
chapter:only-child { color: gray; }
chapter:first-of-type { color: purple; }
```

`:first-child`, `:last-child` and `:only-child` count all sibling elements. The `-of-type` forms count only siblings with the same element name and namespace.

Nth-position selectors

An `:nth-*` pseudo-class selects elements by their numbered position. Positions start at one. `odd` and `even` select alternating elements, and `An+B` describes a repeating sequence.

```
chapter:nth-child(odd) { color: navy; }
chapter:nth-child(2n + 1) { color: teal; }
chapter:nth-child(2 of [status="active"]) { color: purple; }
```

The `of` clause of `:nth-child()` and `:nth-last-child()` limits the count to siblings that match a selector. The `-last-` forms count from the end.

`:nth-col()` and `:nth-last-col()` select table cells by the position of their column. Columns are found as described in Combinators.

Root, empty and scope

`:root` selects the root element. `:empty` selects an element with no child elements and no text other than white space. `:scope` selects the scoping element, which is the root element unless a rule sets another scope.

```
:root { color: navy; }
chapter:empty { display: none; }
:scope > chapter { color: teal; }
```

Selector-list and relational pseudo-classes

These pseudo-classes combine selectors. `:is()` matches an element that matches at least one selector of its list. `:where()` does the same but adds no specificity. `:not()` excludes the elements that match its list. `:has()` selects an element by the elements related to it.

```
chapter:not([status="draft"]) { color: navy; }
:is(chapter, appendix) > title { color: teal; }
:where([compact]) paragraph { font-size: 10pt; }
section:has(> figure) { color: purple; }
title:has(+ paragraph) { color: gray; }
```

An invalid selector in a `:not()` list makes the whole selector invalid. `:is()` and `:where()` ignore invalid selectors. `:matches()` is accepted as another name for `:is()`.

Language and direction

`:lang()` selects elements by language. The language of an element is its nearest `xml:lang` value, or the value that the `-cssmedia-lang` property gives. See Document language.

```
:lang(fr) { font-family: serif; }
```

`:dir(ltr)` and `:dir(rtl)` select elements by text direction. CSSMedia takes the direction from the first strong character in the element's text, as the Unicode Bidirectional Algorithm does. Without one, it uses the direction in which the element's language is written, and left-to-right when there is no language. The `direction` property does not change the result.

Links and document location

An XML element is a link only when you make it one with `-cssmedia-link`, a CSSMedia extension that sets the destination of an element. The destination can be a place in the same document or an external address. See Links.

`:link` and `:any-link` select the elements that have a destination. `:local-link` selects the links that point into the same document. An optional integer limits the number of path segments that are compared.

```
xref { -cssmedia-link: attr(target); }
:any-link { color: blue; }
:local-link { color: navy; }
```

States that do not occur in a PDF

A PDF has no pointer, keyboard focus, media playback, form controls or browsing history. Pseudo-classes that test those states are valid selectors, but they never match an element. Examples are `:hover`, `:focus`, `:visited`, `:target`, `:playing`, `:checked` and `:disabled`. The same holds for `:state()` and `:heading`, because an XML element has no custom state and no heading role.

`:read-only` and `:defined` match every element. The selector reference gives the result for each pseudo-class.

Pseudo-elements

A pseudo-element starts with `::` and selects a part of an element or a piece of generated content. It follows the element selector.

```
paragraph::first-letter { color: navy; }
paragraph::before { content: "Note: "; }
```

The selector reference lists every pseudo-element that CSSMedia accepts.

Text parts

`::first-line` selects the first formatted line of a block. `::first-letter` selects its first letter, together with the punctuation around it. `::first-letter::prefix` and `::first-letter::suffix` select that punctuation.

```
paragraph::first-line { font-weight: bold; }
paragraph::first-letter { font-size: 150%; }
```

Generated content, markers and footnotes

`::before` and `::after` are positions before and after the content of an element. They exist only when the `content` property gives them something to show.

```
paragraph::before { content: "Note: "; }
item::marker { color: navy; }
```

`::marker` selects the marker of a list item. `::before::marker` and `::after::marker` select the marker of generated content that is a list item.

`::footnote-call` and `::footnote-marker` select the reference that replaces a footnote in the text and the marker at the start of the footnote.

To copy the text of an element into a page margin box, use `string-set`. See Named strings.

Pseudo-elements without a target in a PDF

Some pseudo-elements select ranges or widgets that exist only on screen. `::selection`, `::search-text`, `::target-text`, `::spelling-error`, `::grammar-error` and `::highlight()` select highlighted text. `::placeholder`, `::file-selector-button`, `::details-content` and `::backdrop` select parts of form controls and interactive elements. CSSMedia accepts them, but an XML document has nothing for them to select.

Specificity

Specificity helps the cascade choose between declarations from matching rules. It is one part of the cascade, after origin, importance and layer priority.

Comparing selector specificity

Each part of a selector adds to its specificity, as Selectors Level 4 defines. An ID selector counts more than an attribute selector or a pseudo-class. Those count more than an element name or a pseudo-element.

```
chapter { color: gray; }
[role="notice"] { color: teal; }
#important-notice { color: navy; }
```

If all other cascade priorities are equal, the ID rule wins over the attribute rule, and the attribute rule wins over the element-name rule. If competing rules have the same specificity, source order decides and the later declaration wins.

`:where()` contributes no specificity. For `:is()`, `:not()` and `:has()`, specificity is based on the most specific selector in the argument. The `&` nesting selector contributes specificity according to the parent selector list.

An attribute selector such as `[class~="warning"]` has the specificity of an attribute selector. See Basic selectors and namespaces for how to select a `class` attribute in XML.

Selector reference

These tables list every selector that CSSMedia accepts. The `In a PDF` column says whether the selector matches elements while CSSMedia generates a PDF.

Basic selectors

Basic selectors match an element by its name, ID or attributes. See Basic selectors and namespaces.

Name	Description	Values	In a PDF	Specification
`E`	Type selector. Matches elements named `E`.	An element name.	Matches	Selectors 4 §5.1
`*`	Universal selector. Matches any element.		Matches	Selectors 4 §5.2
`#id`	ID selector. Matches the element whose `id` or `xml:id` attribute has this value.	An identifier.	Matches	Selectors 4 §6.7
`.name`	Class selector. Matches inside SVG and MathML content, which define a `class` attribute.	An identifier.	Matches in SVG and MathML only	Selectors 4 §6.6
`[attr]`	Matches elements that have the attribute.	An attribute name, with an optional namespace prefix.	Matches	Selectors 4 §6
`[attr=value]`	Matches when the attribute value is exactly `value`.	A string or an identifier. Add `i` to ignore ASCII case or `s` to require it.	Matches	Selectors 4 §6.1

Name	Description	Values	In a PDF	Specification
`[attr~=value]`	Matches when the value is one of the white-space separated words of the attribute.	A word without white space.	Matches	Selectors 4 §6.1
`[attr|=value]`	Matches when the attribute is `value` or starts with `value-`.	A string or an identifier.	Matches	Selectors 4 §6.1
`[attr^=value]`	Matches when the attribute starts with `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`[attr$=value]`	Matches when the attribute ends with `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`[attr*=value]`	Matches when the attribute contains `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`ns|E`, `*|E`, `|E`	Limits a type selector to a declared namespace, to any namespace, or to no namespace. The same prefixes work in attribute selectors.	A prefix declared with `@namespace`.	Matches	Selectors 4 §5.3, CSS Namespaces 3 §3
`&`	Nesting selector. Stands for the selector of the parent rule.		Matches	CSS Nesting 1

Combinators

A combinator relates two selectors. See Combinators.

Name	Description	Values	In a PDF	Specification
`A B`	Descendant combinator. Matches `B` inside `A` at any depth.	Two selectors separated by white space.	Matches	Selectors 4 §14.1
`A > B`	Child combinator. Matches `B` when its parent is `A`.	Two selectors.	Matches	Selectors 4 §14.2

Name	Description	Values	In a PDF	Specification
`A + B`	Next-sibling combinator. Matches B when it directly follows A.	Two selectors.	Matches	Selectors 4 §14.3
`A ~ B`	Subsequent-sibling combinator. Matches B when A is an earlier sibling.	Two selectors.	Matches	Selectors 4 §14.4
`A || B`	Column combinator. Matches the cells B of the column that A describes.	Two selectors.	Matches	Selectors 5 §8.1
`A / attr/ B`	Reference combinator. Matches B when the attribute `attr` of A refers to its ID.	Two selectors and an attribute name between slashes.	Matches	Selectors 5 §9.1

Pseudo-classes

A pseudo-class tests a property of an element other than its name and attributes. See Pseudo-classes.

Name	Description	
`:root`	The root element of the document.	
`:empty`	An element with no child elements and no text other than white space.	
`:scope`	The scoping element. Without another scope, the root element.	
`:first-child`, `:last-child`, `:only-child`	The first, last or only element among its siblings.	
`:first-of-type`, `:last-of-type`, `:only-of-type`	The first, last or only element of its type among its siblings.	

Name	Description	
`:nth-child()`, `:nth-last-child()`	Elements at a position among their siblings, counted from the start or from the end.	odd, An+B follo and list.
`:nth-of-type()`, `:nth-last-of-type()`	Elements at a position among the siblings of the same type.	odd, An+B
`:nth-col()`, `:nth-last-col()`	Table cells at a position among the columns of their table.	odd, An+B
`:is()`, `:matches()`	An element that matches any selector of the list. `:matches()` is the old name of `:is()`.	A sel Inval are i
`:where()`	Works as `:is()` and adds no specificity.	A sel Inval are i
`:not()`	An element that matches none of the selectors of the list.	A sel An in selec the v selec
`:has()`	An element that has a related element matching the selector.	A rel selec such `figu` cann anot or a elem

Name	Description	
`:lang()`	Elements in a language. The language comes from `xml:lang` or the `-cssmedia-lang` property.	One lang rang iden strin
`:dir()`	Elements by text direction, read from the text and the language, not from the `direction` property.	`ltr`
`:link`, `:any-link`	Elements that are hyperlinks, because `-cssmedia-link` gives them a destination.	
`:local-link`	Hyperlinks that point to the same document. The optional number limits how many path segments are compared.	An o nega in pa
`:read-only`, `:defined`	Every element is read-only and defined, because an XML element has no editable or undefined state.	
`:hover`, `:active`, `:focus`, `:focus-visible`, `:focus-within`	User action states, such as pointing at an element or giving it keyboard focus.	

Name	Description	
`:interest-source`, `:interest-target`	Source and target of a user's declared interest.	
`:visited`, `:target`	Links that were visited, and the element that a link points to. A PDF has no browsing history and no current target.	
`:playing`, `:paused`, `:seeking`, `:buffering`, `:stalled`, `:muted`, `:volume-locked`	Media playback, loading and sound states.	
`:current`, `:current()`, `:past`, `:future`	Elements in relation to a timeline, such as speech or video playback.	A sel for :
`:open`, `:popover-open`, `:modal`, `:fullscreen`, `:picture-in-picture`	Display states of interactive elements.	
`:enabled`, `:disabled`, `:read-write`, `:placeholder-shown`, `:autofill`, `:default`, `:checked`, `:unchecked`, `:indeterminate`, `:blank`	Form control states. An XML element is never a form control.	
`:valid`, `:invalid`, `:in-range`, `:out-of-range`, `:required`, `:optional`, `:user-valid`, `:user-invalid`	Form validation states.	
`:state()`	A custom state that a document language exposes. XML has none.	An id
`:heading`, `:heading()`	Headings, as a document language defines them. CSSMedia does not give XML elements a heading role.	A list for :

Pseudo-elements

A pseudo-element selects a part of an element or generated content. See Pseudo-elements.

Name	Description	Values	In a PDF	Specification
`::before`, `::after`	Generated content before or after the content of an element. The `content` property gives it.		Matches	CSS Pseudo-Elements 4 §4.1
`::marker`	The marker of a list item. Also `::before::marker` and `::after::marker`.		Matches	CSS Pseudo-Elements 4 §4.2
`::first-line`	The first formatted line of a block.		Matches	CSS Pseudo-Elements 4 §2.1
`::first-letter`	The first letter of a block, with the punctuation around it. `::first-letter::prefix` and `::first-letter::suffix` select that punctuation.		Matches	CSS Pseudo-Elements 4 §2.2
`::footnote-call`	The reference that replaces a footnote in the text. By default it is the value of the `footnote` counter, as a superscript.		Matches	CSS GCPM 3 §2.6
`::footnote-marker`	The marker at the start of a footnote body. By default it is the value of the `footnote` counter, followed by a period and a space.		Matches	CSS GCPM 3 §2.7
`::selection`, `::search-text`, `::target-text`, `::spelling-error`, `::grammar-error`, `::highlight()`	Highlighted ranges created by selection, search, navigation, spelling and grammar checking, or scripts.	A custom identifier for `::highlight()`.	No range exists	CSS Pseudo-Elements 4 §3.1

Name	Description	Values	In a PDF	Specification
`::placeholder`, `::file-selector-button`, `::details-content`	Parts of form controls and disclosure widgets.		No such part exists	CSS Pseudo-Elements 4 §4.3, §5.1, §5.2
`::backdrop`	The backdrop of an element shown in the top layer.		No such part exists	CSS Position 4 §3.2

Define pages

Use an @page rule to set the size, margins and other properties of the pages in the PDF.

Procedure

1. Write an @page rule

   An @page rule has an optional list of page selectors, separated by commas, and a block of declarations:

   ```
   @page {
       size: A4;
   }

   @page :first, :left {
       margin-top: 25mm;
   }
   ```

   The first rule has no selector and applies to every page. The second applies to pages that match either selector. A selector is an optional page type name followed by page pseudo-classes, with no white space between them.

   A block can contain page properties, the `size` descriptor, and page-margin rules.

2. Set the page size with `size`

   The `size` descriptor sets the size of the page. Its initial value is `auto`, which gives an A4 page.

   ```
   @page {
       size: A4;
   }

   @page landscape {
       size: A4 landscape;
   }

   @page square {
       size: 150mm;
   }

   @page custom {
       size: 210mm 297mm;
   }
   ```

   - A page size name, such as A4, selects a standard sheet.
   - `portrait` puts the shorter sides horizontal and `landscape` puts the longer sides horizontal. Use one of them with a name, or alone to orient the default A4 sheet.
   - One length sets the width and the height, which gives a square page.
   - Two lengths set the width and then the height. A length cannot be negative.

   These are the page size names:

Name	Millimeters	Inches
A3	297 x 420	11.69 x 16.54
A4	210 x 297	8.27 x 11.69
A5	148 x 210	5.83 x 8.27
B4	250 x 353	9.84 x 13.90
B5	176 x 250	6.93 x 9.84
JIS-B4	257 x 364	10.12 x 14.33
JIS-B5	182 x 257	7.17 x 10.12
letter	215.9 x 279.4	8.5 x 11
legal	215.9 x 355.6	8.5 x 14
ledger	279.4 x 431.8	11 x 17

The sizes are width by height, in portrait orientation.

7. Set the page margins

   Page margins are the space between the edge of the page and the page area, where the content of the document is placed. If you do not set a margin, it is `1in`.

   The `margin` property sets all four margins. It takes one to four values, which are lengths, percentages or `auto`:

   - One value sets all four sides.
   - Two values set the top and bottom, then the right and left.
   - Three values set the top, then the left and right, then the bottom.
   - Four values set the top, right, bottom and left, in that order.

   A percentage is a fraction of the page width for the left and right margins, and of the page height for the top and bottom margins.

```
@page {
    size: A4;
    margin: 20mm 25mm;
}
```

   Use `margin-top`, `margin-right`, `margin-bottom` and `margin-left` to set one side. For example, use wider inner margins in a two-sided document:

```
@page :left {
    margin-left: 30mm;
}

@page :right {
    margin-right: 30mm;
}
```

12. Select pages

A page selector chooses the pages that a rule applies to, by page type or by position.

Page types

A page type name matches the pages that get that name from the `page` property. For example, `@page wide` matches the pages of type `wide`. Names are case-sensitive.

Page pseudo-classes

- `:first` matches the first page.
- `:left` and `:right` match left and right pages. In a document that is read left to right, the first page is a right page.
- `:blank` matches the empty pages that forced page breaks produce.
- `:nth(An+B)` matches pages by their position in the document. `:nth(2n)` selects pages 2, 4, 6 and so on, and `:nth(3n+1)` selects pages 1, 4, 7 and so on.

Add `of` and a page type to count only the pages of one page group, as in `:nth(1 of chapter)`. An element that has a named page and a forced page break starts a new page group.

```
@page :first {
    margin-top: 30mm;
}

@page :nth(2n) {
    margin-top: 25mm;
}
```

17. Assign a page type with `page`

    The `page` property gives the pages of an element a page type. Its value is `auto` or a name that you also use in an `@page` rule.

    ```
    @page wide {
        size: A4 landscape;
    }

    wide-table {
        page: wide;
    }
    ```

    With `auto`, an element takes the page type of its nearest ancestor that has a name. A box with a different page type than the page before it starts on a new page.

18. Fill the page margins

    A page-margin rule is an at-rule inside an `@page` rule, such as `@top-center` or `@bottom-right`. Each one defines a box in the margin. The box appears only if its `content` property has a value other than `none` or `normal`. Use `counter(page)` to show the page number.

    For example, put a running title in the top-center box and the page number in the bottom-center box:

```
@page {
    @top-center {
        content: "Document title";
    }

    @bottom-center {
        content: counter(page);
    }
}
```

The page reference lists the 16 page-margin rules and where each box is. See Named strings to show the text of an element in a margin box.

Page setup reference

This reference lists the rules, descriptors and properties that CSSMedia uses to define pages.

Page rule, size and page type

Name	Description	Values	Default	Specification
`@page`	Defines page-box declarations and may contain page-margin rules.	Optional comma-separated page selectors		CSS Paged Media 3, `@page`
`size`	Sets page-box size and orientation. `auto` uses A4.	`auto`; one or two lengths; or A3, A4, A5, B4, B5, `JIS-B4`, `JIS-B5`, `ledger`, `legal`, or `letter`, optionally combined with `portrait` or `landscape`	A4	CSS Paged Media 3, `size`
`page`	Assigns a named page type to a box.	`auto` or a custom identifier	`auto`	CSS Paged Media 3, `page`

Page margins

Name	Description	Values	Default	Specification
`margin`	Sets the space outside all four sides of a box.	One to four lengths or percentages, `auto`, or `anchor-size()`.		CSS Box Model 4
`margin-top`	Sets the space outside the top side of a box.	Length, percentage, `auto`, or `anchor-size()`.		CSS Box Model 4
`margin-right`	Sets the space outside the right side of a box.	Length, percentage, `auto`, or `anchor-size()`.		CSS Box Model 4

Name	Description	Values	Default	Specification
`margin-bottom`	Sets the space outside the bottom side of a box.	Length, percentage, `auto`, or `anchor-size()`.		CSS Box Model 4
`margin-left`	Sets the space outside the left side of a box.	Length, percentage, `auto`, or `anchor-size()`.		CSS Box Model 4

Page-margin rules

Name	Description	Values	Default	Specification
`@top-left-corner`	Defines the fixed-size box at the intersection of the top and left page margins.			CSS Paged Media 3, `@top-left-corner`
`@top-left`	Defines the box between the top-left corner and top-center boxes.			CSS Paged Media 3, `@top-left`
`@top-center`	Defines the centered box in the top page margin.			CSS Paged Media 3, `@top-center`
`@top-right`	Defines the box between the top-center and top-right corner boxes.			CSS Paged Media 3, `@top-right`
`@top-right-corner`	Defines the fixed-size box at the intersection of the top and right page margins.			CSS Paged Media 3, `@top-right-corner`
`@right-top`	Defines the box between the top-right corner and right-middle boxes.			CSS Paged Media 3, `@right-top`
`@right-middle`	Defines the centered box in the right page margin.			CSS Paged Media 3, `@right-middle`
`@right-bottom`	Defines the box between the right-middle and bottom-right corner boxes.			CSS Paged Media 3, `@right-bottom`
`@bottom-right-corner`	Defines the fixed-size box at the intersection of the bottom and right page margins.			CSS Paged Media 3, `@bottom-right-corner`
`@bottom-right`	Defines the box between the bottom-center and bottom-right corner boxes.			CSS Paged Media 3, `@bottom-right`
`@bottom-center`	Defines the centered box in the bottom page margin.			CSS Paged Media 3, `@bottom-center`

Name	Description	Values	Default	Specification
`@bottom-left`	Defines the box between the bottom-left corner and bottom-center boxes.			CSS Paged Media 3, `@bottom-left`
`@bottom-left-corner`	Defines the fixed-size box at the intersection of the bottom and left page margins.			CSS Paged Media 3, `@bottom-left-corner`
`@left-bottom`	Defines the box between the left-middle and bottom-left corner boxes.			CSS Paged Media 3, `@left-bottom`
`@left-middle`	Defines the centered box in the left page margin.			CSS Paged Media 3, `@left-middle`
`@left-top`	Defines the box between the top-left corner and left-middle boxes.			CSS Paged Media 3, `@left-top`

Page orientation, bleed and marks

Named strings

A named string saves text from an element so that you can use it elsewhere, such as in a running header or footer. `string-set` saves the text and `string()` retrieves it.

About this task

The element names in the examples, such as `title` and `h1`, are placeholders. Use the names that occur in your XML.

Procedure

1. Save a string with `string-set`

   The `string-set` property takes none, which is the initial value, or a comma-separated list of assignments. Each assignment is a name that you choose, followed by what to save.

   ```
   h1 {
       string-set: chapter-title content(text);
   }
   ```

   Each `h1` element saves its text in `chapter-title`. When a later `h1` is found, it replaces the saved value. One declaration can save several strings:

   ```
   h1 {
       string-set: chapter-title content(text), chapter-label "Chapter "
   counter(chapter);
   }
   ```

   What to save is a list of one or more of these items, joined in order:

   - A quoted string, such as `"Chapter "`.
   - `content()` gives text of the element. `content(text)`, which is the default, gives the text of the element and its descendants, with white space collapsed. `content(before)` and `content(after)` give the text generated by `::before` and `::after`. `content(first-letter)` gives the first letter.
   - `attr(name)` gives the value of an attribute of the element. `attr(name, "fallback")` gives the fallback when the attribute is missing.
   - `counter(name)` gives the value of the innermost counter with that name. `counters(name, ".")` gives the values of all the counters with that name, from the outermost, joined with the string. Both accept a counter style as the last argument, such as `upper-roman`.

   This XML and stylesheet save the text, a prefix and an attribute of a title:

   ```
   <title code="A-4">Install <em>the package</em></title>
   ```

```
title {
    string-set: title-text content(text), title-prefix content(before), title-
code attr(code, "No code");
}

title::before {
    content: "Chapter ";
}

@page {
    @top-left {
        content: string(title-prefix) string(title-code);
    }
    @top-right {
        content: string(title-text);
    }
}
```

The left margin shows "Chapter A-4" and the right margin shows "Install the package". For a title without a `code` attribute, the left margin shows "Chapter No code".

A string is saved where the box of its element starts, or where it would start if the element has `display: none`. That is why a page can show different values of the same string, depending on which elements it contains.

6. Retrieve a string with `string()`

   Use `string(name)` in the `content` property of a page-margin box. The name is the one you wrote in `string-set`. An optional second argument chooses which value to use on the page.

   ```
   @page {
       @top-center {
           content: string(chapter-title, first);
       }
   }
   ```

   - `first` is the first value saved on the page. If the page does not save one, it is the value in effect at the end of the previous page. This is the default.
   - `start` is the value in effect at the top of the page. It is empty if the string has not been saved yet.
   - `last` is the value in effect at the end of the page.
   - `first-except` works like `first`, but is empty on a page that saves the string. It is useful to hide a running header on the page where a chapter starts.

Named strings reference

This reference lists the property and the functions that CSSMedia uses for named strings.

Name	Description	Values	Default	Specification
`string-set`	Assigns named values from an element for retrieval in page content.	none or comma-separated pairs of a name and one or more strings		CSS Content 3 §2.7.1
`content()`	Gives text of an element. Use it in the value of `string-set`.	`text`, `before`, `after` or `first-letter`	`text`	CSS GCPM 3 §1.1.1.1
`string()`	Gives the value of a named string saved with `string-set`. Use it in the `content` property of a page-margin box.	A name, and an optional mode: `first`, `start`, `last` or `first-except`	`first`	CSS GCPM 3 §1.1.2

Links

A link points from an element to another part of the document or to an external address. XML has no standard names for links, so you use the CSSMedia property `-cssmedia-link` to tell CSSMedia which elements are links and where they go. The link reference lists the property, the function and the pseudo-classes used for links.

About this task

The element and attribute names in the examples, such as `xref` and `target`, are placeholders. Use the names that occur in your XML.

Procedure

1. Set a link destination

   The `-cssmedia-link` property sets the destination of an element. It is a CSSMedia extension, so its name has the prefix `-cssmedia-`. Its value is none, which is the initial value and means that the element is not a link, or one or more of these:

   - A string, such as `"chapter-2.xml#installation"` or `"#details"`.
   - A `url()` value.

   CSSMedia joins the values in order, without spaces, to make the destination. The property is not inherited, so set it on each element that is a link. Use none to remove a link.

   This XML has an `xref` element with a `target` attribute:

   ```
   <xref target="chapter-2.xml#installation">Installation instructions</xref>
   ```

   Use `attr()` to take the destination from the attribute:

   ```
   xref {
       -cssmedia-link: attr(target);
   }
   ```

   The text of the element is the text of the link, and the attribute is its destination. If the attribute is missing, `attr()` gives an empty string, unless you give a fallback.

   To link to a place in the same document, add # before the ID:

   ```
   <xref target="details">More details</xref>
   ```

   ```
   xref {
       -cssmedia-link: "#" attr(target);
   }
   ```

   The link goes to `#details`, the element of the document with the ID `details`.

   To link to an external address, write the address as a string:

   ```
   xref[role="external"] {
       -cssmedia-link: "https://example.org/manual/installation.html";
   }
   ```

A relative string, such as `"chapter-2.xml#installation"`, is resolved against the address of the XML document. A `url()` value is resolved against the address of the stylesheet. Use a string when the address is relative to the XML document.

4. Style the links

   Setting `-cssmedia-link` does not change how a link looks. Use `:any-link` to select the elements that have a destination, and set the properties that you want:

   ```
   :any-link {
       color: navy;
   }
   ```

   `:link` selects the same elements, because a PDF has no visited links. `:visited` never matches.

5. Select links by destination

   `:local-link` selects the links that point to the same document. CSSMedia ignores the fragment when it compares the addresses, so a link to `#details` is a local link.

   ```
   xref {
       -cssmedia-link: attr(target);
   }

   xref:local-link {
       color: navy;
   }
   ```

   Add a number to compare part of the address: `:local-link(0)` selects the links with the same origin, which is the scheme, the host and the port. `:local-link(1)` also compares the first segment of the path, `:local-link(2)` the first two, and so on. Combine it with `:not()` to select the links to other origins:

   ```
   xref:not(:local-link(0)) {
       color: maroon;
   }
   ```

6. Show a counter of the destination

   `-cssmedia-link-counter()` gives the value of a counter at the destination of the link of an element. Use it in the `content` property, for example to show the number of the footnote that a link points to:

   ```
   <footnoteref linkend="note-1"/>
   ```

   ```
   footnoteref {
       -cssmedia-link: "#" attr(linkend);
       content: -cssmedia-link-counter(footnote);
   }
   ```

   The element links to the element with the ID `note-1` and shows the value of the `footnote` counter of that element. An optional second argument sets the counter style, as in `-cssmedia-link-counter(footnote, lower-alpha)`.

Link reference

This reference lists the property, the function and the pseudo-classes that CSSMedia uses for links.

Property and function

Name	Description	Values	Default	Specification
`-cssmedia-link`	Makes an XML element a link to another part of the document or to an external address. The values are joined in order to make the destination.	none, or one or more strings or `url()` values	none	CSSMedia extension
`-cssmedia-link-counter()`	Gives the value of a counter at the destination of the link of the element, such as the number of a footnote that the link points to. CSSMedia extension.	The name of a counter, and an optional counter style.		CSSMedia extension

Pseudo-classes

Name	Description	Values	In a PDF	Specification
`:link`, `:any-link`	Elements that are hyperlinks, because `-cssmedia-link` gives them a destination.		Matches	Selectors 4 §8.1, §8.2
`:local-link`	Hyperlinks that point to the same document. The optional number limits how many path segments are compared.	An optional non-negative integer in parentheses.	Matches	Selectors 5 §2.1
`:visited`, `:target`	Links that were visited, and the element that a link points to. A PDF has no browsing history and no current target.		Never matches	Selectors 4 §8.2, §8.3

Footnotes

A footnote is an element that CSSMedia moves to the bottom of the page where the text refers to it. In the text, a number takes its place. The footnote reference lists the properties, rules and pseudo-elements used here.

About this task

These terms are used in this chapter:

- The **footnote body** is the content of the footnote element.
- The **footnote call** is the number that replaces the footnote in the text.
- The **footnote marker** is the number at the start of the footnote body.
- The **footnote area** is the box at the bottom of the page that holds the footnote bodies.

The element names in the examples, such as `fn`, are placeholders. Use the names that occur in your XML.

Procedure

1. Mark an element as a footnote

   Set `float: footnote` on the element that holds the text of the footnote:

   ```
   <p>Swordfish stores translations in memories<fn>A memory is a database of
   translated segments.</fn>.</p>
   ```

   ```
   fn {
       float: footnote;
       font-size: 8pt;
   }
   ```

   CSSMedia removes `fn` from the text, puts a number in its place, and shows the content of `fn` in the footnote area of the same page, in document order. If the footnote does not fit on the page, the rest continues on the next pages.

   The rule of the footnote element styles the footnote body, so set the font size and the line height there.

2. Number the footnotes

   Footnotes are numbered with the `footnote` counter. A counter is a number that CSS keeps for you and that you can show in the content of a document. The `footnote` counter already exists: CSSMedia creates it at the start of the document with the value 0 and adds 1 each time it finds a footnote. The first footnote is 1, the second is 2, and so on through the whole document.

   The call and the marker of a footnote show the same value, because both use `counter(footnote)`.

   You change the numbering with the properties `counter-reset`, `counter-set` and `counter-increment`:

   - `counter-reset: footnote` sets the counter to 0, so the next footnote is 1. `counter-reset: footnote 9` sets it to 9, so the next footnote is 10.

- `counter-set: footnote 4` also sets the value of the counter.
- `counter-increment: footnote 2` adds 2 to the counter, and `counter-increment: footnote -1` subtracts 1.

Where you write the declaration decides where the numbering changes. In an element rule, it changes when the element starts. This example starts the numbers at 1 in each chapter:

```
chapter {
    counter-reset: footnote;
}
```

In an `@page` rule, it changes on every page. This example starts the numbers at 1 on each page:

```
@page {
    counter-reset: footnote;
}
```

With the rule in `@page`, the number of a footnote depends on the page of its call, and a footnote that continues on the next page keeps its number.

The optional second argument of `counter()` is a counter style, which sets how the number is written. For example, `counter(footnote, lower-alpha)` shows a, b, c, and `counter(footnote, upper-roman)` shows I, II, III. A style can also list the symbols to use:

```
fn::footnote-call {
    content: counter(footnote, symbols("*" "†" "‡"));
}

fn::footnote-marker {
    content: counter(footnote, symbols("*" "†" "‡")) " ";
}
```

The first three footnotes show *, † and ‡. Write the same style in the call and in the marker.

6. Style the call and the marker

   The pseudo-element `::footnote-call` is the number in the text. The pseudo-element `::footnote-marker` is the number at the start of the footnote. By default, the call is the value of the `footnote` counter as a superscript, and the marker is the value followed by a period and a space.

   ```
   ::footnote-call {
       font-size: 70%;
       vertical-align: super;
       line-height: 0;
   }

   ::footnote-marker {
       font-weight: bold;
   }
   ```

   Use the `content` property to change what they show. For example, show the number in brackets:

```
fn::footnote-call {
    content: "[" counter(footnote) "]";
}

fn::footnote-marker {
    content: "[" counter(footnote) "] ";
}
```

7. Use your own callouts

   To show a symbol or a label from the XML instead of a number, give the call and the marker that text, and stop the footnote from using a number. Every footnote adds 1 to the counter, and `counter-increment: footnote -1` takes it back, so the other footnotes keep their numbers.

```
<p>The price<fn callout="*">Prices do not include taxes.</fn> is final.</p>
```

```
fn[callout] {
    counter-increment: footnote -1;
}

fn[callout]::footnote-call {
    content: attr(callout);
}

fn[callout]::footnote-marker {
    content: attr(callout) " ";
}
```

8. Style the footnote area

   Use an `@footnote` rule inside an `@page` rule to style the footnote area. The area sits at the bottom of the page area. A border and a margin separate it from the text.

```
@page {
    @footnote {
        border-top: 0.5pt solid black;
        margin-top: 6pt;
        padding-top: 3pt;
    }
}
```

   Set `max-height` to limit the height of the area, so that a page does not fill up with footnotes. The limit does not apply to a page that has only footnotes. Because `@footnote` belongs to an `@page` rule, you can set a different limit for a named page:

```
@page limited {
    @footnote {
        max-height: 25mm;
    }
}
```

9. Choose how footnotes are displayed

   The `footnote-display` property sets how a footnote body appears in the footnote area:

- `block` puts each footnote in its own block. This is the initial value.
- `inline` runs the footnotes together in one paragraph.
- `compact` puts footnotes in the same line when they fit, and in separate blocks when they do not.

With `inline` and `compact`, add a margin after each footnote to separate it from the next one:

```
fn {
    float: footnote;
    footnote-display: compact;
    margin-inline-end: 1em;
}
```

13. Choose what happens when a footnote does not fit

    The `footnote-policy` property sets what CSSMedia does when the body of a footnote does not fit on the page of its call:

    - `auto` starts the footnote on the page of its call and continues it on the next pages. This is the initial value.
    - `line` moves the line that has the call to the next page, together with the footnote. Widows and orphans are respected, so more than one line can move.
    - `block` moves the whole paragraph that has the call to the next page, together with the footnote.

    ```
    fn {
        float: footnote;
        footnote-policy: line;
    }
    ```

Footnote reference

This reference lists the properties, the rule, the counter and the pseudo-elements that CSSMedia uses for footnotes.

Properties, rule and counter

Name	Description	Values	Default	Specification
`float`	Takes a box out of the normal flow and places it at the side, at the top or at the bottom of its container. The value `footnote` makes the element a footnote.	`none`, `left`, `right`, `top`, `bottom`, `block-start`, `block-end`, `inline-start`, `inline-end`, `snap-block`, `snap-inline` or `footnote`	`none`	CSS Page Floats 3 §3.2

Name	Description	Values	Default	Specification
`footnote-display`	Sets how a footnote body appears in the footnote area.	`block`, `inline`, or `compact`	`block`	CSS GCPM 3 §2.3
`footnote-policy`	Sets what happens when a footnote does not fit on the page of its call. `auto` continues it on the next pages, `line` moves the line with the call to the next page, and `block` moves the paragraph.	`auto`, `line`, or `block`	`auto`	CSS GCPM 3 §2.8
`footnote` counter	The counter that numbers the footnotes. Every footnote adds 1 to it. Change it with `counter-reset`, `counter-set` and `counter-increment`, in an element rule to number by section or in an `@page` rule to number by page.	Show it with `counter(footnote)`, with an optional counter style.	0 at the start of the document.	CSS GCPM 3 §2.5
`@footnote`	Styles the footnote area at the bottom of the page. It goes inside an `@page` rule.	A block of declarations, such as borders, margins, padding and `max-height`.		CSS GCPM 3 §2.4

Pseudo-elements

Name	Description	Values	In a PDF	Specification
`::footnote-call`	The reference that replaces a footnote in the text. By default it is the value of the `footnote` counter, as a superscript.		Matches	CSS GCPM 3 §2.6

Name	Description	Values	In a PDF	Specification
`::footnote-marker`	The marker at the start of a footnote body. By default it is the value of the `footnote` counter, followed by a period and a space.		Matches	CSS GCPM 3 §2.7

Table of contents

Text and fonts

The font properties in your stylesheet decide how text looks in the PDF.

Choose a family, size and weight first. Load your own fonts when you need ones that are not available, and use font features for finer control. These topics explain each step:

- Font selection and sizing
- Font resources and descriptors
- Font variants and features
- Variable and color fonts

Font selection and sizing

Font declarations choose a family, a size and a face within that family. They can also control line spacing and whether the processor may create a style that the font does not contain. Put declarations inside a rule that selects the XML elements you want to style:

About this task

```
chapter {
    font-family: "Noto Serif", serif;
    font-size: 12pt;
    font-weight: 400;
}
```

For variant and OpenType feature properties, see Font variants and features. For variable fonts and color palettes, see Variable and color fonts.

Procedure

1. Choosing the font family and size

   `font-family`

   CSS Fonts 4 §2.1, Font family: the font-family property; CSS Fonts 4 §2.1.5, Generic font families.

   Use `font-family` to name the typeface family. Separate alternatives with commas. Put a family name containing spaces in quotes. If a family is unavailable or cannot provide a character, the processor can try the next family in the list.

   ```
   body { font-family: "Noto Serif", "Times New Roman", serif; }
   ```

   The final `serif` is a generic family name. It describes a broad typeface style rather than naming a font file. Other generic names include `sans-serif` and `monospace`. Available family names depend on the fonts CSSMedia can access.

   PDF standard fonts

   ISO 32000-1 §9.6.2, Standard Type 1 Fonts.

   PDF defines 14 standard typefaces. They are grouped here by design and face:

Group	Standard typefaces
Serif	`Times-Roman`, `Times-Bold`, `Times-Italic`, `Times-BoldItalic`
Sans-serif	`Helvetica`, `Helvetica-Bold`, `Helvetica-Oblique`, `Helvetica-BoldOblique`
Monospaced	`Courier`, `Courier-Bold`, `Courier-Oblique`, `Courier-BoldOblique`
Symbol	`Symbol`
Dingbats	`ZapfDingbats`

- The Times faces have serifs. The Helvetica faces do not. The Courier faces use fixed-width advances, so each glyph occupies the same width.
- `Symbol` contains a symbol character set. `ZapfDingbats` contains ornamental marks and symbols.
- In each four-face group, `Bold` identifies the heavier face, `Italic` the italic face, and `Oblique` the slanted face. `Times-BoldItalic` combines bold and italic; `Helvetica-BoldOblique` and `Courier-BoldOblique` combine bold and oblique.
- CSSMedia includes these standard font faces as font data. Its built-in fallback for the generic family `serif` uses the four Times faces, `sans-serif` uses the four Helvetica faces, and `monospace` uses the four Courier faces.
- CSS family lookup also includes the standard groups `Times`, `Helvetica`, `Courier`, `Symbol` and `ZapfDingbats`. The generic keywords describe broad typographic categories; the standard group names identify the bundled font families.

`font-size`

CSS Fonts 4 §2.5, Font size: the font-size property.

Use `font-size` to set the size of text. You can use a length such as `12pt`, a percentage, a relative keyword such as `larger`, or a length relative to another font size.

```
body { font-size: 12pt; }
h1 { font-size: 1.5em; }
small { font-size: smaller; }
```

An `em` value is based on the font size inherited by the selected element. A percentage is based on the parent element's font size. Use a length such as `pt` when you want to state an explicit size.

`font-size-adjust`

CSS Fonts 5 §3.4, Relative sizing: the font-size-adjust property.

Use `font-size-adjust` to request a consistent size for a chosen font measurement, such as the height of lowercase letters. This can help keep text visually similar when the preferred font is unavailable and a fallback font has different proportions.

```
body { font-size-adjust: 0.5; }
```

The property can use a metric name such as `ex-height` or `cap-height`, followed by a number, or `from-font` to use the metric from the font.

7. Selecting a font face

   `font-weight`

   CSS Fonts 4 §2.2, Font weight: the font-weight property.

   Use `font-weight` to choose how heavy the letters look. The common values are `normal` and `bold`. You can also use a number from `1` to `1000`. In the usual scale, `400` is normal and `700` is bold.

   ```
   body { font-weight: 400; }
   strong { font-weight: 700; }
   ```

   The values `bolder` and `lighter` choose a weight relative to the inherited weight. A font family may not contain every requested weight. CSSMedia uses this property when matching an available font face.

   `font-width`

   CSS Fonts 4 §2.3, Font width: the font-width property.

   Use `font-width` to request narrower or wider letters. You can use a percentage or a keyword from `ultra-condensed` through `ultra-expanded`.

   ```
   heading { font-width: condensed; }
   label { font-width: 110%; }
   ```

   The older name `font-stretch` is accepted by CSSMedia as an alias for `font-width`. A font family may not provide every width. CSSMedia uses the requested width when matching a font face.

   `font-style`

   CSS Fonts 4 §2.4, Font style: the font-style property.

   Use `font-style` to request upright or slanted letters. The usual values are `normal`, `italic` and `oblique`.

   ```
   body { font-style: normal; }
   em { font-style: italic; }
   ```

   An `oblique` value can include an angle. The specification also defines `left` and `right` oblique directions. A font family may not contain a face for every style. CSSMedia uses this property when matching a font face.

   `font-synthesis`

   CSS Fonts 4 §2.8, Controlling synthetic faces.

   If a font family does not contain a requested face, a processor may create a synthetic version, such as making regular letters look heavier. Use `font-synthesis` to allow or disable these substitutions.

   ```
   body { font-synthesis: none; }
   ```

   none disables synthetic weight, style, small-caps and subscript or superscript faces. The keyword `weight`, `style`, `small-caps` or `position` allows that kind of synthesis. The longhand properties control those choices separately.

`font-synthesis-weight`

CSS Fonts 4 §2.8.1, Controlling synthesized bold: The font-synthesis-weight property.

This property controls whether the processor may make a font look heavier when the family lacks the requested weight. Use none to forbid that synthetic bolding, or `auto` to allow it.

`font-synthesis-style`

CSS Fonts 4 §2.8.2, Controlling synthesized oblique: The font-synthesis-style property.

This property controls whether the processor may slant a font when the family lacks the requested italic or oblique face. Use none to forbid a synthetic slant. `oblique-only` allows synthesis for oblique styles but not italic styles. `auto` allows the normal synthesis behavior.

`font-synthesis-small-caps`

CSS Fonts 4 §2.8.3, Controlling synthesized small caps: The font-synthesis-small-caps property.

This property controls whether the processor may create small capitals when the font does not provide them. Use none to forbid synthetic small capitals, or `auto` to allow them.

`font-synthesis-position`

CSS Fonts 4 §2.8.4, Controlling synthesized super- and subscripts: The font-synthesis-position property.

This property controls whether the processor may create subscript or superscript forms when the font does not provide them. Use none to forbid those synthetic forms, or `auto` to allow them.

8. Shorthand and line spacing

   `font`

   CSS Fonts 4 §2.7, Shorthand font property: the font property.

   Use `font` to set several font properties in one declaration. Write optional style, variant, weight and width values first. Then write the required size, an optional slash and line height, and the required family.

   ```
   chapter { font: italic bold 12pt/1.4 "Noto Serif", serif; }
   ```

   The size and family are required. The shorthand also accepts system font keywords such as `caption` and `menu`. It resets other properties in the shorthand to their initial values. Use separate declarations when you want to set advanced font features without resetting them.

   CSS Fonts 4 allows a user agent to substitute its default font when the requested system font is unavailable. CSSMedia uses the `system-ui` family and initial values for the other font subproperties as that fallback. It does not resolve platform-specific settings for roles such as `caption` or `menu`.

   `line-height`

   CSS Inline Layout 3 §5.1, Line Spacing: the line-height property.

   Use `line-height` to control the height of a line of text. A unitless number multiplies the font size. A length states the line height directly. A percentage is relative to the font size. `normal` lets the processor choose a value.

```
body { font-size: 12pt; line-height: 1.4; }
caption { line-height: 16pt; }
```

With a font size of `12pt`, `line-height: 1.4` requests a line height of 1.4 times the font size. A unitless number scales when the font size changes.

Font resources and descriptors

Use `@font-face` to make a font face available to a stylesheet. The rule names the family, identifies one or more sources and describes the face so the font matcher can choose it.

About this task

CSS Fonts 4 §4.1, The `@font-face` rule.

```
@font-face {
    font-family: "Example Serif";
    src: url("example-serif.woff2") format("woff2");
    font-weight: 400 700;
    font-style: normal;
    unicode-range: U+0000-00FF;
}

chapter {
    font-family: "Example Serif", serif;
}
```

The descriptors inside the rule describe the font face, not the selected XML element. Use separate rules for separate faces in the family, such as regular, bold and italic faces. A rule needs valid `font-family` and `src` descriptors. Invalid declarations are ignored. When a descriptor occurs more than once, the last valid value is used.

Procedure

1. Identify the family and source

   `font-family`

   CSS Fonts 4 §4.2, Font family descriptor.

   This descriptor gives the face the family name used by a `font-family` property. It replaces family names stored in the font file. Write the name as a quoted string or as one or more unquoted identifiers. Separate words in an unquoted name with spaces. A name that is a CSS-wide keyword must be quoted. A declared family name also takes precedence over an installed family with the same name.

   `src`

   CSS Fonts 4 §4.3, Font reference descriptor; parsing; source selection; CSS Fonts 5 `src` descriptor.

   This descriptor is a comma-separated, ordered list of sources. Each source is either a URL with optional hints or a `local()` reference. When choosing a source, a processor checks the entries in order and can skip entries whose declared format or technology it does not support.

```
@font-face {
    font-family: "Example Serif";
    src: local("Example Serif Regular"),
         url("example-serif.woff2") format("woff2"),
         url("example-serif.otf") format("opentype");
}
```

`url()` identifies a font resource. A relative URL is resolved against the stylesheet's base URL. `local()` identifies an installed font by its full name or PostScript name. It does not refer to a file. The processor tries sources in order and proceeds to the next source if it cannot find or use the current one.

`format()`

CSS Fonts 4 `format()` hint; font format definitions.

The optional `format()` hint identifies a URL resource's format. Use one format keyword or a quoted format string.

- `collection` identifies a TrueType or OpenType font collection, which can contain multiple font faces.
- `embedded-opentype` identifies Embedded OpenType resources.
- `opentype` identifies OpenType resources. It does not promise a particular glyph outline format or that the font contains OpenType layout tables.
- `svg` identifies an SVG font.
- `truetype` identifies TrueType resources.
- `woff` identifies WOFF 1.0 resources.
- `woff2` identifies WOFF 2.0 resources.

The `opentype` and `truetype` hints are treated as synonyms by CSS Fonts. The hint lets a processor avoid a resource format it cannot use. It does not convert the resource.

For compatibility with older stylesheets, CSS Fonts 4 also accepts these quoted forms: `format("woff2")`, `format("woff")`, `format("truetype")`, `format("opentype")` and `format("collection")`. They mean the corresponding unquoted format keywords.

The legacy strings `"woff2-variations"`, `"woff-variations"`, `"truetype-variations"` and `"opentype-variations"` mean the corresponding format keyword followed by `tech(variations)`.

`tech()`

CSS Fonts 4 `tech()` hint; font technology definitions.

The optional `tech()` function lists technologies required by the resource. Separate multiple values with commas. The processor can skip the source if it does not support any listed requirement.

- `features-opentype`, `features-aat` and `features-graphite` indicate support for OpenType, Apple Advanced Typography or Graphite layout features, respectively.
- `color-COLRv0` and `color-COLRv1` indicate color glyphs in the corresponding COLR table versions.
- `color-SVG` indicates SVG color glyphs.

- `color-sbix` indicates the sbix bitmap color glyph format.
- `color-CBDT` indicates the CBDT bitmap color glyph format.
- `variations` indicates support for font variation axes.
- `palettes` indicates color-font palette support.
- `incremental` indicates support for incremental font transfer.
- `avar2` indicates support for OpenType version 2 axis variations, which remap variable-font axes.

`format()` precedes `tech()` when a source uses both. The four `-variations` strings listed above each mean the corresponding format hint plus `tech(variations)`.

18. Describe which face the source provides

    `font-weight`

    CSS Fonts 4 font property descriptors; weight values.

    This descriptor tells the font matcher which weight or weight range the face provides. `auto` uses the weight information in the font. Otherwise, provide one absolute weight or two ascending endpoints. Two endpoints include every weight in the interval.

    - `normal` is `400`.
    - `bold` is `700`.
    - A number from `1` through `1000` gives the weight directly. Larger numbers mean heavier weights.

    The descriptor's range tells the matcher which requested weights this face can satisfy. It does not change the font's data.

    `font-width`

    CSS Fonts 4 font property descriptors; width values.

    This descriptor tells the matcher which width or width range the face provides. `auto` uses the width information in the font. Otherwise, provide one value or two ascending endpoints. A non-negative percentage represents glyph width relative to normal width. Values below `100%` are narrower; values above `100%` are wider.

    The named values correspond to these percentages:

    - `ultra-condensed` : `50%`.
    - `extra-condensed` : `62.5%`.
    - `condensed` : `75%`.
    - `semi-condensed` : `87.5%`.
    - `normal` : `100%`.
    - `semi-expanded` : `112.5%`.
    - `expanded` : `125%`.
    - `extra-expanded` : `150%`.
    - `ultra-expanded` : `200%`.

    The legacy descriptor name `font-stretch` is an alias for `font-width`.

`font-style`

CSS Fonts 4 font property descriptors; style values.

This descriptor tells the matcher which style or style range the face provides. `auto` uses the style information in the font.

- `normal` describes an upright face with no slant.
- `italic` describes an italic face. If one is unavailable, matching can use an oblique face.
- `left` describes an italic or oblique face with a positive, clockwise slant.
- `right` describes an italic or oblique face with a negative, counter-clockwise slant.
- `oblique` describes a slanted face. It can take one angle or two ascending endpoints from `-90deg` to `90deg`. With no angle, the value is `14deg`. Positive angles are clockwise; negative angles are counter-clockwise.

Two values define an inclusive angle range. Fractional angles are allowed.

`font-size`

CSS Fonts 5 font size descriptor.

This descriptor limits the font sizes for which the face is a match. `auto` matches any size. One number matches that size; two numbers define an inclusive range. The values are unitless numbers, not CSS lengths.

`font-named-instance`

CSS Fonts 4 `font-named-instance` descriptor.

This descriptor selects a named instance in a variable font. `auto` leaves instance selection to the normal font variation process. A string selects the first named instance whose localized name matches the string. If no such instance exists, the descriptor behaves as `auto`. The requested `font-weight`, `font-width`, and `font-style` variation values are applied before the named instance's axis values.

36. Limit a face to characters

    `unicode-range`

    CSS Fonts 4 character range descriptor.

    This descriptor gives the Unicode code points for which the face is eligible. Its value is a comma-separated list:

    - A single code point has a form such as `U+0041`.
    - A range has a form such as `U+0400-04FF`. Both endpoints are included.
    - A wildcard uses one or more trailing question marks, as in `U+4??`. Each question mark stands for one hexadecimal digit.

    Hexadecimal digits are ASCII case-insensitive. Endpoints must be valid Unicode code points, and a range's end cannot precede its start. The descriptor filters the characters for which the face can be considered. It does not guarantee that the font contains a glyph for each one. The default range includes all Unicode code points.

40. Set face-specific font behavior

    `font-display`

CSS Fonts 4 font display descriptor.

This descriptor controls the font's loading behavior while text is being displayed. The processor divides loading into a block period, when text may be invisible, and a swap period, when a loaded font may replace the fallback.

- `auto` lets the processor choose the policy.
- `block` uses an invisible-text block period, then allows the downloaded font to replace the fallback.
- `swap` shows fallback text immediately and allows the downloaded font to replace it.
- `fallback` allows a short block period and a limited swap period. A late font load does not replace the fallback.
- `optional` allows a very short block period and no swap period. The processor may use the font only when it is available promptly.

The user agent determines the durations of the block and swap periods.

`font-language-override`

CSS Fonts 4 font language override descriptor.

This descriptor sets the OpenType language-system tag used for the face's language-sensitive features. `normal` uses the language selected for the text. A string supplies a case-sensitive OpenType language-system tag, such as `"SRB"`. Find tags in the font's documentation or in Microsoft's OpenType language system tags. These are not BCP 47 tags such as `sr` or `sr-Latn`; use BCP 47 tags in `xml:lang` to identify the text's language. The OpenType tag is matched as four characters; shorter strings are padded at the end with U+0020 spaces. Unknown tags are ignored.

`font-feature-settings`

CSS Fonts 4 font feature settings descriptor; feature tag values.

This descriptor sets initial low-level OpenType feature settings for the face. It does not affect font selection. `normal` makes no explicit change to glyph selection or positioning. Otherwise, give a comma-separated list of feature tags. Each tag is a case-sensitive string of exactly four printable ASCII characters. A tag can be followed by an integer, `on` or `off` . If the value is omitted, it is `1`. on means `1`; `off` means `0`. `0` disables the feature. For a boolean feature, `1` enables it. For a non-boolean feature, an integer of `1` or greater enables it and selects that feature index. Tags that the font does not contain are ignored. See the feature tag value syntax.

`font-variation-settings`

CSS Fonts 4 font variation settings descriptor.

This descriptor sets initial low-level variation-axis settings for the face. It does not affect font selection. `normal` does not request additional axis settings. Otherwise, give a comma-separated list of case-sensitive axis tags paired with numbers, such as `"wght" 650`. An axis tag is a string of exactly four printable ASCII characters. Each number is an axis value; values outside the range supported by the font are clamped to that range. An axis tag the font does not contain is ignored. See the `font-variation-settings` property for the axis-tag rules. Prefer `font-weight`, `font-width` or `font-style` for the standard weight, width and slant axes. Those properties express the requested font characteristics and can also match non-variable faces.

46. Override font metrics

    `size-adjust`

    CSS Fonts 5 glyph size multiplier descriptor.

    This descriptor scales a face's glyph outlines and associated metrics by a non-negative percentage. `100%` leaves them unchanged. Other values scale glyphs, advances and metrics together. The CSS `font-size` remains unchanged.

    `ascent-override` , `descent-override` and `line-gap-override`

    CSS Fonts 5 font metrics override descriptors.

    `ascent-override` replaces the ascent metric. `descent-override` replaces the descent metric. `line-gap-override` replaces the line-gap metric. Each accepts `normal` or a non-negative percentage. The percentage replaces the corresponding metric with that percentage of the effective font size after adjustment. `normal` uses the metric from the font as if the descriptor were absent. Each descriptor accepts one value for the x-axis and an optional second value for the y-axis. The second value defaults to `normal`.

    Superscript and subscript metric overrides

    CSS Fonts 5 superscript and subscript metric override descriptors.

    `superscript-position-override` and `subscript-position-override` replace the corresponding glyph position metrics. `superscript-size-override` and `subscript-size-override` replace their glyph size metrics. These metrics are used when `font-variant-position` synthesizes superscript or subscript glyphs.

    Each descriptor accepts `normal`, `from-font` or a percentage. For a position descriptor, the percentage can be positive or negative. For a size descriptor, it cannot be negative.

    - `normal` lets the processor choose the metric from the font or a heuristic.
    - `from-font` uses the corresponding font metric when present. If it is absent, it behaves as `normal`.
    - A percentage replaces the metric with that percentage of the used font size.

    The descriptors accept one value for the x-axis and an optional second value for the y-axis. If the second value is omitted, the x-axis value is used. The y-axis metric is used for upright text in vertical typographic modes.

Font variants and features

Fonts can contain alternate letter shapes, number styles and ligatures. Use the properties in this guide to request those forms. The result depends on the selected font and its `font-synthesis` settings. See Font selection and sizing for the synthesis properties and their values.

Procedure

1. Common font features

   `font-kerning`

   CSS Fonts 4 §6.3, Kerning: the font-kerning property.

Kerning adjusts the space between particular pairs of letters when the font provides kerning data. Use `normal` to request kerning, `none` to turn it off, or `auto` to let the processor decide.

```
body { font-kerning: normal; }
```

`font-variant-ligatures`

CSS Fonts 4 §6.4, Ligatures: the font-variant-ligatures property.

Ligatures replace certain letter combinations with a single designed glyph. Use this property to control common, discretionary, historical or contextual ligatures.

```
body { font-variant-ligatures: common-ligatures; }
code { font-variant-ligatures: none; }
```

`normal` uses the default feature behavior. For OpenType fonts, common ligatures and contextual forms are on by default. Discretionary and historical ligatures are off by default. `none` disables all kinds of ligatures and contextual forms covered by this property.

- `common-ligatures` enables common ligatures, such as `fi` or `fl`, when the font provides them. `no-common-ligatures` disables them.
- `discretionary-ligatures` enables optional ligatures intended for particular styles. `no-discretionary-ligatures` disables them.
- `historical-ligatures` enables historical ligatures. `no-historical-ligatures` disables them.
- `contextual` enables contextual forms, where a glyph changes according to nearby characters. `no-contextual` disables them.

You can combine one value from each pair. Required ligatures used to render complex scripts are not affected by these settings.

`font-variant-caps`

CSS Fonts 4 §6.6, Capitalization: the font-variant-caps property.

Use `font-variant-caps` to request small capitals or other capitalization forms from a font.

```
term { font-variant-caps: small-caps; }
```

`normal` uses the usual letter forms. `small-caps` uses small capitals for lowercase letters. `all-small-caps` uses small capitals for both lowercase and uppercase letters. `petite-caps` uses petite capitals for lowercase letters. `all-petite-caps` uses petite capitals for both cases. `unicase` uses small capitals for uppercase letters with ordinary lowercase letters. `titling-caps` requests capitals designed for titles.

If the font does not provide the requested forms, `font-synthesis-small-caps` determines whether the processor synthesizes them.

`font-variant-position`

CSS Fonts 4 §6.5, Subscript and superscript forms: the font-variant-position property.

Use `font-variant-position` to request subscript or superscript glyph forms from the font.

```
formula { font-variant-position: sub; }
```

`normal` uses the ordinary glyphs. `sub` requests subscript glyphs. `super` requests superscript glyphs. The alternate glyphs use the same em-box and baseline as ordinary glyphs, so this property does not resize or reposition them. It does not change the meaning or position of the XML element. If the font does not provide the requested forms, `font-synthesis-position` determines whether the processor synthesizes them.

`font-variant-numeric`

CSS Fonts 4 §6.7, Numerical formatting: the font-variant-numeric property.

Use this property to request number shapes, spacing and fractions. `normal` uses the font's default forms. You may combine one value from each mutually exclusive pair:

```
table { font-variant-numeric: tabular-nums; }
```

- `lining-nums` requests figures aligned with the capital-letter height. `oldstyle-nums` requests figures with ascenders and descenders.
- `proportional-nums` requests figures whose widths vary. `tabular-nums` requests figures with equal widths.
- `diagonal-fractions` requests a numerator and denominator separated by a diagonal slash. `stacked-fractions` requests a numerator above a denominator.
- `ordinal` requests letter forms used with ordinal numbers, such as the suffix in `1st`. `slashed-zero` requests a zero glyph with a slash.

The font must provide each requested feature.

10. Alternate glyphs and language forms

    `font-variant-alternates`

    CSS Fonts 4 §6.8, Alternates and swashes: the font-variant-alternates property.

    Use `normal` to request no named alternate. Otherwise, combine one or more alternate requests:

    ```
    heading { font-variant-alternates: historical-forms; }
    ```

    `stylistic(name)` requests one named stylistic alternate. `historical-forms` requests historical glyph forms. `styleset(name, ...)` requests one or more named stylistic sets. `character-variant(name, ...)` requests named variants for individual characters. `swash(name)` requests a named swash form. `ornaments(name)` requests a named ornament. `annotation(name)` requests a named annotation form.

    Names in these functions are case-sensitive. A name can be defined for a font family by `@font-feature-values`. If it is not defined for the selected family and feature type, it has no effect. The font must support the mapped feature.

    `@font-feature-values`

    CSS Fonts 4 §6.9, Defining font specific alternates: the @font-feature-values rule.

    Use `@font-feature-values` to give readable names to alternate features that a font identifies by OpenType feature tags. Its prelude is one or more named font families

separated by commas. Generic and system font names are not allowed. Inside the block, use feature-value rules to map names to the font-specific integer indexes.

```
@font-feature-values "Example Serif" {
    @styleset {
        headings: 1;
    }
}

h1 { font-variant-alternates: styleset(headings); }
```

Feature-value rules

CSS Fonts 4 §6.9.1, Basic syntax.

Each nested rule has a declaration block. A declaration maps a case-sensitive name to a non-negative integer or integer list. The meaning and number of integers depend on the rule:

- `@stylistic` maps a name to one stylistic-alternate index for OpenType feature `salt`.
- `@historical-forms` maps a name to one or more historical-form indexes for OpenType feature `hist`.
- `@styleset` maps a name to one or more style-set indexes. Values from `1` through `99` select the corresponding `ss01` through `ss99` features. The OpenType standard defines `ss01` through `ss20`; later feature tags may be used by fonts that support them. The value `0` and values above `99` are syntactically valid but enable no OpenType feature. See CSS Fonts 4 §6.9.2.
- `@character-variant` maps a name to one or two indexes. The first selects `cv01` through `cv99`; the optional second sets that feature's variant value. A first index of `0` or greater than `99` is valid but enables no feature. See CSS Fonts 4 §6.9.2.
- `@swash` maps a name to one swash index for OpenType feature `swsh` or `cswh`.
- `@ornaments` maps a name to one ornament index for OpenType feature `ornm`.
- `@annotation` maps a name to one annotation index for OpenType feature `nalt`.

Repeated feature-value rules of the same type contribute their declarations together. If the same family, rule type and name are defined more than once, the last definition is used.

`@font-feature-values` also accepts a `font-display` descriptor. Its initial value is `auto` , which leaves the policy to the processor. The descriptor sets the default font-display policy for `@font-face` rules for the listed family when those rules do not set their own `font-display`. `block` uses a short block period, with 3 seconds recommended, followed by an infinite swap period. `swap` uses an extremely short block period, with 100 milliseconds or less recommended, followed by an infinite swap period. `fallback` uses an extremely short block period, with 100 milliseconds or less recommended, followed by a short swap period, with 3 seconds recommended. `optional` uses the font only if it is available for the first rendering. Otherwise, the block and swap periods are treated as expired, so the fallback remains in use.

`font-variant-east-asian`

CSS Fonts 4 §6.10, East Asian text rendering: the font-variant-east-asian property.

Use `normal` to request none of the variants below. You can combine a glyph-variant value, a width value and `ruby`:

```
ruby { font-variant-east-asian: ruby; }
```

- `jis78` , `jis83` , `jis90` and `jis04` request Japanese glyph forms from the corresponding JIS standard editions.
- `simplified` requests simplified-character forms. `traditional` requests traditional-character forms.
- `full-width` requests full-width glyphs. `proportional-width` requests proportionally spaced glyphs.
- `ruby` requests glyph forms designed for ruby text.

The font must provide the requested forms. Ruby-specific glyphs are designed to remain legible at the smaller sizes used for ruby text.

`font-language-override`

CSS Fonts 4 §6.13, Font language override: the font-language-override property.

Set the text's language with a BCP 47 tag in `xml:lang`. For example, `tr` identifies Turkish and `en-GB` identifies British English. Use a language tag that follows BCP 47 and its registered subtags in the IANA Language Subtag Registry.

Fonts can use OpenType language-system tags to select language-specific glyph features. Normally, the text's BCP 47 language lets the processor choose the corresponding OpenType system. Set `font-language-override` only when you need to select a different OpenType system, such as when the font lacks the system for the text's language.

```
chapter { font-language-override: "SRB"; }
```

For example, use `xml:lang="mk"` to identify Macedonian text. If the selected font lacks a Macedonian language system but provides Serbian conventions that suit the text, `"SRB"` selects its Serbian OpenType system.

`normal` uses the OpenType system inferred from the text's BCP 47 language. A `string` must be a case-sensitive OpenType language-system tag, not a BCP 47 tag. Find the tag in the font's documentation or in Microsoft's OpenType language system tags. The tag is matched as four characters; shorter strings are padded at the end with U+0020 spaces. Unknown tags are ignored. Leave the value as `normal` unless you need to override the inferred system.

`font-variant-emoji`

CSS Fonts 4 §9.3, Selecting the text presentation style: The font-variant-emoji property.

`normal` lets the processor choose text or emoji presentation according to its usual policy. `text` treats each character that can have emoji presentation as if followed by U+FE0E, the text presentation selector. `emoji` treats each such character as if followed by U+FE0F, the emoji presentation selector. `unicode` follows the Unicode default presentation for each character. An explicit text or emoji variation selector in the text overrides that default.

```
label { font-variant-emoji: text; }
```

The selected font must contain the requested glyphs and presentation.

22. Shorthand and low-level control

    `font-variant`

    CSS Fonts 4 §6.11, Overall shorthand for font rendering: the font-variant property.

    Use `font-variant` to set several variant properties in one declaration. `normal` resets every subproperty to its initial value. `none` sets `font-variant-ligatures` to none and resets the other variant subproperties. Otherwise, combine compatible values from the ligature, capitalization, alternate, numeric, East Asian, position and emoji groups described above. Omitted subproperties reset to their initial values.

    ```
    body { font-variant: small-caps; }
    table { font-variant: lining-nums tabular-nums; }
    ```

    Use a longhand such as `font-variant-numeric` when you want to control only one kind of feature. The `font` shorthand is different. It resets font variant settings rather than setting all these detailed choices. `font-variant` does not reset `font-language-override`, `font-feature-settings` or `font-variation-settings`.

    `font-feature-settings`

    CSS Fonts 4 §6.12, Low-level font feature settings control: the font-feature-settings property.

    Use `font-feature-settings` to set a specific OpenType feature directly. Write the feature tag as a four-character string, followed by an optional number, on or `off`.

    ```
    body { font-feature-settings: "liga" 0; }
    ```

    `normal` makes no change to glyph selection or positioning. Otherwise, give one or more comma-separated `<feature-tag-value>` items. Each tag is a case-sensitive string of four printable ASCII characters, from U+0020 through U+007E. The font must define the tag. Unsupported tags are ignored. This property does not synthesize their behavior, except that a processor may synthesize kerning for fonts with a legacy `kern` table.

    For each tag, omit the value or use on to enable the feature with value `1`. Use `off` or `0` to disable it. The optional integer value cannot be negative. A positive integer enables the feature and selects its indexed form when it has multiple forms. on is equivalent to `1`; `off` is equivalent to `0`. If a tag is repeated, its last value takes precedence.

    In the example, `liga` is the standard ligature feature and `0` disables it. Prefer a higher-level property such as `font-variant-ligatures` when it describes the feature you need.

Variable and color fonts

Variable fonts can change their design along axes such as weight, width or optical size. Color fonts can draw glyphs with indexed colors from a palette. The font file must provide the axis or palette that you request.

Procedure

1. Variable font controls

   `font-optical-sizing`

   CSS Fonts 4 §8.1, Optical sizing control.

An optical-size axis lets a font adjust glyph details for different text sizes. The property accepts `auto` or `none` .

```
body { font-optical-sizing: auto; }
```

With `auto`, the processor may adjust glyphs for the used font size. Variable OpenType and TrueType fonts commonly do this through the `opsz` axis. The processor must stay within the axis range in the font and must not synthesize optical sizing when the font does not provide it. With none, it must not change glyph shapes for optical size.

`font-variation-settings`

CSS Fonts 4 §8.2, Low-level font variation settings control.

Use this property to set one or more variable-font axes directly. Each item is a four-character, case-sensitive `<opentype-tag>` followed by a number. The tag must contain printable ASCII characters, from U+0020 through U+007E. The number may be fractional or negative.

```
heading { font-variation-settings: "wght" 650, "wdth" 90; }
```

`normal` means this property makes no change to glyph shape, matching or positioning. An unknown axis tag or an axis not supported by the selected font has no effect. A number outside the supported range for an axis is clamped to that range. If a tag occurs more than once, its last value is used.

Use higher-level properties for common axes. `font-weight` controls the `wght` axis, `font-width` controls `wdth`, `font-style` controls `slnt` or `ital`, and `font-optical-sizing` controls `opsz`. These properties participate in font matching; `font-variation-settings` does not.

2. Color font palettes

   `font-palette`

   CSS Fonts 4 §9.1, Controlling color font palettes.

   Use this property to choose the colors used by a color font. It accepts `normal` , `light` , `dark` , a named palette, or `palette-mix()` .

   ```
   emoji { font-palette: dark; }
   ```

   - `normal` uses the font's normal palette selection. For OpenType CPAL fonts, this is usually palette index `0`. When `color-scheme` requests a light or dark scheme, the specification says the processor should select the first matching palette.
   - `light` selects the first palette marked for light backgrounds. If the font format has no such metadata, or no palette is marked, this behaves as `normal`.
   - `dark` selects the first palette marked for dark backgrounds. If the font format has no such metadata, or no palette is marked, this behaves as `normal`.
   - A palette identifier names an author-defined palette from `@font-palette-values`. The identifier is a `<dashed-ident>` , so it starts with two hyphens, as in `--night`. If no applicable rule defines it, it behaves as `normal`.

- `palette-mix()` makes a palette by interpolating corresponding colors from one or more other palettes. Its arguments are `font-palette` values, which may be keywords, named palettes or nested `palette-mix()` functions. Each may have a percentage from `0%` to `100%`. Omitted percentages share the remaining percentage equally. If specified percentages total more than `100%`, they are normalized; if they total less, the result's opacity is reduced by the unassigned share. The rules follow CSS Color 5 §3.2 and `color-mix()` .

For example, `palette-mix(in oklab, dark 25%, --night)` blends one quarter of the dark palette with three quarters of `--night`. The optional `<color-interpolation-method>` selects the space used to blend corresponding colors. If omitted, the default is `oklab`.

- Rectangular RGB spaces include `srgb` , `srgb-linear` , `display-p3` , `display-p3-linear` , `a98-rgb` , `prophoto-rgb` and `rec2020` . The `-linear` forms interpolate light rather than gamma-encoded channel values.
- `lab` and `oklab` use lightness and two opponent-color coordinates. The `xyz` spaces use CIE XYZ tristimulus coordinates: `xyz`, `xyz-d50` and `xyz-d65` identify the reference white.
- Polar spaces represent colors with an angle and other coordinates. `hsl` uses hue, saturation and lightness. `hwb` uses hue, whiteness and blackness. `lch` and `oklch` use lightness, chroma and hue.
- A custom color space is a dashed identifier declared by `@color-profile` .
- A polar space may also specify a hue direction using `shorter` , `longer` , `increasing` or `decreasing` . `shorter` follows the shorter route around the hue circle, `longer` the longer route, `increasing` moves toward larger hue angles, and `decreasing` moves toward smaller hue angles. If no direction is written, `shorter` is used. See the color interpolation grammar for the complete syntax.

`@font-palette-values`

CSS Fonts 4 §9.2, User-defined font color palettes.

Use this rule to give a palette a name, choose its starting palette and replace selected colors. Its name must be a `<dashed-ident>`, such as `--night`. When rules use the same name, the last rule in document order takes effect.

```
@font-palette-values --night {
    font-family: "Example Emoji";
    base-palette: dark;
    override-colors: 0 navy, 1 white;
}

emoji {
    font-family: "Example Emoji";
    font-palette: --night;
}
```

The rule requires a `font-family` descriptor. It accepts one or more named font families, separated by commas. A family name may be a string or a sequence of identifiers, such as

`"Example Emoji"` or `Example Sans`. Generic family names such as `serif` are not allowed. The palette applies only to font faces whose family name matches.

`base-palette`

CSS Fonts 4 §9.2.2, Specifying the base palette.

This descriptor chooses the palette in the font on which the named palette is based.

- `light` selects the first palette marked for light backgrounds. If the font format has no such metadata, or no palette is marked, it means palette index `0`.
- `dark` selects the first palette marked for dark backgrounds. If the font format has no such metadata, or no palette is marked, it means palette index `0`.
- A non-negative integer selects a zero-based palette index. For example, `0` selects the first palette and `2` selects the third.

If this descriptor is omitted, or the font has no palette at that index, the base is palette `0`. Colors not replaced by `override-colors` remain from the base palette.

`override-colors`

CSS Fonts 4 §9.2.3, Overriding colors from a palette.

This descriptor takes a comma-separated list of palette entries and replacement colors. Each entry is a non-negative integer followed by an absolute CSS color. Palette indexes start at `0`.

```
@font-palette-values --night {
    font-family: "Example Emoji";
    base-palette: 0;
    override-colors: 0 navy, 1 rgb(255 255 255);
}
```

Each pair replaces the color at that index in the base palette. An index outside the palette's range is ignored. If an index is specified more than once, the last color is used. Every color in the descriptor must be absolute, meaning its computed value has a colorimetric meaning independent of the current text color or color mode. Named colors, hexadecimal colors, and numeric `rgb()` colors are examples. `currentColor`, system colors, `light-dark()`, `contrast-color()` and `device-cmyk()` are not absolute. A non-absolute color makes the descriptor invalid. Wrapping a relative color in `color-mix()` or relative color syntax does not make it absolute. An absolute color may use any supported color space.

Math functions, `var()` and `env()` may be used in descriptor values. They are evaluated in the root element's context. When a descriptor occurs more than once in one rule, the last valid declaration is used.

Languages and hyphenation

Document language

The language of the text decides how CSSMedia hyphenates words, which quotation marks it uses, how it changes letter case and which elements the `:lang()` selector matches. CSSMedia reads the language from the `xml:lang` attribute. If your XML names the language in another attribute, use the CSSMedia `-cssmedia-lang` property to say which one.

About this task

A language tag is a short code that names a language, such as `en` for English, `fr` for French or `zh-Hans` for Chinese in simplified characters. The `xml:lang` attribute is defined by the XML specification. It holds a language tag and applies to the element that carries it and to everything inside that element.

XML can use any element and attribute names, so CSSMedia cannot know which other attribute holds a language. A vocabulary such as DocBook 4 uses a plain `lang` attribute. Names such as `para` and `lang` below are examples. Replace them with names used in your XML.

Procedure

1. Mark the language with `xml:lang`

   When your XML already has `xml:lang` attributes, you do not write anything in the stylesheet. An element has the language of the nearest `xml:lang` on itself or on an ancestor.

   ```
   <article xml:lang="en">
       <para>This paragraph is in English.</para>
       <para xml:lang="fr">Ce paragraphe est en français.</para>
   </article>
   ```

2. Take the language from another attribute

   The `-cssmedia-lang` property is a CSSMedia extension. It is not a standard CSS property, so its name has the vendor prefix `-cssmedia-`. It sets the language of the elements it is applied to.

Value	Meaning
none	The property does not set the language of the element. This is the initial value.
`<string>`	A language tag written between quotation marks, such as `"fr"`. An empty string is the same as none.

   The property is not inherited, but the language it sets reaches the content of the element, as `xml:lang` does. Use `attr()` to read the language from an attribute:

```
<book lang="en">
    <para>This paragraph is in English.</para>
    <para lang="de">Dieser Absatz ist auf Deutsch.</para>
</book>
```

```
* {
    -cssmedia-lang: attr(lang);
}
```

`*` selects every element. `attr(lang)` reads the value of the `lang` attribute of each element. The first `para` has no `lang` attribute, so `attr()` supplies an empty string and the paragraph keeps the language of `book`. The second `para` is in German.

You can also write the language in the stylesheet:

```
quote[role="greek"] {
    -cssmedia-lang: "el";
}
```

3. Know which language wins

   For each element, CSSMedia takes the first of these that exists:

   4. The language set by `-cssmedia-lang` on the element.
   5. The `xml:lang` attribute of the element itself.
   6. The language of its parent element.

   An element with both a `lang` attribute that your rule reads and an `xml:lang` attribute takes the language from `lang`.

   A rule that sets `-cssmedia-lang` cannot depend on the language. A selector such as `:lang(fr)` or `:dir(rtl)` never matches while CSSMedia decides the language, so those selectors have no effect on the `-cssmedia-lang` declarations. They work as usual on all other properties, and they see the language that `-cssmedia-lang` gave.

Ruby and vertical text

Boxes, backgrounds and borders

Every element in your document generates a box that you can size, space and fill with color.

A box has content, padding, a border and a margin. These topics explain how to size and space boxes and how to set their backgrounds:

- Box sizing, margins and padding
- Box dimensions
- Background color and images
- Background positioning and size
- Background repetition and attachment
- Background clipping, shorthand and blending

Box sizing, margins and padding

What is a CSS box?

A CSS box is the area layout creates to place an element and its content on a page. It is not an XML tag and does not have to be drawn with a visible border. CSS uses the element's `display` value to decide whether it generates a box. Usually an element generates one box, but some display values generate none or more than one. See CSS Display 3, Box generation.

The XML element is the source that CSS selects. The box is the area that layout sizes and positions. The box's content area holds text, child boxes, images or other content. Around it, CSS can add padding, a border and margin:

- **Content** is the text or other content inside the box.
- **Padding** is space between the content and the border.
- **Border** is the edge around the padding. It is visible when border styling gives it a visible style.
- **Margin** is space outside the border, between this box and surrounding boxes.

These areas may have zero size. Their relationship is defined by CSS Box Model 4 §2, The CSS Box Model. Sizing properties set the content area's dimensions, with `box-sizing` choosing whether the declared size measures the content box or border box. Margin and padding properties set the corresponding outer and inner spaces.

The properties described here also accept the CSS-wide keywords. See Inheritance and default values.

The **containing block** is the reference box used to calculate some sizes and percentages. It is often formed by an ancestor box. Its inline size is the size along the text line. In horizontal writing, that is usually the width. See CSS Display 3, Containing Blocks and CSS Writing Modes 4, Abstract Box Terminology.

Units in size and spacing values

CSS Values 4 §5.5, Percentages; CSS Values 4 §6, Distance Units.

A length is a number followed by a unit, such as `12pt` or `1em`. The unit tells CSS how to interpret the number. A percentage, such as `60%`, is a different kind of value: it means a fraction of a

reference size. Which size it refers to depends on the property. For the sizing properties below, it is generally a corresponding size of the containing block; for margins and padding, it is based on the containing block's inline size.

Common length units used in this guide are:

Unit	Meaning
`px`	CSS pixel. CSS defines `96px` as `1in`.
`in`	Inch. `1in` equals `2.54cm` or `96px`.
`cm`	Centimeter.
`mm`	Millimeter.
`Q`	Quarter-millimeter. `4Q` equals `1mm`.
`pt`	Point. `72pt` equals `1in`.
`pc`	Pica. `6pc` equals `1in`.
`em`	Font-relative length based on the selected element's font size.
`rem`	Font-relative length based on the root element's font size.

- For example, `padding: 12pt` sets twelve points of padding on each side. `margin-block: 1em` sets a margin based on the element's font size. The same `1em` can therefore represent different lengths on elements with different font sizes.
- `60%` is not a length unit; it scales with the property's percentage reference size.
- CSS also defines other font-relative, viewport-relative and container-relative length units. See CSS Values 4 §6, Distance Units for the full list and definitions.
- A zero length may be written as `0` without a unit; nonzero lengths need a unit. The allowed range depends on the property. For example, margins may be negative, but padding cannot.

Choose what a size measures

CSS Sizing 3 §3, Specifying Box Sizes.

`box-sizing`

`box-sizing` determines whether a fixed size measures the content box or the border box.

- `content-box` measures the content area. Padding and border are added outside the specified size.
- `border-box` measures through the outside border edge. The content area is reduced to make room for padding and border.

The initial value is `content-box` .

For example, start with an element in your XML that you want to size:

```
<notice>Keep this information with the procedure.</notice>
```

The selector `notice` matches that element name. With `content-box`, `inline-size` measures the content area. Padding and border are added outside it. With `border-box`, the specified size includes the padding and border:

```
notice {
    inline-size: 12em;
    padding: 1em;
    border: 1px solid black;
    box-sizing: content-box;
}
```

Here, the content area is `12em` wide. The padding adds `1em` on each inline side, and the border adds another `1px` on each side. Change `box-sizing` to `border-box` to keep the total border-edge size at `12em`; the content area becomes narrower to make room for the padding and border. The `padding` and `border` declarations make the edges visible in this example.

Preferred width, height, minimum and maximum dimensions are covered in Box dimensions.

Set margins outside the border

CSS Box Model 4 §3, Margins; CSS Logical Properties 1 §4.2, Flow-Relative Margins.

Physical margin properties

`margin-top` , `margin-right` , `margin-bottom` and `margin-left` set the space outside each physical side of the border.

Each accepts a length such as `12pt`, a percentage such as `5%`, `auto`, or `anchor-size()` . Negative lengths and percentages are allowed. A percentage is calculated from the inline size of the containing block, including for top and bottom margins. `auto` lets the layout algorithm distribute available space according to the formatting context.

`anchor-size()` accepts an optional anchor name, an optional size keyword, or both, followed by an optional length or percentage fallback. The size keyword can be `width`, `height`, `block`, `inline`, `self-block` or `self-inline`. The fallback is used when the anchor cannot be resolved.

Each physical margin longhand starts at `0`.

Adjacent block margins can collapse into one margin. Their used separation is not always the sum of both values.

`margin` shorthand

`margin` sets one to four physical side values. One value applies to all sides. Two values set top and bottom, then left and right. Three set top, left and right, then bottom. Four set top, right, bottom and left, clockwise from the top. Its initial value is `0`.

```
section { margin: 1em 2em; }
```

In this declaration, the section has `1em` of margin above and below it, and `2em` to its left and right. Replace `section` with the name of an element in your XML. The shorthand changes the space outside the border; it does not add space between the content and the border.

Logical margin properties

`margin-block-start` and `margin-block-end` set margins at the start and end of the block axis. `margin-inline-start` and `margin-inline-end` set margins at the start and end of the inline axis. Their sides follow `writing-mode` and `direction`.

`margin-block` sets the block-start and block-end margins. `margin-inline` sets the inline-start and inline-end margins. With one value both sides are equal; with two, the first sets the start side and the second the end side.

The four logical longhands accept the same values as the physical margin properties and start at `0`. Each logical shorthand accepts one or two values from that same grammar. The shorthand's initial values come from its longhands, each of which starts at `0`.

For example, a `step` element can have different spacing before and after it, and equal spacing at both inline sides:

```
<procedure>
    <step>Close the cover.</step>
    <step>Check the indicator.</step>
</procedure>
```

```
step {
    margin-block: 0.5em 1em;
    margin-inline: 1em;
}
```

`margin-block` gives each step `0.5em` at block-start and `1em` at block-end. `margin-inline` gives it `1em` at both inline sides. In the usual horizontal, left-to-right writing mode, these are top, bottom, left and right respectively. In another writing mode or direction, the logical sides follow that mode instead of staying tied to the same physical edges. Replace `step` with the element name used for your procedure steps.

`margin-trim`

`margin-trim` removes in-flow child margins that touch the corresponding edge of a container.

Its initial value is `none` .

- `none` does not trim margins. Normal margin collapsing can still occur.
- `block` trims block-start and block-end margins at the container's edges.
- `inline` trims inline-start and inline-end margins.
- `block-start` , `block-end` , `inline-start` and `inline-end` select individual edges.

You can combine `block` with `inline`, or combine individual edge keywords with one another. Do not mix these two groups in one value. The property applies to block, multi-column, flex and grid containers.

`margin-break`

`margin-break` controls the box's block-axis margins where they adjoin a page or column break. See CSS Fragmentation 4 §5.2.

- `auto` is the initial value. At an unforced break, margins adjoining the break are limited to the remaining space before the break and become zero after it. At a forced break, margins before the break are truncated, while margins after it are preserved. Cloned margins are always truncated to zero.

- `keep` preserves margins adjoining the break.
- `discard` truncates adjoining margins, including at the beginning and end of a fragmentation context.

When margins collapse, margins from more than one box can adjoin the same break.

For example, this rule removes a chapter's margins where they meet a page or column break:

```
chapter {
    margin-break: discard;
}
```

Set padding inside the border

CSS Box Model 4 §4, Padding; CSS Logical Properties 1 §4.4, Flow-Relative Padding.

Physical padding properties

`padding-top` , `padding-right` , `padding-bottom` and `padding-left` set the space between the content and the corresponding border edge.

Each accepts a non-negative length, such as `12pt`, or percentage, such as `5%`. A percentage is calculated from the inline size of the containing block, including for top and bottom padding. Padding cannot be negative.

Physical and logical padding longhands and shorthands have an initial value of `0`.

`padding` shorthand

`padding` sets one to four physical side values using the same top, right, bottom, left order as `margin`. One value applies to all sides; two set vertical then horizontal sides; three set top, horizontal sides, then bottom; four set each side clockwise.

For example, a `warning` element can have more room inside its border at the top and bottom than at the sides:

```
<warning>Disconnect the power before servicing.</warning>
```

```
warning {
    padding: 1em 1.5em;
}
```

The first value adds `1em` above and below the content. The second adds `1.5em` to the left and right. Padding is inside the border, unlike margin. Replace `warning` with the element name used for this kind of message in your XML.

Logical padding properties

`padding-block-start` and `padding-block-end` set padding at the start and end of the block axis. `padding-inline-start` and `padding-inline-end` set padding at the start and end of the inline axis.

`padding-block` sets both block-axis sides. `padding-inline` sets both inline-axis sides. One value applies to both sides; two values set the start side followed by the end side. The physical sides change with the writing mode and text direction.

Each logical padding longhand uses the same non-negative length-percentage values as the physical padding properties. Each logical shorthand accepts one or two values from the same grammar.

Use logical padding when the spacing should follow the text direction. In this example, `padding-inline-start` adds space at the beginning of the text line:

```
<definition>Meaning of the selected term.</definition>
```

```
definition {
    padding-block: 0.5em;
    padding-inline-start: 1em;
}
```

The block padding adds `0.5em` at both ends of the block axis. The inline-start padding adds `1em` before the text begins. In a right-to-left writing direction, inline-start is on the right. Replace `definition` with the element name used for this content in your XML.

Property reference

See the box model property reference for the generated value grammars, initial values, specification links, and implementation-check status.

Box dimensions

CSS sizing properties set preferred dimensions or place minimum and maximum limits on them. They apply to CSS boxes, not directly to XML tags. Select an element from your XML, then use the property that matches the dimension or limit you need.

For the meaning of CSS boxes and the length units used below, see Box sizing, margins and padding.

These properties also accept the CSS-wide keywords. See Inheritance and default values.

The `Values` entries describe CSS syntax. CSS Anchor Positioning defines `anchor-size()` for physical sizing properties, not their logical counterparts.

Preferred sizes

CSS Sizing 3 §3, Specifying Box Sizes; CSS Logical Properties 1 §4.1, Logical Height and Logical Width.

`width` and `height` refer to the physical axes. `inline-size` and `block-size` refer to the text's inline and block axes. Their physical direction follows `writing-mode`.

Name	Description	Values	Default	Specification
`width`	Sets the preferred width of a box.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3

Name	Description	Values	Default	Specification
`height`	Sets the preferred height of a box.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`inline-size`	Sets the preferred size along the inline axis.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, or `calc-size()`.		CSS Sizing 3
`block-size`	Sets the preferred size along the block axis.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, or `calc-size()`.		CSS Sizing 3

- `<length-percentage>` means a length such as `12pt`, a percentage such as `60%`, or a `calc()` expression combining them.
- `[0,∞]` in `fit-content(<length-percentage [0,∞]>)` means the length or percentage cannot be negative.
- The `fit-content()` and `calc-size()` functions are defined in CSS Sizing 4 §3.2.
- `auto` lets the layout algorithm determine the size.
- `min-content` uses the smallest size the content can take without avoidable overflow.
- `max-content` uses the size the content would take without soft wrapping.
- `fit-content` fits the content within available space, bounded by its min-content and max-content sizes.
- `fit-content(<length-percentage>)` uses the supplied non-negative length or percentage as the available-space limit.
- `stretch` sizes the margin box to fill the available space in that axis.
- `contain` uses contain-fit sizing. It fills the available space while preserving a preferred aspect ratio when one exists.
- `calc-size()` calculates a size from an intrinsic-size basis. The basis can be an intrinsic keyword, another `calc-size()`, any, or a calculation. The second argument is a calculation using that basis.
- – `anchor-size()` accepts an optional anchor name, an optional size keyword, or both, followed by an optional length or percentage fallback. Its size keyword can be `width`, `height`, `block`, `inline`, `self-block` or `self-inline`. The function is for absolutely positioned boxes.
- A percentage size is relative to the corresponding dimension of the containing block. The allowed length and percentage forms are described in CSS Values 4 §5.5 and §6.

For example, if your XML contains a `chapter` element and a `figure` inside it, these rules request a 30em inline size for the chapter and a 60 percent inline size for the figure:

```
<chapter>
    <figure>Figure description and content.</figure>
</chapter>
```

```
chapter {
    inline-size: 30em;
}

figure {
    inline-size: 60%;
    block-size: fit-content;
}
```

Replace the example element names with names from your XML. If your document uses a vertical writing mode, inline and block dimensions map to different physical directions.

Minimum and maximum sizes

CSS Sizing 3 §3, Specifying Box Sizes; CSS Sizing 4 §3.1, Sizing Properties; CSS Logical Properties 1 §4.1.

Minimum sizes set a lower limit. Maximum sizes set an upper limit. The layout algorithm applies the limits to the preferred size.

Name	Description	Values	Default	Specification
`min-width`	Sets a lower limit for the width.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`min-height`	Sets a lower limit for the height.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`min-inline-size`	Sets a lower limit along the inline axis.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, or `calc-size()`.		CSS Sizing 3
`min-block-size`	Sets a lower limit along the block axis.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, or `calc-size()`.		CSS Sizing 3

Name	Description	Values	Default	Specification
`max-width`	Sets an upper limit for the width.	none, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`max-height`	Sets an upper limit for the height.	none, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`max-inline-size`	Sets an upper limit along the inline axis.	none, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, or `calc-size()`.		CSS Sizing 3
`max-block-size`	Sets an upper limit along the block axis.	none, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, or `calc-size()`.		CSS Sizing 3

- `<length-percentage>` means a length such as `12pt`, a percentage such as `60%`, or a `calc()` expression combining them.
- `[0,∞]` in `fit-content(<length-percentage [0,∞]>)` means the length or percentage cannot be negative.
- `auto` on a minimum size lets the layout mode choose its automatic minimum. none on a maximum size means there is no maximum limit.
- For example, `max-inline-size` prevents a wide illustration from exceeding `40em`, while `inline-size: 100%` lets it use the containing block's full inline size when there is room:

```
<illustration>Diagram of the assembly.</illustration>
```

```
illustration {
    inline-size: 100%;
    max-inline-size: 40em;
}
```

Paired size shorthands

CSS Sizing 4 §3.1, Sizing Properties.

Name	Description	Values	Default	Specification
`size`	Sets both the preferred width and height.	One or two space-separated values. One value sets both dimensions; two set width then height.		CSS Sizing 4
`min-size`	Sets the minimum width and height.	One or two space-separated values. One value sets both dimensions; two set width then height.		CSS Sizing 4
`max-size`	Sets the maximum width and height.	One or two space-separated values. One value sets both dimensions; two set width then height.		CSS Sizing 4

- Each shorthand accepts one or two space-separated values. One value applies to both dimensions. Two values set width then height. Each value uses the grammar of its corresponding longhand.
- For `aspect-ratio` , the `contain-intrinsic-*` properties, `frame-sizing` , and `min-intrinsic-sizing` , see the complete dimension property reference.

Background color and images

A background can have a solid color, one or more images, or both. These properties describe the color and image layers of an element's background.

Procedure

1. `background-color`

   "CSS Backgrounds 4 §2.1, Base Color: the background-color property".

   Use `background-color` to set a solid color behind an element's content. Write a CSS color value, such as a named color, a hexadecimal color or an `rgb()` value.

   ```
   chapter {
       background-color: #f2f2f2;
   }
   ```

   The CSS initial value is `transparent`. In CSS, the background color is painted beneath the element's background images.

2. `background-image`

   "CSS Backgrounds 4 §2.2, Image Sources: the background-image property".

   Use `background-image` to specify images behind an element's content. A value can be a `url()` image, a gradient or none. Separate multiple layers with commas.

   ```
   chapter {
       background-image: url("paper.png"), linear-gradient(white, #ddd);
   }
   ```

The first image in the list is the top layer. Later images are below it, and the background color is below all image layers. Use none when a layer should not show an image.

Background positioning and size

These CSS properties specify where a background image is placed, how large it is and which box is used as its positioning area.

Procedure

1. `background-position`

   "CSS Backgrounds 4 §2.7, Background Positioning Shorthand: the background-position shorthand property".

   Use `background-position` to specify each background image's position inside its positioning area. A position can use keywords, lengths or percentages. With two values, the first sets the horizontal position and the second sets the vertical position.

   ```
   chapter {
       background-position: right 1em bottom 2em;
   }
   ```

   The CSS initial value is `0% 0%`, which places the image at the top-left. A percentage is calculated from the difference between the size of the positioning area and the size of the image. For example, `100% 100%` aligns the image's bottom-right corner with the area's bottom-right corner.

2. `background-position-x`, `background-position-y`, `background-position-inline` and `background-position-block`

   "CSS Backgrounds 4 §2.6, Background Positioning: the background-position-x, background-position-y, background-position-inline, and background-position-block properties".

   Use these properties to specify a position on one axis. `background-position-x` and `background-position-y` use the physical horizontal and vertical axes. `background-position-inline` and `background-position-block` use the logical axes, which follow the element's writing mode.

   ```
   chapter {
       background-position-x: right 1em;
       background-position-block: end 2em;
   }
   ```

   The physical properties use values such as `left`, `right`, `top` and `bottom`. The logical properties use `start` and end. Each also accepts `center`, a length or a percentage.

   CSSMedia expands `background-position` into physical or logical axis values and also expands the `background` shorthand's position.

3. `background-size`

   "CSS Backgrounds 4 §2.10, Sizing Images: the background-size property".

Use `background-size` to specify the width and height of each background image. Use one or two non-negative lengths or percentages. If you give one length or percentage, the other dimension defaults to `auto`.

```
chapter {
    background-size: 12em auto;
}
```

`auto` uses the image's intrinsic size or proportion. `cover` scales the image until it covers the positioning area, possibly cropping part of it. `contain` scales it until the whole image fits, possibly leaving part of the area uncovered. Separate sizes for multiple image layers with commas.

4. `background-origin`

   "CSS Backgrounds 4 §2.9, Positioning Area: the background-origin property".

   Use `background-origin` to specify the box that defines the positioning area for background images.

   ```
   chapter {
       background-origin: content-box;
   }
   ```

   `border-box` uses the area inside the outer border edges, `padding-box` uses the area inside the outer padding edges, and `content-box` uses the content area. The CSS initial value is `padding-box`. This property does not affect an image whose `background-attachment` is `fixed`.

Background repetition and attachment

These CSS properties specify whether a background image repeats to fill its area and what the image is attached to.

Procedure

1. `background-repeat`

   "CSS Backgrounds 4 §2.4, Tiling Images Shorthand: the background-repeat property".

   Use `background-repeat` to specify whether an image repeats horizontally, vertically, both or neither. One value applies to both axes. Two values set the horizontal and vertical behavior, in that order.

   ```
   chapter {
       background-repeat: no-repeat;
   }

   table {
       background-repeat: repeat no-repeat;
   }
   ```

   Use `repeat` to tile the image, `no-repeat` to show it once, `space` to distribute whole copies with space between them, or `round` to resize the copies so they fit evenly. `repeat-x` and `repeat-y` repeat on one physical axis only. `repeat-block` and `repeat-inline` repeat on

the logical block or inline axis, respectively, and do not repeat on the other axis. Separate values for multiple image layers with commas.

CSSMedia expands `background-repeat` into its physical or logical axis properties.

2. `background-repeat-x`, `background-repeat-y`, `background-repeat-inline` and `background-repeat-block`

   "CSS Backgrounds 4 §2.3, Tiling Images: the background-repeat-x, background-repeat-y, background-repeat-block, and background-repeat-inline properties".

   Use these properties to specify repetition on one axis. The `x` and `y` properties use the physical horizontal and vertical axes. The `inline` and `block` properties use the logical axes, which follow the element's writing mode.

   ```
   chapter {
       background-repeat-x: repeat;
       background-repeat-block: no-repeat;
   }
   ```

   Each accepts `repeat`, `no-repeat`, `space` or `round`. Use the shorthand when you want to set both axes together.

3. `background-attachment`

   "CSS Backgrounds 4 §2.5, Affixing Images: the background-attachment property".

   Use `background-attachment` to specify how a background image is positioned in relation to its box or its contents. In paged media, CSS defines `fixed` relative to the page box, so that the background is repeated on each page.

   ```
   chapter {
       background-attachment: fixed;
   }
   ```

   In paged media, `fixed` attaches the image to the page box. CSSMedia's PDF default is `fixed`. Separate values for multiple image layers with commas.

Background clipping, shorthand and blending

These CSS properties specify the area where a background is painted, combine background settings in one declaration, and control how image layers combine.

Procedure

1. `background-clip`

   "CSS Backgrounds 4 §2.8, Painting Area: the background-clip property".

   Use `background-clip` to specify how far a background color or image extends across an element's box.

   ```
   chapter {
       background-clip: content-box;
   }
   ```

`border-box` allows the background to extend beneath the border, `padding-box` clips it at the outside edge of the padding, and `content-box` clips it to the content area. `border-area` clips it to the area covered by the border. `text` clips it to the shape of the text glyphs. `border-area` and `text` can be combined. The CSS initial value is `border-box`. Separate values for multiple image layers with commas.

2. `background`

   "CSS Backgrounds 4 §2.12, Backgrounds Shorthand: the background property".

   Use `background` to specify several background properties in one declaration. For each layer, you can provide an image, position and size, repeat style, attachment and clipping or positioning boxes. Set the background color in the final layer.

   ```
   chapter {
       background: #f2f2f2 url("paper.png") center / cover no-repeat;
   }
   ```

   The slash separates `background-position` from `background-size`. A box value can set both the positioning and painting areas. Two box values set the positioning area first and the painting area second. The CSS shorthand resets its longhand properties to their initial values when you leave them out. It also resets `background-blend-mode` to `normal`.

3. `background-blend-mode`

   "Compositing and Blending 1 §3.4.3, The background-blend-mode property".

   Use `background-blend-mode` to specify how each background image layer combines with the layer below it and the element's background color.

   ```
   chapter {
       background-image: url("texture.png"), linear-gradient(white, gray);
       background-blend-mode: multiply, normal;
   }
   ```

   The available modes are `normal`, `multiply`, `screen`, `overlay`, `darken`, `lighten`, `color-dodge`, `color-burn`, `hard-light`, `soft-light`, `difference`, `exclusion`, hue, `saturation`, `color` and `luminosity`. The first value applies to the top image layer. If the list has fewer values than the image list, the values repeat. The CSS initial value is `normal`.

Shadows, opacity, transforms and masks

Multi-column layout

Flexbox and grid

Images, SVG and MathML

Tables

Cascade and declarations

CSS rules associate selectors with declarations. A declaration has a property name, a colon, a value and a semicolon:

```
chapter {
    color: navy;
    margin: 1em;
}
```

The **cascade** is the process that decides which declaration sets a property on an element when several declarations apply to it. It compares the declarations one criterion at a time and keeps the winner.

For each element and property, the cascade chooses among the declarations that apply. It does not choose one declaration for an entire rule. One rule can win for `color` while another wins for `margin`.

From a declaration to a value

CSS Cascade 5 §4, Value Processing; CSS Cascade 5 §5, Filtering.

CSS processes a property through several value stages:

1. A declared value is the value written in a declaration.
2. A cascaded value is the result of comparing the applicable declarations.
3. A specified value resolves CSS-wide keywords and uses inheritance or the property's initial value when needed.
4. A computed value resolves values that can be determined without knowing the final layout.
5. A used value incorporates information available during layout.
6. An actual value is the value that can be represented in the output medium.

The cascade concerns the first two stages. Inheritance and defaulting follow the cascade. Computation and layout follow those stages.

Declarations are filtered before they compete. A rule must apply to the element, its selector must match, and its enclosing conditions must hold. A declaration with an unknown property or invalid value does not provide a candidate. A value that becomes invalid only after substitution is different: it fails after the cascade has already chosen it. See Inheritance and default values and Custom properties and substitution.

CSSMedia processes declarations from its user-agent rules and author stylesheets. The cascade guides explain the CSS rules used to select values from those declarations.

Guides in this topic

- Cascade order and layers: importance, origins, layers, specificity, scope proximity and source order.
- Inheritance and default values: inherited values, initial values and CSS-wide keywords.
- Shorthand properties: how shorthand declarations set and reset longhands.
- Custom properties and substitution: custom properties, `var()`, `attr()` and `@property`.
- Conditional rules: when rules participate in the cascade and how imported stylesheets are ordered.

Cascade order and layers

When declarations set the same property on the same element, CSS sorts them by precedence. It compares one criterion at a time. The first criterion that differs decides which declaration wins.

Sorting declarations

CSS Cascade 6 §2.1, Cascade Sorting Order; CSS Cascade 5 §6.1, Cascade Sorting Order; Selectors 4 §15, Calculating a selector's specificity.

The cascade compares declarations in this order:

1. Origin and importance.
2. Encapsulation context.
3. Element-attached style declarations.
4. Cascade layer order.
5. Selector specificity.
6. Scope proximity.
7. Source order.

Each step matters only when the declarations tie at every preceding step.

Origin and importance

CSS Cascade 5 §6.1, Cascade Sorting Order.

An origin identifies the source of a declaration. CSS defines user-agent, user, and author origins, plus animation and transition origins. Within the author, user, and user-agent origins, important declarations and normal declarations have different priorities. From highest to lowest, the CSS cascade order is:

Priority	Declaration
1	Transition
2	Important user-agent
3	Important user
4	Important author
5	Animation
6	Normal author
7	Normal user
8	Normal user-agent

- `!important` follows the value and raises that declaration above normal declarations in its origin. It does not make a declaration universally unbeatable. A higher-priority origin can still win.
- CSSMedia's stylesheet input consists of author stylesheets and built-in user-agent rules. It does not load a user-origin stylesheet.

Encapsulation context

CSS Cascade 6 §2.1, Cascade Sorting Order.

Some host environments divide a document into nested encapsulation contexts, such as shadow trees. For normal declarations, the outer context wins a tie against an inner context. For important declarations, the inner context wins. This comparison occurs after origin and importance, and before layers.

Element-attached styles

CSS Cascade 6 §2.1, Cascade Sorting Order; CSS Style Attributes §2, CSS Style Attributes.

When a host language defines a style attribute, declarations attached directly to an element take precedence over declarations from style rules at the same origin and importance. This rule is separate from selector specificity.

Cascade layers

CSS Cascade 5 §6.4, Cascade Layers; CSS Cascade 5 §6.4.3, Layer Ordering.

`@layer` groups declarations within an origin. A layer's order is established when its name first appears. A layer statement establishes order without adding rules. A layer block establishes a layer and contains its rules. Layers can be nested; a dotted name such as `components.buttons` identifies a sublayer.

For normal declarations, later layers outrank earlier layers. Unlayered declarations form an implicit final layer and outrank normal declarations in every named layer. For important declarations, this order is reversed: earlier layers outrank later layers, and layered important declarations outrank unlayered important declarations.

Within a parent layer, declarations assigned directly to the parent are ordered after its sublayers for normal declarations. Important declarations reverse that order as well.

```
@layer base, theme;

@layer base {
    chapter { color: black; }
}

@layer theme {
    chapter { color: navy; }
}

chapter { color: gray; }
```

All three declarations are normal and have the same specificity. The unlayered `gray` declaration wins. If all three declarations were important, the `base` layer would win.

Specificity

Selectors 4 §15, Calculating a selector's specificity.

Specificity compares the selector that matched. It is a three-part weight: ID selectors, class/attribute/pseudo-class selectors, and type/pseudo-element selectors. Compare the parts from left to right. A larger earlier part takes precedence, regardless of the later parts.

For a selector list, the matching selector with the greatest specificity supplies the rule's specificity. Specificity does not override origin, importance or layer order. See Specificity for how each selector form contributes.

Scope proximity

CSS Cascade 6 §2.5, Scoping Styles: the @scope rule.

When declarations tie on origin, importance, layer and specificity, a declaration from a scope whose root is closer to the matched element wins. Count the ancestor steps from the element to each matching scope root. The smaller count is closer. An unscoped rule has no scope root and loses this tie-breaker to a scoped rule.

Source order

CSS Cascade 6 §2.1, Cascade Sorting Order.

If declarations still tie, the one that appears later wins. Imported rules take part at the position of their `@import`. Within one declaration block, the later declaration for the same property wins.

Scoped rules

CSS Cascade 6 §2.5, Scoping Styles: the @scope rule.

`@scope` limits the elements matched by its nested rules. The optional start selector identifies the scope root. The optional `to` selector identifies a scope limit: matching elements at the limit, and their descendants, are outside that scope.

```
@scope (chapter) to (appendix) {
    title { color: navy; }
}
```

This rule matches `title` elements inside a `chapter` scope, but not inside an `appendix` boundary. If the start selector is omitted, the document root is the scope root. A nested scope is relative to its enclosing scope.

Scope proximity affects precedence only after origin, importance, layer and specificity tie. It does not make a less-specific selector outrank a more-specific one.

Page-context declarations

CSS Paged Media 3 §4.4, Cascading in the page context; CSS GCPM 3 §3.1, Page Selectors; §3.2, Page groups.

Declarations in `@page` rules and page-margin rules cascade within their page context. Page selectors have their own specificity. A page type name contributes to the first component; `:first` and `:blank` contribute to the second; `:left` and `:right` contribute to the third. Each occurrence counts, including repeated pseudo-classes. The `:nth()` page pseudo-class contributes no specificity. It selects a page by its position in the document or, with a page-group name, its position in that group. When matching page rules tie on origin, importance, layer and specificity, source order decides. These declarations do not compete with declarations for an XML element.

Inheritance and default values

The cascade selects a cascaded value for a property on an element. CSS then determines the property's specified value. If no declaration supplies a cascaded value, inheritance or the property's initial value provides the default.

Inheritance

CSS Cascade 5 §7.2, Inheritance.

Some properties inherit automatically. Their definitions say whether they are inherited. A child element uses the parent's computed value unless a declaration gives the child another value.

```
chapter { color: navy; }
title { color: teal; }
```

The title uses teal. Other descendants inherit navy. A property that does not normally inherit uses its initial value when it has no cascaded value.

Inheritance follows the XML element tree. It is not determined by whether the property describes text, a box, or another feature.

CSS-wide keywords

CSS Cascade 5 §7.3, Explicit Defaulting.

CSS-wide keywords can be used as the complete value of any property. They affect defaulting after the cascade has selected the declaration.

`initial`

CSS Cascade 5 §7.3.1, Resetting a Property: the initial keyword.

`initial` selects the property's initial value from its definition. It does not mean the value inherited from the parent, even for a property that normally inherits.

```
title { color: initial; }
```

`inherit`

CSS Cascade 5 §7.3.2, Explicit Inheritance: the inherit keyword.

`inherit` uses the parent's computed value, whether or not the property normally inherits. At the root, where there is no parent, it uses the initial value.

```
title { margin-left: inherit; }
```

`unset`

CSS Cascade 5 §7.3.3, Erasing All Declarations: the unset keyword.

`unset` acts like `inherit` for a property that normally inherits. It acts like `initial` for a property that does not normally inherit.

```
chapter { color: unset; }
title { margin-left: unset; }
```

The first declaration follows the normal inheritance behavior of `color`. The second uses the initial value of `margin-left`.

`revert`

CSS Cascade 5 §7.3.4, Rolling Back Cascade Origins: the revert keyword.

`revert` removes declarations from the winning declaration's origin for that property and runs the cascade again. For an author declaration, this means the author-origin value is rolled back so a lower origin can supply a value. If no declaration remains, normal defaulting and inheritance apply.

```
title { color: navy; }
title { color: revert; }
```

`revert-layer`

CSS Cascade 5 §7.3.5, Rolling Back Cascade Layers: the revert-layer keyword.

`revert-layer` removes declarations from the winning declaration's layer at that origin and reruns the cascade. A declaration in a lower-priority layer can then win. If no declaration remains in that origin, the cascade can fall back to a lower origin.

```
@layer base, theme;

@layer base {
    title { color: black; }
}

@layer theme {
    title { color: revert-layer; }
}
```

The `theme` declaration rolls back to the `base` layer's value.

The `all` property

CSS Cascade 5 §3.2, Resetting All Properties: the all property.

The `all` property accepts a CSS-wide keyword and applies it to nearly every property. It does not affect `direction`, `unicode-bidi`, or custom properties.

```
title { all: unset; }
```

This uses each property's normal `unset` behavior. Inherited properties inherit; other properties use their initial values. Use `all` carefully because it can reset declarations from many unrelated property groups.

A declaration invalid after the cascade

CSS Variables 1 §2.3, Resolving Dependency Cycles; CSS Variables 1 §3.1, Invalid Variables.

A value can pass the declaration's initial syntax check and become invalid later, for example when a `var()` reference has no usable value or substitution produces text that does not match the property's grammar. The cascade has already selected that declaration. CSS does not return to an earlier, lower-priority declaration for that property; the invalid-at-computed-value-time declaration acts as `unset`.

This differs from a declaration that is invalid before it enters the cascade. An invalid declaration is discarded, so another valid declaration can win.

Shorthand properties

A shorthand sets several related longhand properties in one declaration. The longhands are the properties that hold the individual values.

Shorthands set every sub-property

CSS Cascade 5 §3, Shorthand Properties.

A shorthand sets all of its sub-properties, including properties whose values are omitted from the shorthand. Unless the shorthand's definition says otherwise, each omitted sub-property receives its initial value.

For example, `margin` sets the four side margins:

```
chapter { margin: 1em; }
chapter { margin: 1em 2em; }
chapter { margin: 1em 2em 3em 4em; }
```

One value sets all four sides. Two values set the top and bottom margins, then the right and left margins. Three values set top, horizontal sides and bottom. Four values set top, right, bottom and left, respectively.

Some shorthands also reset sub-properties that do not appear in their syntax. Such a sub-property is reset-only. For example, the `border` shorthand resets border-image properties even though its syntax cannot set their other values. Consult each shorthand's property definition to see its full set of sub-properties and reset behavior.

Shorthand values and importance

CSS Cascade 5 §3, Shorthand Properties.

The shorthand is expanded into its longhands at the location of its declaration. Those longhands then compete in the cascade like declarations written separately.

```
chapter {
    margin-left: 2em;
    margin: 1em;
}
```

The `margin` shorthand comes later and sets `margin-left` to `1em`. If a later declaration sets `margin-left` directly, that later longhand can win.

An invalid shorthand contributes no longhand declarations. If a shorthand is declared `!important`, its sub-properties are important too. A CSS-wide keyword used as the shorthand's entire value applies to every sub-property, including reset-only sub-properties. A CSS-wide keyword cannot be combined with other shorthand values.

Resetting properties with `all`

CSS Cascade 5 §3.2, Resetting All Properties: the all property.

`all` is a shorthand that sets almost every property to one CSS-wide keyword. It excludes `direction`, `unicode-bidi` and custom properties.

```
title { all: initial; }
```

This sets the included properties to their initial values. Use `inherit`, `unset`, `revert` or `revert-layer` to apply the corresponding CSS-wide behavior to those properties. See Inheritance and default values for the meaning of each keyword.

Shorthand definitions

Shorthands do not all use the same value order or number of values. Their property definitions specify the grammar, defaults for omitted values and reset-only sub-properties. The guides for backgrounds, fonts and other property groups describe the shorthands in those groups.

Custom properties and substitution

Custom properties store values that can be substituted into other declarations. The cascade selects their values like it selects values for other properties.

Procedure

1. Defining custom properties

   CSS Variables 1 §2, Defining Custom Properties: the --* family of properties.

   A custom property name is a dashed identifier that begins with two hyphens, such as `--accent`. Names are case-sensitive. An unregistered custom property accepts a sequence of CSS component values rather than a property's normal value grammar.

   ```
   chapter {
       --accent: navy;
       --space: 1em;
   }
   ```

   Custom properties participate in the cascade. Unregistered custom properties inherit by default, and their initial value is the guaranteed-invalid value. This value is different from an empty string and cannot be substituted unless a fallback is provided.

2. Substituting values with `var()`

   CSS Variables 1 §3, Using Cascading Variables: the var() notation; CSS Variables 1 §2.3, Resolving Dependency Cycles; CSS Variables 1 §3.1, Invalid Variables.

   Use `var()` with a custom property name. An optional fallback follows a comma:

   ```
   title {
       color: var(--accent, black);
       margin-left: var(--space, 0);
   }
   ```

   The fallback is used when the referenced custom property has the guaranteed-invalid value. A fallback can contain another `var()` call. The fallback is not used merely because the substituted text fails the consuming property's grammar.

   Custom properties can refer to one another. CSS resolves their dependencies for each element. A cycle makes every custom property in that cycle invalid on that element. Descendants inherit the parent's computed custom-property values; they do not re-evaluate the parent's declarations in the descendant's context.

   Substitution happens after the cascade. If the resulting value does not match the consuming property's grammar, the winning declaration becomes invalid at computed-value time and acts as `unset`. The cascade does not select an earlier declaration instead.

3. Registering a custom property with `@property`

CSS Properties and Values API 1 §3, The @property Rule; §3.1, The syntax Descriptor; §3.2, The inherits Descriptor; §3.3, The initial-value Descriptor.

`@property` registers a custom property with a syntax, an inheritance setting and, for typed syntax, an initial value:

```
@property --accent {
    syntax: "<color>";
    inherits: true;
    initial-value: navy;
}
```

The `syntax` descriptor is a string containing the property's accepted grammar. `"*"` accepts an arbitrary token sequence. A typed syntax uses component types such as `"<length>"`, which accepts a length such as `12pt`, or `"<color>"`, which accepts a CSS color. It can combine alternatives with `|` and use `+` or `#` multipliers. The `inherits` descriptor is required and must be `true` or `false`. A typed syntax requires an `initial-value` that matches the syntax and is computationally independent. With `"*"`, `initial-value` may be omitted.

A registered property's value is checked against its syntax when computed. Its `inherits` descriptor controls whether it inherits when no value is specified. Its `initial-value` supplies the default when it does not inherit or has no inherited value. If more than one valid registration names the same custom property, the last one in stylesheet order is used.

4. Reading XML attributes with `attr()`

   CSS Values 5 §7.7, Attribute References: the attr() notation.

   `attr()` reads an attribute from the XML element being styled. Without a type, it produces a string. A syntax can request a parsed CSS value, and an optional fallback supplies a value when the attribute is absent or cannot be parsed:

   ```
   figure {
       content: attr(title);
       width: attr(data-width <length>, 100pt);
   }
   ```

   In `attr(data-width <length>, 100pt)`, `<length>` means a CSS length such as `12pt`. An untyped `attr()` has an empty-string fallback when none is written. A typed `attr()` has no implicit fallback. Attribute references can include a namespace prefix declared by `@namespace`.

   An attribute-derived value is tainted. If it is used as or within a URL, the declaration becomes invalid at computed-value time. After substitution, the resulting declaration must still match the property's grammar.

Conditional rules

A conditional rule contains rules that participate in the cascade only when its condition is true. A condition filters rules; it does not give declarations extra priority. Declarations that pass the condition compete by the usual cascade order.

Procedure

1. Media conditions with `@media`

   CSS Conditional 3 §5, Media-specific style sheets: the @media rule; Media Queries 5 §2, Media Queries.

   `@media` tests a media query list. Queries can test a media type, media features, or both. A comma separates alternatives: the list matches if at least one query is true. `and` combines a media type or feature with another test. `not` negates the complete query. `only` has no effect on whether a query matches.

   ```
   @media print and (width >= 6in) {
       chapter { color: black; }
   }
   ```

   CSSMedia evaluates media types for paged output. `print` and `all` match. `screen` does not. A query without a media type is treated as `all`. Media feature queries are evaluated against the page and environment values CSSMedia uses.

2. Feature tests with `@supports`

   CSS Conditional 3 §6, Feature queries: the @supports rule; CSS Conditional 5 §2, Extensions to the @supports rule.

   `@supports` tests whether CSSMedia accepts a declaration or selector. A declaration test uses parentheses:

   ```
   @supports (color: navy) {
       title { color: navy; }
   }
   ```

   Use `selector()` to test one complex selector. CSS also defines `font-tech()` and `font-format()` tests. `at-rule()` tests whether an at-rule is supported, and a named support condition can test a group declared with `@supports-condition`. Conditions can combine tests with `not`, `and` and `or`; use parentheses when combining `and` and `or`.

   ```
   @supports selector(chapter > title) and (display: block) {
       chapter > title { display: block; }
   }
   ```

   A true feature test means the tested syntax is accepted. It does not test whether a selector matches any element or whether a particular document contains a value.

3. Named support conditions

   CSS Conditional 5 §8, Defining Custom Support Queries: the @supports-condition rule.

   `@supports-condition` gives a name to a group of feature tests. Declarations and nested selectors in the block are tested for support; they do not add styling declarations. Define the name before using it:

```
@supports-condition --basic-layout {
    display: block;
    color: navy;
}

@supports (--basic-layout) {
    chapter { display: block; }
}
```

If more than one definition uses the same name, the last definition in document order replaces the earlier one.

4. General conditions with `@when`

   CSS Conditional 5 §3, Generalized Conditional Rules: the @when rule.

   `@when` tests a boolean condition that can combine `media()` and `supports()` tests. It provides a single condition syntax for tests that would otherwise appear in separate `@media` or `@supports` rules.

   ```
   @when media(overflow-block: paged) and supports(color: navy) {
       title { color: navy; }
   }
   ```

   Conditions can be grouped with parentheses and combined with `and`, `or` and `not`.

5. Conditional chains with `@else`

   CSS Conditional 5 §4, Chained Conditionals: the @else rule.

   An `@else` rule follows an `@when`, `@media`, `@supports` or `@container` rule to form a chain. Only the first branch whose condition is true applies. `@else` can have its own condition; without one, it is the final fallback.

   ```
   @when supports(color: navy) {
       title { color: navy; }
   } @else supports(color: black) {
       title { color: black; }
   } @else {
       title { color: inherit; }
   }
   ```

   Each `@else` belongs to the immediately preceding conditional chain in the same rule list. An unrelated rule or declaration ends the chain.

6. Reusable media queries with `@custom-media`

   Media Queries 5 §10, Custom Media Queries.

   `@custom-media` assigns a name to a media query list, or to the boolean value `true` or `false`. Use the name in a media query by writing it inside parentheses:

   ```
   @custom-media --wide-page (width >= 6in);

   @media (--wide-page) {
       chapter { color: black; }
   }
   ```

A definition can refer to other custom media names. Circular references do not produce a true query. When a name is defined more than once, the last definition is used.

7. Container queries with `@container`

   CSS Conditional 5 §5.1, Creating Query Containers: the container-type property; §5.4, Container Queries: the @container rule.

   `@container` selects a query container from the element's ancestors. A container name before the condition restricts which ancestor is selected. A style query tests a computed style or custom property on the selected container. Conditions can combine tests with `and`, `or` and `not`.

   ```
   @container style(--theme: dark) {
       chapter { color: white; }
   }
   ```

   A style query can use an element as a container without size containment. Without a name, the nearest eligible ancestor is used. With a name, the nearest ancestor whose `container-name` includes that name is used.

8. Importing stylesheets

   CSS Cascade 5 §2, Importing Style Sheets: the @import rule; CSS Cascade 5 §2.1, Conditional @import Rules.

   `@import` loads another stylesheet. Its rules take part in the cascade at the import's position. The URL or quoted string comes first. Optional clauses can assign the imported rules to a layer, test feature support, or restrict them to matching media:

   ```
   @import "print.css" layer(print-theme) supports(color: navy) print;
   ```

   An optional `@charset`, layer-order statements and `@supports-condition` rules can precede imports. Imports precede `@namespace`; namespace declarations precede ordinary rules. An import after another rule is ignored. Imports can be conditional. If an import condition is false, its rules do not participate. A layer clause places the imported stylesheet in that layer; an unnamed `layer` creates an anonymous layer.

9. Layer rules

   CSS Cascade 5 §6.4, Cascade Layers.

   `@layer` declares a layer order or groups rules into a layer:

   ```
   @layer reset, base, theme;

   @layer theme {
       title { color: navy; }
   }
   ```

   Declarations in layers use the layer precedence described in Cascade order and layers. Layer statements establish order, and layer blocks group rules. An `@import` can assign its imported rules to a layer with a layer clause.

10. Other stylesheet declarations

    CSS Namespaces 3 §3.1, Syntax.

`@namespace` declares prefixes used by selectors. See Basic selectors and namespaces. Conditional rules and imports can also contain `@page` rules; page declarations use the page-context cascade described in Cascade order and layers.

Complete reference

Look up every selector, property, value and at-rule by subject.

Each table links to the CSS specification that defines its entries. The chapters of this guide end with a shorter reference that lists the entries they explain. The complete references are grouped by subject:

- Selector reference
- Font reference
- Box model properties
- Dimension properties
- Background and image reference
- Display and positioning
- Page reference
- Links and generated content reference
- Footnotes
- Links
- Named strings
- Cascade reference

Selector reference

These tables list every selector that CSSMedia accepts. The `In a PDF` column says whether the selector matches elements while CSSMedia generates a PDF.

Basic selectors

Basic selectors match an element by its name, ID or attributes. See Basic selectors and namespaces.

Name	Description	Values	In a PDF	Specification
`E`	Type selector. Matches elements named `E`.	An element name.	Matches	Selectors 4 §5.1
`*`	Universal selector. Matches any element.		Matches	Selectors 4 §5.2
`#id`	ID selector. Matches the element whose `id` or `xml:id` attribute has this value.	An identifier.	Matches	Selectors 4 §6.7

Name	Description	Values	In a PDF	Specification
`.name`	Class selector. Matches inside SVG and MathML content, which define a `class` attribute.	An identifier.	Matches in SVG and MathML only	Selectors 4 §6.6
`[attr]`	Matches elements that have the attribute.	An attribute name, with an optional namespace prefix.	Matches	Selectors 4 §6
`[attr=value]`	Matches when the attribute value is exactly `value`.	A string or an identifier. Add `i` to ignore ASCII case or `s` to require it.	Matches	Selectors 4 §6.1
`[attr~=value]`	Matches when the value is one of the white-space separated words of the attribute.	A word without white space.	Matches	Selectors 4 §6.1
`[attr|=value]`	Matches when the attribute is `value` or starts with `value-`.	A string or an identifier.	Matches	Selectors 4 §6.1
`[attr^=value]`	Matches when the attribute starts with `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`[attr$=value]`	Matches when the attribute ends with `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`[attr*=value]`	Matches when the attribute contains `value`.	A non-empty string.	Matches	Selectors 4 §6.2
`ns|E`, `*|E`, `|E`	Limits a type selector to a declared namespace, to any namespace, or to no namespace. The same prefixes work in attribute selectors.	A prefix declared with `@namespace`.	Matches	Selectors 4 §5.3, CSS Namespaces 3 §3

Name	Description	Values	In a PDF	Specification
`&`	Nesting selector. Stands for the selector of the parent rule.		Matches	CSS Nesting 1

Combinators

A combinator relates two selectors. See Combinators.

Name	Description	Values	In a PDF	Specification
`A B`	Descendant combinator. Matches `B` inside `A` at any depth.	Two selectors separated by white space.	Matches	Selectors 4 §14.1
`A > B`	Child combinator. Matches `B` when its parent is `A`.	Two selectors.	Matches	Selectors 4 §14.2
`A + B`	Next-sibling combinator. Matches `B` when it directly follows `A`.	Two selectors.	Matches	Selectors 4 §14.3
`A ~ B`	Subsequent-sibling combinator. Matches `B` when `A` is an earlier sibling.	Two selectors.	Matches	Selectors 4 §14.4
`A || B`	Column combinator. Matches the cells `B` of the column that `A` describes.	Two selectors.	Matches	Selectors 5 §8.1
`A /attr/ B`	Reference combinator. Matches `B` when the attribute `attr` of `A` refers to its ID.	Two selectors and an attribute name between slashes.	Matches	Selectors 5 §9.1

Pseudo-classes

A pseudo-class tests a property of an element other than its name and attributes. See Pseudo-classes.

Name	Description	
`:root`	The root element of the document.	

Name	Description	
`:empty`	An element with no child elements and no text other than white space.	
`:scope`	The scoping element. Without another scope, the root element.	
`:first-child`, `:last-child`, `:only-child`	The first, last or only element among its siblings.	
`:first-of-type`, `:last-of-type`, `:only-of-type`	The first, last or only element of its type among its siblings.	
`:nth-child()`, `:nth-last-child()`	Elements at a position among their siblings, counted from the start or from the end.	odd, An+B follo and list.
`:nth-of-type()`, `:nth-last-of-type()`	Elements at a position among the siblings of the same type.	odd, An+B
`:nth-col()`, `:nth-last-col()`	Table cells at a position among the columns of their table.	odd, An+B
`:is()`, `:matches()`	An element that matches any selector of the list. `:matches()` is the old name of `:is()`.	A sel Inval are i
`:where()`	Works as `:is()` and adds no specificity.	A sel Inval are i

Name	Description	
`:not()`	An element that matches none of the selectors of the list.	A sel An ir selec the v selec
`:has()`	An element that has a related element matching the selector.	A rel selec such `figu` cann anot or a elem
`:lang()`	Elements in a language. The language comes from `xml:lang` or the `-cssmedia-lang` property.	One lang rang iden strin
`:dir()`	Elements by text direction, read from the text and the language, not from the `direction` property.	`ltr`
`:link`, `:any-link`	Elements that are hyperlinks, because `-cssmedia-link` gives them a destination.	
`:local-link`	Hyperlinks that point to the same document. The optional number limits how many path segments are compared.	An o nega in pa

Name	Description	
`:read-only`, `:defined`	Every element is read-only and defined, because an XML element has no editable or undefined state.	
`:hover`, `:active`, `:focus`, `:focus-visible`, `:focus-within`	User action states, such as pointing at an element or giving it keyboard focus.	
`:interest-source`, `:interest-target`	Source and target of a user's declared interest.	
`:visited`, `:target`	Links that were visited, and the element that a link points to. A PDF has no browsing history and no current target.	
`:playing`, `:paused`, `:seeking`, `:buffering`, `:stalled`, `:muted`, `:volume-locked`	Media playback, loading and sound states.	
`:current`, `:current()`, `:past`, `:future`	Elements in relation to a timeline, such as speech or video playback.	A sel for :
`:open`, `:popover-open`, `:modal`, `:fullscreen`, `:picture-in-picture`	Display states of interactive elements.	
`:enabled`, `:disabled`, `:read-write`, `:placeholder-shown`, `:autofill`, `:default`, `:checked`, `:unchecked`, `:indeterminate`, `:blank`	Form control states. An XML element is never a form control.	
`:valid`, `:invalid`, `:in-range`, `:out-of-range`, `:required`, `:optional`, `:user-valid`, `:user-invalid`	Form validation states.	

Name	Description	
`:state()`	A custom state that a document language exposes. XML has none.	An ic
`:heading`, `:heading()`	Headings, as a document language defines them. CSSMedia does not give XML elements a heading role.	A list for :

Pseudo-elements

A pseudo-element selects a part of an element or generated content. See Pseudo-elements.

Name	Description	Values	In a PDF	Specification
`::before`, `::after`	Generated content before or after the content of an element. The `content` property gives it.		Matches	CSS Pseudo-Elements 4 §4.1
`::marker`	The marker of a list item. Also `::before::marker` and `::after::marker`.		Matches	CSS Pseudo-Elements 4 §4.2
`::first-line`	The first formatted line of a block.		Matches	CSS Pseudo-Elements 4 §2.1
`::first-letter`	The first letter of a block, with the punctuation around it. `::first-letter::prefix` and `::first-letter::suffix` select that punctuation.		Matches	CSS Pseudo-Elements 4 §2.2

Name	Description	Values	In a PDF	Specification
`::footnote-call`	The reference that replaces a footnote in the text. By default it is the value of the `footnote` counter, as a superscript.		Matches	CSS GCPM 3 §2.6
`::footnote-marker`	The marker at the start of a footnote body. By default it is the value of the `footnote` counter, followed by a period and a space.		Matches	CSS GCPM 3 §2.7
`::selection`, `::search-text`, `::target-text`, `::spelling-error`, `::grammar-error`, `::highlight()`	Highlighted ranges created by selection, search, navigation, spelling and grammar checking, or scripts.	A custom identifier for `::highlight()`.	No range exists	CSS Pseudo-Elements 4 §3.1
`::placeholder`, `::file-selector-button`, `::details-content`	Parts of form controls and disclosure widgets.		No such part exists	CSS Pseudo-Elements 4 §4.3, §5.1, §5.2
`::backdrop`	The backdrop of an element shown in the top layer.		No such part exists	CSS Position 4 §3.2

Font reference

This table covers the font properties, at-rules, and descriptors introduced in the font guides.

Name	Description	Values	Default	Specification
`font`	Sets the font family, size, style, weight, width, variant, and line height in one declaration.	A system-font keyword (`caption`, `icon`, `menu`, `message-box`, `small-caption`, or `status-bar`), or optional font-style, `normal` or `small-caps`, font-weight, and font-width values followed by required font size and family and optional line height		CSS Fonts 4, `font`

Name	Description	Values	Default	Specification
`font-family`	Selects a font family or ordered list of fallback families.	Family names and `serif`, `sans-serif`, `system-ui`, `cursive`, `fantasy`, `math`, `monospace`, `ui-serif`, `ui-sans-serif`, `ui-monospace`, `ui-rounded`, or `generic(fangsong)`, `generic(kai)`, `generic(khmer-mul)`, `generic(nastaliq)`		CSS Fonts 4, `font-family`
`font-size`	Sets the font size.	`xx-small`, `x-small`, `small`, `medium`, `large`, `x-large`, `xx-large`, `xxx-large`, `larger`, `smaller`, a non-negative length or percentage, or `math`		CSS Fonts 4, `font-size`
`font-size-adjust`	Adjusts font sizing using a selected font metric.	none, optional metric (`ex-height`, `cap-height`, `ch-width`, `ic-width`, or `ic-height`), then `from-font` or a non-negative number		CSS Fonts 5, `font-size-adjust`
`font-weight`	Selects the requested weight of a font face.	`normal`, `bold`, `bolder`, `lighter`, or a number from 1 to 1000		CSS Fonts 4, `font-weight`
`font-width`	Selects the requested width of a font face.	`normal`, a non-negative percentage, `ultra-condensed`, `extra-condensed`, `condensed`, `semi-condensed`, `semi-expanded`, `expanded`, `extra-expanded`, or `ultra-expanded`		CSS Fonts 4, `font-width`
`font-stretch`	Legacy alias for `font-width`.	`normal`, a non-negative percentage, `ultra-condensed`, `extra-condensed`, `condensed`, `semi-condensed`, `semi-expanded`, `expanded`, `extra-expanded`, or `ultra-expanded`		CSS Fonts 4, `font-stretch`

Name	Description	Values	Default	Specification
`font-style`	Selects the requested upright or slanted face.	`normal`, `italic`, `left`, `right`, or `oblique` with an optional angle from -90deg to 90deg		CSS Fonts 4, `font-style`
`font-synthesis`	Controls synthetic weight, style, small-cap, and position forms.	none, or any combination of `weight`, `style`, `small-caps`, and `position`		CSS Fonts 4, `font-synthesis`
`font-synthesis-weight`	Controls synthetic bolding.	`auto` or none		CSS Fonts 4, `font-synthesis-weight`
`font-synthesis-style`	Controls synthetic italic or oblique styling.	`auto`, none, or `oblique-only`		CSS Fonts 4, `font-synthesis-style`
`font-synthesis-small-caps`	Controls synthetic small capitals.	`auto` or none		CSS Fonts 4, `font-synthesis-small-caps`
`font-synthesis-position`	Controls synthetic subscript and superscript forms.	`auto` or none		CSS Fonts 4, `font-synthesis-position`
`font-variant`	Shorthand for typographic variant properties.	`normal`, none, or combinations of variant longhand values		CSS Fonts 4, `font-variant`
`font-variant-alternates`	Requests named alternate glyphs and stylistic forms.	`normal` or combinations of `stylistic()`, `historical-forms`, `styleset()`, `character-variant()`, `swash()`, `ornaments()`, and `annotation()`		CSS Fonts 4, `font-variant-alternates`

Name	Description	Values	Default	Specification
`font-variant-caps`	Requests small-cap and other capitalization forms.	`normal`, `small-caps`, `all-small-caps`, `petite-caps`, `all-petite-caps`, `unicase`, or `titling-caps`		CSS Fonts 4, `font-variant-caps`
`font-variant-east-asian`	Requests East Asian glyph variants and widths.	`normal` or combinations of `jis78`, `jis83`, `jis90`, `jis04`, `simplified`, `traditional`, `full-width`, `proportional-width`, and `ruby`		CSS Fonts 4, `font-variant-east-asian`
`font-variant-emoji`	Requests text or emoji presentation for supported characters.	`normal`, `text`, `emoji`, or `unicode`		CSS Fonts 4, `font-variant-emoji`
`font-variant-ligatures`	Controls common, discretionary, historical, and contextual ligatures.	`normal`, none, or combinations of `common-ligatures/no-common-ligatures`, `discretionary-ligatures/no-discretionary-ligatures`, `historical-ligatures/no-historical-ligatures`, and `contextual/no-contextual`		CSS Fonts 4, `font-variant-ligatures`
`font-variant-numeric`	Requests number shapes, spacing, and fractions.	`normal` or combinations of `lining-nums/oldstyle-nums`, `proportional-nums/tabular-nums`, `diagonal-fractions/stacked-fractions`, `ordinal`, and `slashed-zero`		CSS Fonts 4, `font-variant-numeric`
`font-variant-position`	Requests subscript or superscript glyph forms.	`normal`, `sub`, or `super`		CSS Fonts 4, `font-variant-position`

Name	Description	Values	Default	Specification
`font-kerning`	Controls use of kerning information in the font.	`auto`, `normal`, or none		CSS Fonts 4, `font-kerning`
`font-feature-settings`	Sets low-level OpenType feature tags and values.	`normal` or a comma-separated list of feature tags, optionally followed by a non-negative integer, on, or `off`		CSS Fonts 4, `font-feature-settings`
`font-language-override`	Sets the OpenType language system tag.	`normal` or a string		CSS Fonts 4, `font-language-override`
`font-optical-sizing`	Enables or disables automatic optical-size axis selection for variable fonts.	`auto` or none		CSS Fonts 4, `font-optical-sizing`
`font-variation-settings`	Sets low-level variable-font axis tags and values.	`normal` or a comma-separated list of four-character tags and numbers		CSS Fonts 4, `font-variation-settings`
`font-palette`	Chooses a color palette for a color font.	`normal`, `light`, `dark`, a palette identifier, or `palette-mix()`		CSS Fonts 4, `font-palette`
`line-height`	Sets the preferred height of inline content used to determine line-box height.	`normal`, a non-negative number, non-negative length, or non-negative percentage		CSS Inline 3, `line-height`
`@font-face`	Declares a font face and its source for use by a family name.	Font-face rule with descriptors		CSS Fonts 4, `@font-face`

Name	Description	Values	Default	Specification
`@font-face font-family`	Names the family for this face.	A family name		CSS Fonts 4, `font-family` descriptor
`@font-face src`	Lists font resources or installed font names to try in order.	Comma-separated `url()` or `local()` sources with optional format and technology hints		CSS Fonts 4, `src` descriptor
`@font-face font-display`	Sets the fallback and swap behavior while a font loads.	`auto`, `block`, `swap`, `fallback`, or `optional`		CSS Fonts 4, `font-display` descriptor
`@font-face font-weight`	Describes the weight or weight range supplied by the face.	`auto`, or one or two values, each `normal`, `bold`, or a number from 1 to 1000		CSS Fonts 4, `font-weight` descriptor
`@font-face font-width`	Describes the width or width range supplied by the face.	`auto`, or one or two `font-width` values, each `normal`, a width keyword, or a non-negative percentage		CSS Fonts 4, `font-width` descriptor
`@font-face font-stretch`	Legacy alias for the `font-width` descriptor.	`auto`, or one or two values, each `normal`, a non-negative percentage, `ultra-condensed`, `extra-condensed`, `condensed`, `semi-condensed`, `semi-expanded`, `expanded`, `extra-expanded`, or `ultra-expanded`		CSS Fonts 4, `font-stretch` descriptor
`@font-face font-style`	Describes the style or style range supplied by the face.	`auto`, `normal`, `italic`, `left`, `right`, or `oblique` with no angle or one or two angles from -90deg to 90deg		CSS Fonts 4, `font-style` descriptor
`@font-face unicode-range`	Limits which character code points this face can serve.	One or more Unicode ranges		CSS Fonts 4, `unicode-range` descriptor

Name	Description	Values	Default	Specification
`@font-face font-feature-settings`	Sets default low-level OpenType feature settings for a face.	`normal` or a comma-separated list of feature tags, optionally followed by a non-negative integer, on, or `off`		CSS Fonts 4, `font-feature-settings` descriptor
`@font-face font-language-override`	Sets the default OpenType language system tag for a face.	`normal` or a string		CSS Fonts 4, `font-language-override` descriptor
`@font-face font-named-instance`	Selects a named variation instance in the font.	`auto` or a string		CSS Fonts 4, `font-named-instance` descriptor
`@font-face font-variation-settings`	Sets the face's default variable-font axes.	`normal` or a list of axis tags and numbers		CSS Fonts 4, `font-variation-settings` descriptor
`@font-face font-size`	Describes the face's optical-size value or range.	`auto` or one or two numbers		CSS Fonts 5, `font-size` descriptor
`@font-face ascent-override`	Overrides the face's ascent metric.	One or two values, each `normal` or a non-negative percentage		CSS Fonts 5, `ascent-override`
`@font-face descent-override`	Overrides the face's descent metric.	One or two values, each `normal` or a non-negative percentage		CSS Fonts 5, `descent-override`
`@font-face line-gap-override`	Overrides the face's line-gap metric.	One or two values, each `normal` or a non-negative percentage		CSS Fonts 5, `line-gap-override`
`@font-face size-adjust`	Scales the face's glyph outlines and metrics.	Non-negative percentage		CSS Fonts 5, `size-adjust`

Name	Description	Values	Default	Specification
`@font-face subscript-position-override`	Overrides the subscript position metric.	One or two values, each `normal`, `from-font`, or a percentage		CSS Fonts 5, `subscript-position-override`
`@font-face subscript-size-override`	Overrides the subscript size metric.	One or two values, each `normal`, `from-font`, or a non-negative percentage		CSS Fonts 5, `subscript-size-override`
`@font-face superscript-position-override`	Overrides the superscript position metric.	One or two values, each `normal`, `from-font`, or a percentage		CSS Fonts 5, `superscript-position-override`
`@font-face superscript-size-override`	Overrides the superscript size metric.	One or two values, each `normal`, `from-font`, or a non-negative percentage		CSS Fonts 5, `superscript-size-override`
`@font-feature-values`	Names feature values for one or more font families.	Rule with one or more family names and feature-value blocks		CSS Fonts 4, `@font-feature-values`
`@font-feature-values font-display`	Sets a default font-display policy for the listed families.	`auto`, `block`, `swap`, `fallback`, or `optional`		CSS Fonts 4, `font-display` descriptor
`@font-feature-values @annotation`	Maps names to annotation feature indexes.	Named values mapped to one non-negative integer		CSS Fonts 4, `@annotation`
`@font-feature-values @character-variant`	Maps names to character-variant feature indexes.	One or two integers. Values 1 to 99 select `cv01` to `cv99`; in a two-integer value, the second integer sets that feature's value. Zero and values above 99 parse but select no feature.		CSS Fonts 4, `@character-variant`

Name	Description	Values	Default	Specification
`@font-feature-values @historical-forms`	Maps names to historical-forms feature indexes.	Named values mapped to one or more non-negative integers		CSS Fonts 4, `@historical-forms`
`@font-feature-values @ornaments`	Maps names to ornaments feature indexes.	Named values mapped to one non-negative integer		CSS Fonts 4, `@ornaments`
`@font-feature-values @styleset`	Maps names to styleset feature indexes.	Named values mapped to one or more integers; 1 through 99 select `ss01` through `ss99`, while 0 and values above 99 select no OpenType feature		CSS Fonts 4, `@styleset`
`@font-feature-values @stylistic`	Maps names to stylistic feature indexes.	Named values mapped to one non-negative integer		CSS Fonts 4, `@stylistic`
`@font-feature-values @swash`	Maps names to swash feature indexes.	Named values mapped to one non-negative integer		CSS Fonts 4, `@swash`
`@font-palette-values`	Defines a named palette for a color font.	Rule with a dashed identifier and palette descriptors		CSS Fonts 4, `@font-palette-values`
`@font-palette-values font-family`	Required descriptor that identifies the font family to which the named palette applies.	One or more family names		CSS Fonts 4, `font-family` descriptor
`@font-palette-values base-palette`	Chooses the base palette for the named palette.	`light`, `dark`, or a non-negative integer		CSS Fonts 4, `base-palette`

Name	Description	Values	Default	Specification
`@font-palette-values override-colors`	Replaces selected colors in the base palette.	One or more palette index and color pairs		CSS Fonts 4, `override-colors`

Box model properties

This table lists the box model properties described in Box sizing, margins and padding. A length is a number with a unit, such as `12pt` or `1em`. Margin and padding percentages are relative to the containing block's logical width, or inline size.

These properties set the margins, padding and box sizing of the boxes of a document.

The CSS-wide keywords `inherit`, `initial`, `revert`, `revert-layer`, and `unset` can also be used with these properties.

Name	Description	Values	Default	Specification
`box-sizing`	Chooses whether `width` and `height` measure the content area or the box including padding and border.	`content-box` measures the content; `border-box` includes padding and border.		CSS Sizing 3
`margin`	Sets the space outside all four sides of a box.	One to four lengths or percentages, `auto`, or `anchor-size()`.		CSS Box Model 4
`margin-block`	Sets the margins at the start and end of the block axis.	One or two lengths, percentages, `auto`, or `anchor-size()` values.		CSS Logical Properties 1
`margin-block-end`	Sets the margin at the end of the block axis.	Length, percentage, `auto`, or `anchor-size()`.		CSS Logical Properties 1
`margin-block-start`	Sets the margin at the start of the block axis.	Length, percentage, `auto`, or `anchor-size()`.		CSS Logical Properties 1
`margin-bottom`	Sets the space outside the bottom side of a box.	Length, percentage, `auto`, or `anchor-size()`.		CSS Box Model 4

Name	Description	Values	Default	Specification
`margin-break`	Controls whether a margin is kept or discarded when its box is split by a page break.	`auto` follows the break rules; `keep` keeps the margin; `discard` removes it.		CSS Fragmentation 4
`margin-inline`	Sets the margins at the start and end of the inline axis.	One or two lengths, percentages, `auto`, or `anchor-size()` values.		CSS Logical Properties 1
`margin-inline-end`	Sets the margin at the end of the inline axis.	Length, percentage, `auto`, or `anchor-size()`.		CSS Logical Properties 1
`margin-inline-start`	Sets the margin at the start of the inline axis.	Length, percentage, `auto`, or `anchor-size()`.		CSS Logical Properties 1
`margin-left`	Sets the space outside the left side of a box.	Length, percentage, `auto`, or `anchor-size()`.		CSS Box Model 4
`margin-right`	Sets the space outside the right side of a box.	Length, percentage, `auto`, or `anchor-size()`.		CSS Box Model 4
`margin-top`	Sets the space outside the top side of a box.	Length, percentage, `auto`, or `anchor-size()`.		CSS Box Model 4
`margin-trim`	Removes selected margins where child boxes meet the edges of their container.	none; `block`, `inline`, or both; or one or more of `block-start`, `inline-start`, `block-end`, and `inline-end`.		CSS Box Model 4
`padding`	Sets the space between the content and all four sides of its border.	One to four non-negative lengths or percentages.		CSS Box Model 4
`padding-block`	Sets the padding at the start and end of the block axis.	One or two non-negative lengths or percentages.		CSS Logical Properties 1
`padding-block-end`	Sets the padding at the end of the block axis.	Non-negative length or percentage.		CSS Logical Properties 1

Name	Description	Values	Default	Specification
`padding-block-start`	Sets the padding at the start of the block axis.	Non-negative length or percentage.		CSS Logical Properties 1
`padding-bottom`	Sets the space between the content and the bottom border.	Non-negative length or percentage.		CSS Box Model 4
`padding-inline`	Sets the padding at the start and end of the inline axis.	One or two non-negative lengths or percentages.		CSS Logical Properties 1
`padding-inline-end`	Sets the padding at the end of the inline axis.	Non-negative length or percentage.		CSS Logical Properties 1
`padding-inline-start`	Sets the padding at the start of the inline axis.	Non-negative length or percentage.		CSS Logical Properties 1
`padding-left`	Sets the space between the content and the left border.	Non-negative length or percentage.		CSS Box Model 4
`padding-right`	Sets the space between the content and the right border.	Non-negative length or percentage.		CSS Box Model 4
`padding-top`	Sets the space between the content and the top border.	Non-negative length or percentage.		CSS Box Model 4

Dimension properties

This reference lists the CSS sizing properties documented in Box dimensions. The `Values` column gives CSS syntax.

CSS Anchor Positioning permits `anchor-size()` only on properties accepted in an `@position-try` rule. These include sizing, inset, and margin properties. This page lists the sizing properties.

Preferred sizes

Name	Description	Values	Default	Specification
`width`	Sets the preferred physical width.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`height`	Sets the preferred physical height.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`inline-size`	Sets the preferred size on the inline axis.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`block-size`	Sets the preferred size on the block axis.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3

- `<length-percentage>` means a length such as `12pt`, a percentage such as `60%`, or a `calc()` expression combining them.
- `[0,∞]` in `fit-content(<length-percentage [0,∞]>)` means the length or percentage cannot be negative.
- `anchor-size()` accepts an optional anchor name, an optional size keyword, or both, followed by an optional length or percentage fallback. It resolves to the anchor's size only when the box is absolutely positioned and has a target anchor. Otherwise, it resolves to the fallback. Without a fallback, the declaration becomes invalid at computed-value time. The sizing properties listed here include physical and logical properties.

Minimum and maximum sizes

Name	Description	Values	Default	Specification
`min-width`	Sets a lower limit for the physical width.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`min-height`	Sets a lower limit for the physical height.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`min-inline-size`	Sets a lower limit on the inline axis.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`min-block-size`	Sets a lower limit on the block axis.	`auto`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`max-width`	Sets an upper limit for the physical width.	`none`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`max-height`	Sets an upper limit for the physical height.	`none`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content(<length-percentage [0,∞]>)`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3
`max-inline-size`	Sets an upper limit on the inline axis.	`none`, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3

Name	Description	Values	Default	Specification
`max-block-size`	Sets an upper limit on the block axis.	none, `<length-percentage>`, `stretch`, `contain`, `min-content`, `max-content`, `fit-content`, `fit-content()`, `calc-size()`, or `anchor-size()`.		CSS Sizing 3

- `<length-percentage>` means a length such as `12pt`, a percentage such as `60%`, or a `calc()` expression combining them.
- `[0,∞]` in `fit-content(<length-percentage [0,∞]>)` means the length or percentage cannot be negative.

Paired size shorthands

Name	Description	Values	Default	Specification
`size`	Sets `width` and `height`.	One or two preferred-size values.		CSS Sizing 4
`min-size`	Sets `min-width` and `min-height`.	One or two minimum-size values.		CSS Sizing 4
`max-size`	Sets `max-width` and `max-height`.	One or two maximum-size values.		CSS Sizing 4

- Each pair takes the corresponding longhand's values in width-then-height order. The second value is optional and defaults to the first.
- The `size` shorthand is not valid in an @page context. That context has a separate `size` descriptor for page dimensions.

Aspect ratio and intrinsic sizes

Name	Description	Values	Default	Specification
`aspect-ratio`	Supplies a preferred ratio between the width and height.	`auto`, a non-negative number such as `1.5`, or two non-negative numbers such as `16 / 9`; `auto` can be combined with a number.		CSS Sizing 4
`contain-intrinsic-width`	Fallback intrinsic width under size containment.	Optional `auto`; none or a non-negative length.		CSS Sizing 4
`contain-intrinsic-height`	Fallback intrinsic height under size containment.	Optional `auto`; none or a non-negative length.		CSS Sizing 4

Name	Description	Values	Default	Specification
`contain-intrinsic-inline-size`	Fallback intrinsic inline size under size containment.	Optional `auto`; none or a non-negative length.		CSS Sizing 4
`contain-intrinsic-block-size`	Fallback intrinsic block size under size containment.	Optional `auto`; none or a non-negative length.		CSS Sizing 4
`contain-intrinsic-size`	Sets fallback intrinsic width and height.	One or two longhand values; see below.		CSS Sizing 4

- For `aspect-ratio`, the first number is the width and the second is the height. If you give one number, the second number is `1`. For example, `1.5` means the same ratio as `1.5 / 1`.
- Each `contain-intrinsic-*` longhand accepts an optional `auto`, followed by none or a non-negative length.
- The `contain-intrinsic-size` shorthand takes one or two such values. One value applies to both dimensions.

Replaced elements and intrinsic contributions

Name	Description	Values	Default	Specification
`frame-sizing`	Selects a replaced element's content size for sizing.	`auto` or a `content-*` dimension.		CSS Sizing 4
`min-intrinsic-sizing`	Controls min-content contribution compression.	`legacy`, `zero-if-scroll`, `zero-if-extrinsic`, or both.		CSS Sizing 4

- `frame-sizing` accepts `auto`, `content-width`, `content-height`, `content-block-size`, or `content-inline-size`.

Background and image reference

Name	Description	Values	Default	Specification
`background`	Sets the background color and one or more background image layers. The color, if present, belongs to the final layer.	Comma-separated layers with image, position and optional size, repeat, attachment, and clipping or positioning box values; optional color on the final layer	See individual properties	CSS Backgrounds 4, `background`
`background-attachment`	Selects whether a background image is attached to its box, its contents, or the page box in paged media.	`scroll`, `fixed`, or `local`; comma-separated list	`fixed`	CSS Backgrounds 4, `background-attachment`
`background-blend-mode`	Sets how each background image layer blends with the layers below it and the background color.	`normal`, `darken`, `multiply`, `color-burn`, `lighten`, `screen`, `color-dodge`, `overlay`, `soft-light`, `hard-light`, `difference`, `exclusion`, hue, `saturation`, `color`, or `luminosity`; comma-separated list	`normal`	Compositing and Blending 1, `background-blend-mode`
`background-clip`	Sets the box area where the background is painted.	`border-box`, `padding-box`, `content-box`, `border-area`, `text`, or both `border-area` and `text`; comma-separated list	`border-box`	CSS Backgrounds 4, `background-clip`
`background-color`	Sets the background color.	`<color>`	`transparent`	CSS Backgrounds 4, `background-color`

Name	Description	Values	Default	Specification
`background-image`	Sets one or more background images or gradients.	One or more comma-separated none or `<image>` values	none	CSS Backgrounds 4, `background-image`
`background-origin`	Sets the box used as the positioning area for each background image.	`border-box`, `padding-box`, or `content-box`; comma-separated list	`padding-box`	CSS Backgrounds 4, `background-origin`
`background-position`	Sets the position of each background image.	One- to four-part position using keywords, lengths, or percentages; comma-separated list	`0% 0%`	CSS Backgrounds 4, `background-position`
`background-position-block`	Sets the position of each background image on the block axis.	`center`, or `start` or end, optionally combined with a length or percentage; comma-separated list	`0%`	CSS Backgrounds 4, `background-position-block`
`background-position-inline`	Sets the position of each background image on the inline axis.	`center`, or `start` or end, optionally combined with a length or percentage; comma-separated list	`0%`	CSS Backgrounds 4, `background-position-inline`
`background-position-x`	Sets the position of each background image on the physical horizontal axis.	`center`; `left`, `right`, `x-start`, or `x-end`; a length or percentage; or a side keyword with a length or percentage; comma-separated list	`0%`	CSS Backgrounds 4, `background-position-x`

Name	Description	Values	Default	Specification
`background-position-y`	Sets the position of each background image on the physical vertical axis.	`center`; `top`, `bottom`, `y-start`, or `y-end`; a length or percentage; or a side keyword with a length or percentage; comma-separated list	`0%`	CSS Backgrounds 4, `background-position-y`
`background-repeat`	Sets how each background image repeats along both axes.	`repeat`, `space`, `round`, or `no-repeat`; optional second value; `repeat-x`, `repeat-y`, `repeat-block`, or `repeat-inline`; comma-separated list	`repeat`	CSS Backgrounds 4, `background-repeat`
`background-repeat-block`	Sets how each background image repeats on the block axis.	`repeat`, `space`, `round`, or `no-repeat`; comma-separated list	`repeat`	CSS Backgrounds 4, `background-repeat-block`
`background-repeat-inline`	Sets how each background image repeats on the inline axis.	`repeat`, `space`, `round`, or `no-repeat`; comma-separated list	`repeat`	CSS Backgrounds 4, `background-repeat-inline`
`background-repeat-x`	Sets how each background image repeats on the physical horizontal axis.	`repeat`, `space`, `round`, or `no-repeat`; comma-separated list	`repeat`	CSS Backgrounds 4, `background-repeat-x`
`background-repeat-y`	Sets how each background image repeats on the physical vertical axis.	`repeat`, `space`, `round`, or `no-repeat`; comma-separated list	`repeat`	CSS Backgrounds 4, `background-repeat-y`

Name	Description	Values	Default	Specification
`background-size`	Sets the size of each background image.	One or two values, each `auto` or a non-negative length or percentage; or `cover` or `contain`; comma-separated list	`auto`	CSS Backgrounds 4, `background-size`
`background-tbd`	Sets background layer properties without setting the background color.	One or more background layers	See individual properties	CSS Backgrounds 4, `background-tbd`
`isolation`	Controls whether an element forms an isolated group for blending.	`auto` or `isolate`	`auto`	Compositing and Blending 1, `isolation`
`mix-blend-mode`	Sets how an element blends with the content behind it.	`normal`, `darken`, `multiply`, `color-burn`, `lighten`, `screen`, `color-dodge`, `overlay`, `soft-light`, `hard-light`, `difference`, `exclusion`, hue, `saturation`, `color`, or `luminosity`	`normal`	Compositing and Blending 1, `mix-blend-mode`

- `<color>` means a CSS color, such as navy, `#336699`, or `rgb(51 102 153)`.
- `<image>` means none, an image such as `url("paper.png")`, or a gradient such as `linear-gradient(white, gray)`.

Display and positioning

The `display` property determines the kind of box an element generates. Positioning properties describe how a box is placed relative to its normal position or containing block.

Name	Description	Values	Default	Specification
`display`	Selects whether an element generates a box and describes its outer	none, `contents`, or combinations of outer types (`block`, `inline`, `run-in`), inner types		CSS Display 4 §2

Name	Description	Values	Default	Specification
	and inner display types.	(`flow`, `flow-root`, `table`, `flex`, `grid`, `ruby`), and `list-item`; also internal table and ruby types, legacy combinations such as `inline-block`, `inline-flex`, and `inline-grid`, plus `math`, `grid-lanes`, and `inline-grid-lanes`		
`visibility`	Controls whether a box is visible while preserving its place in layout.	`visible`, `hidden`, `force-hidden`, or `collapse`		CSS Display 4 §5
`order`	Changes the order of flex and grid items for layout and painting.	An integer		CSS Display 4 §3
`reading-flow`	Chooses the reading order of children in a block, flex, or grid container.	`normal`, `source-order`, `flex-visual`, `flex-flow`, `grid-rows`, `grid-columns`, or `grid-order`		CSS Display 4 §4
`reading-order`	Assigns an integer ordering value to a direct child of a reading-flow container.	An integer		CSS Display 4 §4.1
`position`	Chooses the positioning scheme for an element.	`static`, `relative`, `absolute`, `sticky`, `fixed`, or `running()`		CSS Position 3 §2; CSS GCPM 3 §1.2.1
`top`, `right`, `bottom`, `left`	Set an inset from the corresponding side of the containing block for a positioned element.	`auto`, a length or percentage, `anchor()`, or `anchor-size()`		CSS Position 3 §3.1
`inset`	Sets the four physical insets.	One to four values accepted by `top`, `right`, `bottom`, and `left`		CSS Position 3 §3.2

Name	Description	Values	Default	Specification
`inset-block`, `inset-inline`	Set the start and end insets along the block or inline axis.	One or two values accepted by the corresponding logical inset properties		CSS Position 3 §3.2
`inset-block-start`, `inset-block-end`, `inset-inline-start`, `inset-inline-end`	Set one inset on the block or inline axis.	`auto`, a length or percentage, `anchor()`, or `anchor-size()`		CSS Position 3 §3.1
`z-index`	Sets a positioned box's stack level.	`auto` or an integer		CSS 2 §9.9
`float`	Takes a box out of the normal flow and places it at the side, at the top or at the bottom of its container. The value `footnote` makes the element a footnote.	`none`, `left`, `right`, `top`, `bottom`, `block-start`, `block-end`, `inline-start`, `inline-end`, `snap-block`, `snap-inline` or `footnote`	`none`	CSS Page Floats 3 §3.2

- A length is a size with a unit, such as `12pt` or `1em`. A percentage is relative to the containing block, as defined for the relevant inset.
- `anchor()` uses an anchor's side or center as an inset coordinate. `anchor-size()` uses a selected anchor's size. See CSS Anchor Positioning 1 and the `anchor-size()` function.
- `running()` names an element to move into a page margin box. See CSS GCPM 3.
- An integer is a whole number without a fractional part. Negative values, zero, and positive values are allowed for `order` and `z-index`; CSS Display defines how ordering is applied.

Page reference

Name	Description	Values	Default	Specification
`page`	Assigns a named page type to a box.	`auto` or a custom identifier	`auto`	CSS Paged Media 3, page
`@page`	Defines page-box declarations and may contain page-margin rules.	Optional comma-separated page selectors		CSS Paged Media 3, @page

Name	Description	Values	Default	Specification
`size`	Sets page-box size and orientation. `auto` uses A4.	`auto`; one or two lengths; or A3, A4, A5, B4, B5, `JIS-B4`, `JIS-B5`, `ledger`, `legal`, or `letter`, optionally combined with `portrait` or `landscape`	A4	CSS Paged Media 3, `size`
`bleed`	Sets the bleed extension beyond the page box. `auto` uses `6pt` with crop marks and `0` otherwise.	`auto` or a length	`0` (`auto`)	CSS Paged Media 3, `bleed`
`marks`	Requests crop marks, registration marks, or both outside the page box.	none, `crop`, `cross`, or both `crop` and `cross`	none	CSS Paged Media 3, `marks`
`page-orientation`	Rotates the laid-out page in the output.	`upright`, `rotate-left`, or `rotate-right`	`upright`	CSS Paged Media 3, `page-orientation`
`@top-left-corner`	Defines the fixed-size box at the intersection of the top and left page margins.			CSS Paged Media 3, `@top-left-corner`
`@top-left`	Defines the box between the top-left corner and top-center boxes.			CSS Paged Media 3, `@top-left`
`@top-center`	Defines the centered box in the top page margin.			CSS Paged Media 3, `@top-center`
`@top-right`	Defines the box between the top-center and top-right corner boxes.			CSS Paged Media 3, `@top-right`

Name	Description	Values	Default	Specification
`@top-right-corner`	Defines the fixed-size box at the intersection of the top and right page margins.			CSS Paged Media 3, `@top-right-corner`
`@right-top`	Defines the box between the top-right corner and right-middle boxes.			CSS Paged Media 3, `@right-top`
`@right-middle`	Defines the centered box in the right page margin.			CSS Paged Media 3, `@right-middle`
`@right-bottom`	Defines the box between the right-middle and bottom-right corner boxes.			CSS Paged Media 3, `@right-bottom`
`@bottom-right-corner`	Defines the fixed-size box at the intersection of the bottom and right page margins.			CSS Paged Media 3, `@bottom-right-corner`
`@bottom-right`	Defines the box between the bottom-center and bottom-right corner boxes.			CSS Paged Media 3, `@bottom-right`
`@bottom-center`	Defines the centered box in the bottom page margin.			CSS Paged Media 3, `@bottom-center`
`@bottom-left`	Defines the box between the bottom-left corner and bottom-center boxes.			CSS Paged Media 3, `@bottom-left`
`@bottom-left-corner`	Defines the fixed-size box at the intersection of the bottom and left page margins.			CSS Paged Media 3, `@bottom-left-corner`

Name	Description	Values	Default	Specification
`@left-bottom`	Defines the box between the left-middle and bottom-left corner boxes.			CSS Paged Media 3, `@left-bottom`
`@left-middle`	Defines the centered box in the left page margin.			CSS Paged Media 3, `@left-middle`
`@left-top`	Defines the box between the top-left corner and left-middle boxes.			CSS Paged Media 3, `@left-top`
`@footnote`	Styles the footnote area at the bottom of the page. It goes inside an @page rule.	A block of declarations, such as borders, margins, padding and `max-height`.		CSS GCPM 3 §2.4

Declarations in page-margin rules

Each page-margin at-rule contains declarations written as `property: value;`. Use properties that apply to page-margin boxes, including `content`, font and text properties, color, backgrounds, borders, margins, padding, dimensions and `vertical-align`. See the CSS Paged Media page-margin property list.

For example, set a running title in the top-center box and a page number in the bottom-center box:

```
@page {
    @top-center {
        content: "Annual report";
        font-weight: bold;
    }

    @bottom-center {
        content: "Page " counter(page);
        text-align: center;
    }
}
```

Links and generated content reference

The values list the syntax defined by the cited specifications.

Name	Description	Values	Default	Specification
`content`	Replaces an element's contents or supplies generated content for pseudo-elements and page margin boxes.	`normal`, none, an image that replaces the contents, a list of generated items, or `element()`; a list can include alternative text after `/`		CSS Content 3 §1
`bookmark-label`	Sets the text for a generated bookmark.	A list of generated content items		CSS Content 3 §3.2
`bookmark-level`	Sets a bookmark's hierarchy level, or disables its creation.	none or an integer of 1 or greater		CSS Content 3 §3.1
`bookmark-state`	Sets whether a generated bookmark starts open or closed.	open or `closed`		CSS Content 3 §3.3
`quotes`	Selects the quotation marks used by generated `open-quote` and `close-quote` content.	`auto`, none, `match-parent`, or one or more pairs of quoted strings		CSS Content 3 §2.4.1
`string-set`	Assigns named values from an element for retrieval in page content.	none or comma-separated pairs of a name and one or more strings		CSS Content 3 §2.7.1
`string()`	Gives the value of a named string saved with `string-set`. Use it in the `content` property of a page-margin box.	A name, and an optional mode: `first`, `start`, `last` or `first-except`	`first`	CSS GCPM 3 §1.1.2
`content()`	Gives text of an element. Use it in the value of `string-set`.	`text`, `before`, `after` or `first-letter`	`text`	CSS GCPM 3 §1.1.1.1
`-cssmedia-link-counter()`	Gives the value of a counter at the destination of the link of the element, such as the number of a footnote that the link points to. CSSMedia extension.	The name of a counter, and an optional counter style.		CSSMedia extension

Name	Description	Values	Default	Specification
`-cssmedia-link`	Makes an XML element a link to another part of the document or to an external address. The values are joined in order to make the destination.	none, or one or more strings or `url()` values	none	CSSMedia extension
`link-parameters`	Supplies parameters for links to external resources.	none or a comma-separated list of `param()` values		CSS Link Parameters 1 §2.1

- A `<content-list>` is one or more generated content items, such as strings, quote keywords, counters, images, attribute values, or `content()` values. See CSS Content 3 and Named strings.
- A `<content-replacement>` is one image that replaces the element's normal contents. See CSS Content 3.
- The image form of `element()` uses an element ID to produce an image; the running-element form selects a named element for use in page margin boxes. See CSS Images 4 and CSS GCPM 3.
- A positive `<integer>` for `bookmark-level` is a whole number of at least 1. The top level is 1.
- A `<string>` is text between quotes. `url()` and `src()` provide URL values.
- A `<custom-ident>` is an author-chosen identifier, not a CSS-wide keyword or none.
- `param()` takes a dashed parameter name beginning with `--` and an optional `<declaration-value>`. If omitted, the parameter value is empty. A `<declaration-value>` is the sequence of CSS component values after a declaration's colon, excluding top-level semicolons and unmatched closing braces. See CSS Link Parameters 1 and CSS Syntax 3.

For examples of destinations, named strings, and generated content, see Links and Named strings.

Counters, list markers, and footnotes

Name	Description	Values	Default	Specification
`counter-reset`	Creates a new counter instance and initializes it.	none or one or more counter names, each optionally followed by an integer; the default value for each name is `0`		CSS Lists 3 §4.1

Name	Description	Values	Default	Specification
`counter-increment`	Increases each named counter, creating it if needed.	none or one or more counter names, each optionally followed by an integer; the default increment is `1`		CSS Lists 3 §4.2
`counter-set`	Sets the value of existing counters, creating a counter if it does not exist.	none or one or more counter names, each optionally followed by an integer; the default value is `0`		CSS Lists 3 §4.3
`list-style`	Sets the marker's position, image, and type.	Any order of `inside` or `outside`, an image or none, and a counter style, string, or none; any component can be omitted		CSS Lists 3 §3.6
`list-style-image`	Uses an image as the list marker.	An image or none		CSS Lists 3 §3.1
`list-style-position`	Places the marker inside the list item's contents or outside its principal box.	`inside` or `outside`		CSS Lists 3 §3.5
`list-style-type`	Chooses the marker's counter style or a literal marker string.	A counter-style name, a string, or none		CSS Lists 3 §3.4
`marker-side`	Chooses whether an outside marker's position follows the list item's directionality or its parent's.	`match-self` or `match-parent`		CSS Lists 3 §3.7
`@counter-style`	Defines a named counter style for markers and counter functions.	A name followed by counter-style descriptors		CSS Counter Styles 3 §3

Name	Description	Values	Default	Specification
`system` descriptor	Selects the algorithm that converts counter values to symbols.	`cyclic`, `numeric`, `alphabetic`, `symbolic`, `additive`, `fixed` with an optional starting integer, or `extends` with another counter-style name		CSS Counter Styles 3 §3.1
`symbols` descriptor	Lists the symbols used by the cyclic, fixed, symbolic, alphabetic, or numeric systems.	One or more symbols		CSS Counter Styles 3 §3.2
`additive-symbols` descriptor	Maps descending counter weights to symbols for the additive system.	Comma-separated pairs of a non-negative integer and a symbol, in descending weight order		CSS Counter Styles 3 §3.3
`negative` descriptor	Adds symbols before and optionally after a negative counter representation.	One or two symbols		CSS Counter Styles 3 §3.4
`prefix` descriptor	Adds a symbol before each marker.	One symbol		CSS Counter Styles 3 §3.7
`suffix` descriptor	Adds a symbol after each marker.	One symbol		CSS Counter Styles 3 §3.7
`range` descriptor	Limits the values that the counter style can represent before fallback is used.	`auto` or comma-separated pairs of integer bounds or `infinite`		CSS Counter Styles 3 §3.5
`pad` descriptor	Pads short representations to a minimum length with a chosen symbol.	A non-negative integer and a symbol		CSS Counter Styles 3 §3.6
`fallback` descriptor	Names the counter style used when this style cannot represent a value.	A counter-style name		CSS Counter Styles 3 §3.8

Name	Description	Values	Default	Specification
`footnote` counter	The counter that numbers the footnotes. Every footnote adds 1 to it. Change it with `counter-reset`, `counter-set` and `counter-increment`, in an element rule to number by section or in an `@page` rule to number by page.	Show it with `counter(footnote)`, with an optional counter style.	0 at the start of the document.	CSS GCPM 3 §2.5
`footnote-display`	Sets how a footnote body appears in the footnote area.	`block`, `inline`, or `compact`	`block`	CSS GCPM 3 §2.3
`footnote-policy`	Sets what happens when a footnote does not fit on the page of its call. `auto` continues it on the next pages, `line` moves the line with the call to the next page, and `block` moves the paragraph.	`auto`, `line`, or `block`	`auto`	CSS GCPM 3 §2.8

- A `<counter-name>` is an identifier naming a CSS counter. none and CSS-wide keywords are not counter names. See CSS Lists 3.
- An `<integer>` is a whole number without a fractional part. In the `range` descriptor, `infinite` means there is no bound in that direction. See CSS Values 4.
- An `<image>` is a URL, image function, or gradient that can provide image data. See CSS Images 3.
- A `<counter-style>` is a predefined counter style name, a name defined by `@counter-style`, or a `symbols()` function. See CSS Counter Styles 3.
- A `<counter-style-name>` is the identifier naming a predefined or author-defined counter style. See CSS Counter Styles 3.
- A `<symbol>` is a string, an identifier whose characters form the symbol, or an image. See CSS Counter Styles 3.

For example, this rule defines a two-symbol cycle. Replace `item` with an element name in your XML. The `display` declaration makes those elements list items, and their markers use the defined style.

```
@counter-style stars {
    system: cyclic;
    symbols: "★" "□";
    suffix: " ";
}

item {
    display: list-item;
    list-style-type: stars;
}
```

CSSMedia applies counter operations and prepares text marker content while building the box tree.

Footnotes

This reference lists the properties, the rule, the counter and the pseudo-elements that CSSMedia uses for footnotes.

Properties, rule and counter

Name	Description	Values	Default	Specification
`float`	Takes a box out of the normal flow and places it at the side, at the top or at the bottom of its container. The value `footnote` makes the element a footnote.	`none`, `left`, `right`, `top`, `bottom`, `block-start`, `block-end`, `inline-start`, `inline-end`, `snap-block`, `snap-inline` or `footnote`	`none`	CSS Page Floats 3 §3.2
`footnote-display`	Sets how a footnote body appears in the footnote area.	`block`, `inline`, or `compact`	`block`	CSS GCPM 3 §2.3
`footnote-policy`	Sets what happens when a footnote does not fit on the page of its call. `auto` continues it on the next pages, `line` moves the line with the call to the next page, and `block` moves the paragraph.	`auto`, `line`, or `block`	`auto`	CSS GCPM 3 §2.8

Name	Description	Values	Default	Specification
`footnote` counter	The counter that numbers the footnotes. Every footnote adds 1 to it. Change it with `counter-reset`, `counter-set` and `counter-increment`, in an element rule to number by section or in an `@page` rule to number by page.	Show it with `counter(footnote)`, with an optional counter style.	0 at the start of the document.	CSS GCPM 3 §2.5
`@footnote`	Styles the footnote area at the bottom of the page. It goes inside an `@page` rule.	A block of declarations, such as borders, margins, padding and `max-height`.		CSS GCPM 3 §2.4

Pseudo-elements

Name	Description	Values	In a PDF	Specification
`::footnote-call`	The reference that replaces a footnote in the text. By default it is the value of the `footnote` counter, as a superscript.		Matches	CSS GCPM 3 §2.6
`::footnote-marker`	The marker at the start of a footnote body. By default it is the value of the `footnote` counter, followed by a period and a space.		Matches	CSS GCPM 3 §2.7

Links

This reference lists the property, the function and the pseudo-classes that CSSMedia uses for links.

Property and function

Name	Description	Values	Default	Specification
`-cssmedia-link`	Makes an XML element a link to another part of the document or to an external address. The values are joined in order to make the destination.	none, or one or more strings or `url()` values	none	CSSMedia extension
`-cssmedia-link-counter()`	Gives the value of a counter at the destination of the link of the element, such as the number of a footnote that the link points to. CSSMedia extension.	The name of a counter, and an optional counter style.		CSSMedia extension

Pseudo-classes

Name	Description	Values	In a PDF	Specification
`:link`, `:any-link`	Elements that are hyperlinks, because `-cssmedia-link` gives them a destination.		Matches	Selectors 4 §8.1, §8.2
`:local-link`	Hyperlinks that point to the same document. The optional number limits how many path segments are compared.	An optional non-negative integer in parentheses.	Matches	Selectors 5 §2.1
`:visited`, `:target`	Links that were visited, and the element that a link points to. A PDF has no browsing history and no current target.		Never matches	Selectors 4 §8.2, §8.3

Named strings

This reference lists the property and the functions that CSSMedia uses for named strings.

Name	Description	Values	Default	Specification
`string-set`	Assigns named values from an element for retrieval in page content.	none or comma-separated pairs of a name and one or more strings		CSS Content 3 §2.7.1
`content()`	Gives text of an element. Use it in the value of `string-set`.	`text`, `before`, `after` or `first-letter`	`text`	CSS GCPM 3 §1.1.1.1
`string()`	Gives the value of a named string saved with `string-set`. Use it in the `content` property of a page-margin box.	A name, and an optional mode: `first`, `start`, `last` or `first-except`	`first`	CSS GCPM 3 §1.1.2

Cascade reference

The `Default` column lists values used by CSSMedia's current PDF-oriented rule evaluation. Media-query width, height and ratio values use the A4 default page size.

Property and at-rule reference

Name	Description	Values	Default	Specification
`all`	Resets all properties except `direction` and `unicode-bidi`.	`initial` | `inherit` | `unset` | `revert` | `revert-layer`		CSS Cascade 5 §3.2
`@charset`	Declares the stylesheet's fallback character encoding.	`@charset "<label>";`		CSS Syntax 3 §8.3
`@import`	Loads another stylesheet, optionally conditioned on a layer, support test or media query.	`@import <url-or-string> [layer | layer(<layer-name>)]? [supports(<condition>)]? <media-query-list>?;`		CSS Cascade 5 §2

Name	Description	Values	Default	Specification
`@namespace`	Declares a namespace prefix for selectors.	`@namespace <namespace-prefix>? <string-or-url>;`		CSS Namespaces 3 §3
`@layer`	Declares layer order or groups rules in a cascade layer.	`@layer <layer-name>? { <stylesheet> }` or `@layer <layer-name-list>;`		CSS Cascade 5 §6.4
`@property`	Registers a custom property with syntax, inheritance and optional initial value.	`@property <custom-property-name> { <declaration-list> }`		CSS Properties and Values API 1 §3
`syntax`	Defines the grammar of a registered custom property.	`<string>`		CSS Properties and Values API 1 §3.1
`inherits`	Sets whether a registered custom property inherits by default.	`true` | `false`		CSS Properties and Values API 1 §3.2
`initial-value`	Sets the registered custom property's initial value.	`<declaration-value>`	Guaranteed-invalid for `syntax: "*"`; otherwise required	CSS Properties and Values API 1 §3.3
`@media`	Includes rules when a media query matches the PDF environment.	`@media <media-query-list> { <rule-list> }`		CSS Conditional 3 §5

Name	Description	Values	Default	Specification
`@supports`	Includes rules when CSSMedia accepts a declaration, selector or supported feature test.	`@supports <supports-condition> { <rule-list> }`		CSS Conditional 3 §6
`@supports-condition`	Defines a named group of support tests.	`@supports-condition <supports-condition-name> { <block-contents> }`		CSS Conditional 5 §8
`@container`	Includes rules when a selected ancestor container passes the query.	`@container <container-condition># { <block-contents> }`		CSS Conditional 5 §5.4
`@scope`	Limits the selectors in a rule list to a scope root and optional scope limit.	`@scope [(<scope-start>)]? [to (<scope-end>)]? { <rule-list> }`		CSS Cascade 6 §2.5
`@custom-media`	Gives a name to a media query list or boolean value.	`@custom-media <extension-name> [<media-query-list> | true | false];`		Media Queries 5 §10
`@when`	Includes rules when a generalized boolean condition is true.	`@when <boolean-condition> { <rule-list> }`		CSS Conditional 5 §3
`@else`	Supplies the next branch of a conditional chain.	`@else <boolean-condition>? { <rule-list> }`		CSS Conditional 5 §4

Name	Description	Values	Default	Specification
`@function`	Defines a custom CSS function.	`@function <name>(<parameters>) [returns <type>]? { <declaration-rule-list> }`		CSS Functions and Mixins 1 §2.1
`result`	Supplies a custom function's return value.	`<declaration-value>?`		CSS Functions and Mixins 1 §2.2

- ? means optional, | means choose one, # means one or more comma-separated items, || means one or more options in any order, and brackets group parts of the syntax.
- `<label>` is an encoding name.
- `<url-or-string>` and `<string-or-url>` each mean a URL or quoted string.
- `<namespace-prefix>` is the prefix bound to a namespace.
- `<layer-name>` names a cascade layer. `<layer-name-list>` is a comma-separated list of layer names.
- `<stylesheet>` is a CSS stylesheet.
- `<custom-property-name>` is a custom property name beginning with `--`.
- `<declaration-list>` is a list of declarations.
- `<block-contents>` is content inside a rule's braces, including declarations and nested rules.
- `<declaration-value>` is the value part of a declaration: one or more CSS tokens, such as `navy`, `calc(1em + 2px)` or `var(--size)`. It cannot contain a top-level semicolon or `!`, since those delimit declarations and `!important`. Semicolons and exclamation marks inside functions are not top-level. See CSS Syntax 3.
- `<string>` is a quoted CSS string.
- `<media-query-list>` is one or more comma-separated media queries.
- `<rule-list>` is a sequence of rules.
- `<condition>` and `<supports-condition>` are support conditions made from feature tests and boolean operators.
- `<supports-condition-name>` names a reusable support condition.
- `<container-condition>` optionally names a query container and gives its query.
- `<boolean-condition>` combines `media()` and `supports()` tests.
- `<scope-start>` and `<scope-end>` are selector lists that set the scope roots and limits of `@scope`.
- `<extension-name>` names a custom media query.
- `<name>` in the function syntax is a function name beginning with `--`.
- `<parameters>` is a comma-separated list of function parameters.
- `<type>` is the optional syntax expected for the function result.

- `<declaration-rule-list>` contains declarations and nested rules.
- `<number>` in the function example is a CSS number, which can be an integer or include a fractional part.

The `@charset` rule must be the first rule in a stylesheet to affect byte decoding. Use `@namespace` before ordinary rules, and place `@import` rules before namespace declarations and ordinary rules.

Function names start with `--`. Parameters can have a syntax type and a default value. Use the `result` descriptor for the returned value:

```
@function --double(--value <number>) returns <number> {
    result: calc(var(--value) * 2);
}
```

`@supports` declaration tests use parentheses. `selector()` tests one complex selector. `font-tech()`, `font-format()` and `at-rule()` test supported features. A named condition is tested by placing its name inside parentheses:

```
@supports (color: navy) and selector(chapter > title) {
    chapter > title { color: navy; }
}

@supports-condition --has-navy {
    color: navy;
}

@supports (--has-navy) {
    chapter { color: navy; }
}
```

`@when` conditions combine `media()` and `supports()` tests with `and`, `or` and `not`. Parentheses group mixed operators:

```
@when media(width >= 6in) and supports(color: navy) {
    chapter { color: navy; }
}
```

Use `@else` immediately after an `@when`, `@media`, `@supports` or `@container` branch. An unrelated rule ends the chain.

Media features

Media feature values describe the page or PDF environment used for conditional rule evaluation. They do not describe a screen.

Name	Description	Values	Default	Specification
`any-hover`	Whether any pointing device can hover.	`none` | `hover`	`none`	Media Queries 5
`any-pointer`	The accuracy of any available pointing device.	`none` | `coarse` | `fine`	`none`	Media Queries 5

Name	Description	Values	Default	Specification
`aspect-ratio`	The ratio of page width to page height.	`<ratio>`	A4 page ratio	Media Queries 5
`color`	Bits per color component in the output.	`<integer>`	`8`	Media Queries 5
`color-gamut`	The approximate output color gamut.	`srgb` | `p3` | `rec2020`	`srgb`	Media Queries 5
`color-index`	Number of entries in the output color lookup table.	`<integer>`	`0`	Media Queries 5
`device-aspect-ratio`	The ratio of output width to output height.	`<ratio>`	A4 page ratio	Media Queries 5
`device-height`	Output height.	`<length>`	A4 page height	Media Queries 5
`device-width`	Output width.	`<length>`	A4 page width	Media Queries 5
`display-mode`	The presentation mode.	`fullscreen` | `standalone` | `minimal-ui` | `browser` | `picture-in-picture`	Not set	Media Queries 5
`dynamic-range`	The output's dynamic range.	`standard` | `high`	`standard`	Media Queries 5
`environment-blending`	Whether output blends with the surrounding environment.	`opaque` | `additive` | `subtractive`	`opaque`	Media Queries 5
`forced-colors`	Whether the output forces a limited color palette.	`none` | `active`	`none`	Media Queries 5
`grid`	Whether the output is grid-based.	`<mq-boolean>` (`0` or `1`)	`0`	Media Queries 5

Name	Description	Values	Default	Specification
`height`	Page height.	`<length>`	A4 page height	Media Queries 5
`horizontal-viewport-segments`	Number of horizontal viewport segments.	`<integer>`	`1`	Media Queries 5
`hover`	Whether the primary pointing device can hover.	`none` | `hover`	`none`	Media Queries 5
`inverted-colors`	Whether the output inverts colors.	`none` | `inverted`	`none`	Media Queries 5
`monochrome`	Number of bits per pixel in a monochrome output.	`<integer>`	`0`	Media Queries 5
`nav-controls`	Availability of navigation controls.	`none` | `back`	`none`	Media Queries 5
`orientation`	Whether the page is portrait or landscape.	`portrait` | `landscape`	`portrait`	Media Queries 5
`overflow-block`	How the output handles block-axis overflow.	`none` | `scroll` | `paged`	`paged`	Media Queries 5
`overflow-inline`	How the output handles inline-axis overflow.	`none` | `scroll`	`none`	Media Queries 5
`pointer`	The accuracy of the primary pointing device.	`none` | `coarse` | `fine`	`none`	Media Queries 5
`prefers-color-scheme`	The preferred color scheme.	`light` | `dark`	`light`	Media Queries 5
`prefers-contrast`	The preferred contrast level.	`no-preference` | `less` | `more` | `custom`	`no-preference`	Media Queries 5
`prefers-reduced-data`	Whether the user prefers reduced data use.	`no-preference` | `reduce`	`no-preference`	Media Queries 5

Name	Description	Values	Default	Specification
`prefers-reduced-motion`	Whether the user prefers reduced motion.	`no-preference` | `reduce`	`no-preference`	Media Queries 5
`prefers-reduced-transparency`	Whether the user prefers reduced transparency.	`no-preference` | `reduce`	`no-preference`	Media Queries 5
`resolution`	Output pixel density.	`<resolution>` | `infinite`	`infinite`	Media Queries 5
`scan`	The output's scanning process.	`interlace` | `progressive`	Not set	Media Queries 5
`scripting`	Whether scripting is available.	`none` | `initial-only` | `enabled`	`none`	Media Queries 5
`update`	How frequently the output can update.	`none` | `slow` | `fast`	`none`	Media Queries 5
`vertical-viewport-segments`	Number of vertical viewport segments.	`<integer>`	`1`	Media Queries 5
`video-color-gamut`	The color gamut of the output video.	`srgb` | `p3` | `rec2020`	`srgb`	Media Queries 5
`video-dynamic-range`	The dynamic range of the output video.	`standard` | `high`	`standard`	Media Queries 5
`width`	Page width.	`<length>`	A4 page width	Media Queries 5

- `<ratio>` is a ratio of two numbers.
- `<integer>` is a whole number.
- `<length>` is a CSS length, such as `12pt` or `2em`.
- `<mq-boolean>` is an integer with the value `0` or `1`.
- `<resolution>` is a pixel density, such as `300dpi`.

Container style queries

A container style query uses `style()` to test a computed style or custom property on an ancestor. See Conditional rules.

Licenses

CSSMedia

- Available from https://github.com/maxprograms-com/CSSMedia
- Published under Business Source License 1.1

Adobe Core 14 AFM files

- Available from https://download.macromedia.com/pub/developer/opentype/tech-notes/Core14_AFMs.zip
- Published under a permissive license, the text of which is the file `MustRead.html` inside the zip. The files may be used, copied and distributed for any purpose and without charge, with or without modification, provided that the copyright notices are retained, the AFM files are not distributed without `MustRead.html`, and modifications are noted

Adobe Glyph List and ITC Zapf Dingbats Glyph List

- Available from https://github.com/adobe-type-tools/agl-aglfn
- Published under BSD 3-Clause License

css-tree

- Available from https://github.com/csstree/csstree
- Published under MIT License

Noto Fonts

- Available from https://fonts.google.com/noto
- Published under SIL Open Font License 1.1

pdfnative

- Available from https://github.com/Nizoka/pdfnative
- Published under MIT License

TeX hyphenation patterns

- Available from https://github.com/hyphenation/tex-hyphen
- Published under LPPL, MIT, MPL or other permissive licenses

TypesBCP47

- Available from https://github.com/maxprograms-com/TypesBCP47
- Published under Eclipse Public License v1.0

TypesXML

- Available from https://github.com/maxprograms-com/TypesXML
- Published under Eclipse Public License v1.0

Unicode Character Database

- Available from https://www.unicode.org/Public/UCD/latest/ucd/
- Published under Unicode License v3

Unicode CLDR

- Available from https://github.com/unicode-org/cldr-json

- Published under Unicode License v3

Universal Shaping Engine data

- Available from https://github.com/microsoft/font-tools/tree/main/USE
- Published under MIT License

W3C CSS Counter Styles Level 3

- Available from https://www.w3.org/TR/css-counter-styles-3/
- Published under W3C Software and Document License

W3C webref

- Available from https://github.com/w3c/webref
- Published under MIT License

Glossary

CSS

Cascading Style Sheets (CSS) is the language, defined by the W3C, that describes how elements of a document are presented. A stylesheet is a list of rules. Each rule has a selector that chooses elements and declarations that set their properties. CSSMedia reads a CSS file to decide how to format an XML document in a PDF.

DTD

A Document Type Definition (DTD) is a grammar, defined by the XML specification, that declares the elements and attributes an XML document may use. A document points to its DTD in its document type declaration. When the DTD is not at the location the document names, an OASIS Catalog tells CSSMedia where to find it.

MathML

Mathematical Markup Language (MathML) is a W3C XML language for writing mathematical notation. A MathML formula describes the structure of an expression, such as fractions, roots and subscripts. CSSMedia draws MathML found in a document as an image.

Namespace

An XML namespace is a URI that identifies the vocabulary an element or attribute belongs to. An XML document declares it with an `xmlns` attribute, so elements with the same name in different vocabularies do not collide.

OASIS Catalog

An OASIS Catalog is an XML file, defined by the OASIS XML Catalogs standard, that maps the identifiers used in documents to the locations where the files are stored. A catalog lets a parser find a DTD, an XML Schema or a RelaxNG schema without downloading it. Pass a catalog to CSSMedia with the `-g` option.

PDF

Portable Document Format (PDF) is the file format, standardized as ISO 32000, for documents that look the same on any device. A PDF holds pages with text, fonts, images and vector graphics at fixed positions, and may also hold links and bookmarks. CSSMedia writes a PDF file from an XML file and a CSS file.

RelaxNG

RelaxNG is a schema language for XML, standardized as ISO/IEC 19757-2. A RelaxNG schema describes the structure of a document in an XML syntax or in a compact non-XML syntax. It is an alternative to a DTD and to an XML Schema, and it can be located with an OASIS Catalog.

SVG

Scalable Vector Graphics (SVG) is a W3C format for two-dimensional vector graphics, written as XML. An SVG drawing describes shapes, paths, text and images that can be scaled without losing quality. CSSMedia draws SVG found in a document as an image.

XML

Extensible Markup Language (XML) is a W3C standard for writing structured documents. An XML document is a tree of elements, with attributes and text, whose names are chosen by the author or by a vocabulary such as DITA or DocBook. CSSMedia reads an XML file as the content of the PDF.

XML Schema

XML Schema is a W3C language for describing the structure of XML documents and the data types of their content. A schema is itself an XML file, and it is more expressive than a DTD. Like a DTD, a schema can be located with an OASIS Catalog.